Az Ágensek Kora

Hogyan változik meg a szoftverfejlesztés — Útmutató projekt menedzsereknek

2026. február

Tartalomjegyzék

1 Mi történt 2024 végén? 5
1.1 Az AI, ami kódot ír . . . 5
1.2 A fejlődés kronológiája . . . 5
1.3 A SWE-bench — amikor az AI levizsgázott . . . 6
1.4 A „tool use” paradigma — az igazi áttörés . . . 6
1.5 Mit tud ma az AI — és mit nem? . . . 7
1.5.1 Amit már jól csinál . . . 7
1.5.2 Ahol még gyenge . . . 8
1.5.3 Az arany középút . . . 8
1.6 Miért fontos ez egy PM-nek? . . . 8

2 Claude Code – Az új fejlesztői eszköz 10
2.1 Mi az a Claude Code? . . . 10
2.2 Hol fut? . . . 10
2.3 Az agentic loop – hogyan gondolkodik . . . 11
2.4 Az eszközök – mivel dolgozik . . . 12
2.5 Az engedélyezési rendszer – ki mit engedhet . . . 13
2.6 CLAUDE.md – a projekt memóriája . . . 14
2.7 Hookrendszer – automatikus minőségbiztosítás . . . 16
2.8 MCP – az “USB-C” az AI alkalmazásoknak . . . 17
2.9 Subágensek – párhuzamos munkavégzés . . . 18
2.10 GitHub Actions – AI a CI/CD folyamatban . . . 20
2.11 Egy konkrét példa: bugfix a gyakorlatban . . . 21
2.12 Tippek a hatékony használathoz . . . 24
2.13 Összefoglalva . . . 24

3 Vibe Coding — a csábítás és a csapda 26
3.1 Mi az a „vibe coding”? . . . 26
3.2 A való világban: házépítés terv nélkül . . . 26
3.3 A csábítás . . . 27
3.4 Vibe coding kudarcok a valóságban . . . 28
3.5 A három alapvető probléma . . . 28
3.5.1 1. A kontextus probléma . . . 28
3.5.2 2. A minőség probléma . . . 29
3.5.3 3. A skálázás probléma . . . 30
3.6 Részletes összehasonlítás: ugyanaz a feature, két megközelítéssel . . . 31
3.6.1 A) Vibe coding megközelítés . . . 31
3.6.2 B) Spec-driven megközelítés . . . 31
3.7 A technikai adósság — amit minden PM-nek értenie kell . . . 32
3.8 A kontextus ablak probléma konkrét számokkal . . . 33
3.9 Mikor JÓ a vibe coding? . . . 33
3.10 Mikor ROSSZ a vibe coding? . . . 34
3.11 Az összehasonlítás . . . 34
3.12 Miért számít a PM láthatóság? . . . 34
3.13 Mikor válik a vibe coding spec-driven-né — a természetes fejlődés . . . 35
3.14 Hogyan segít mindez a cégednek? . . . 36
3.15 Következő lépések . . . 36

4 OpenSpec – Specifikacio-vezerelt fejlesztes 37
4.1 A problema ujra . . . 37

4.2 Az OpenSpec es az ismert PM eszkozok – forditorozsetta . . . 37
4.3 A pipeline – az otlettol a kodig . . . 38
4.4 1. Proposal – “Mit es miert?” . . . 39
4.4.1 Miert fontos a “miert?” . . . 39
4.5 2. Specs – “Pontosan mit kell tudnia?” . . . 40
4.6 3. Design – “Hogyan epitjuk meg?” . . . 42
4.7 4. Tasks – “Mi a teendo lista?” . . . 43
4.8 A review gate – a PM mint minosegkapu . . . 44
4.9 A PM szerepe az OpenSpec munkafolyamatban . . . 45
4.10 Egy PM napja az OpenSpec-kel – gyakorlati peldak . . . 46
4.11 Hogyan nez ki ez a gyakorlatban? . . . 47
4.12 Visszakovethetos – a proposal-tol a kodig . . . 48
4.13 Mi van, ha a spec hibas? – Iteraciok es frissitesek . . . 48
4.14 Osszehasonlitas: Waterfall vs Agile vs OpenSpec . . . 49
4.15 Az OpenSpec es a Shape Up metodologia . . . 50
4.16 Az OpenSpec es a hagyomanyos PM eszkozok . . . 51
4.17 Tovabbolvasas . . . 51

5 Párhuzamos AI ágensek – Orchestráció 53
5.1 Az ötlet . . . 53
5.2 Git worktree-k – a párhuzamos munkaterületek . . . 53
5.2.1 Az iroda hasonlat . . . 53
5.2.2 Miért jobb ez, mint a “színpadi” párhuzamosság? . . . 54
5.3 A Ralph Loop – az autonóm munkaciklus . . . 55
5.3.1 Miért “loop”? . . . 55
5.3.2 Ralph Loop részletesen – iterációról iterációra . . . 55
5.3.3 Biztonsági mechanizmusok – mi van, ha valami rosszul sül? . . . 56
5.4 Az orchestrátor – a karmester . . . 57
5.4.1 Hogyan működik? . . . 57
5.4.2 A folyamat lépésről lépésre . . . 58
5.4.3 A függőségi gráf (DAG) – a főzési hasonlat . . . 58
5.4.4 Valós példa: egy projekt az orchestrátorral, elejétől a végéig . . . 59
5.5 Merge policy-k – mennyire legyen automatikus? . . . 60
5.5.1 A bizalmi gradiens – manual-tól eager-ig . . . 61
5.6 A GUI – valós idejű áttekintés . . . 61
5.6.1 Milyen metrikákat figyel egy PM a dashboardon? . . . 62
5.7 Hibakezelés: mi történik, ha valami rosszul megy? . . . 62
5.7.1 1. Merge conflict – összeolvasztási ütközés . . . 62
5.7.2 2. Ágens elakadás (stall) . . . 63
5.7.3 3. Teszthiba a beolvasztás után . . . 63
5.7.4 4. API rate limit vagy költség-túllépés . . . 63
5.8 Valós számok . . . 63
5.8.1 Költségbecslés: hogyan tervezz büdzsét AI ágens használatra? . . . 64
5.9 Mikor érdemes orchestrálni? . . . 64
5.9.1 Döntési fa PM-eknek . . . 64
5.10 Összefoglalás: az orchestráció mint PM szupererő . . . 65

6 Memória és tanulás 66
6.1 A probléma: az amnéziás kolléga . . . 66
6.2 A megoldás: Developer Memory . . . 66
6.2.1 Előtte / utána . . . 67
6.3 Két réteg: CLAUDE.md (statikus) és Developer Memory (dinamikus) . . . 68
6.4 A három memória típus . . . 69
6.5 Az 5 réteg — hogyan dolgozik a memória . . . 70
6.5.1 A szemantikus keresés — hogyan talál a memória . . . 71
6.6 Egy teljes nap — mikor aktiválódik a memória . . . 72
6.7 Csapat memória — gépek között . . . 73
6.7.1 Hogyan működik ez egy 5 fős csapatnál? . . . 73
6.8 Memória kontra hagyományos tudáskezelés . . . 74
6.9 Memória higiénia — karbantartás . . . 75
6.9.1 Mikor kell takarítani? . . . 75

6.9.2 Hogyan? . . . 75
6.10 Valós példák: hogyan segít a memória a gyakorlatban . . . 76
6.10.1 1. példa: Az adatbázis hiba (időmegtakarítás) . . . 76
6.10.2 2. példa: A kód stílus konzisztencia (minőség) . . . 76
6.10.3 3. példa: Az új csapattag onboardingja (gyorsaság) . . . 76
6.10.4 4. példa: A sprint közbeni döntés nyomon követése (PM érték) . . . 77
6.11 Az érték PM szemmel . . . 77
6.12 Mi NEM memória . . . 77

7 A szoftverfejlesztés jövője 79
7.1 Hol tartunk most? . . . 79
7.2 Három időhorizont . . . 79
7.2.1 Rövid táv: 2025–2026 – AI mint páros programozó . . . 80
7.2.2 Közép táv: 2026–2028 – AI csapatok emberi felügyelettel . . . 80
7.2.3 Hosszú táv: 2028+ – AI-vezérelt fejlesztés emberi irányítással . . . 81
7.3 A PM szerep evolúciója . . . 82
7.4 Iparági szereplők – részletes összehasonlítás . . . 83
7.4.1 Az eszközök részletesen . . . 83
7.4.2 Összehasonlító táblázat . . . 84
7.5 AI-natív vállalatok: az új paradigma . . . 84
7.5.1 Mi az AI-natív vállalat? . . . 84
7.5.2 Mit jelent ez a PM-nek? . . . 85
7.6 Gazdasági hatás: számok és becslések . . . 85
7.6.1 Produktivitás-növekedés . . . 85
7.6.2 Költségmegtakarítás . . . 85
7.6.3 ROI (Return on Investment) számítás PM-eknek . . . 86
7.7 Kockázatok és kihívások . . . 86
7.7.1 1. AI hallucináció . . . 86
7.7.2 2. Biztonsági kérdések . . . 86
7.7.3 3. Vendor lock-in . . . 86
7.7.4 4. Szabályozási kérdések – az EU AI Act . . . 87
7.7.5 5. Munkaerőpiaci hatás . . . 87
7.7.6 6. Minőségi és megbízhatósági kockázat . . . 88
7.8 Gyakori tévhitek az AI-ról a szoftverfejlesztésben . . . 88
7.8.1 Tévhit #1: “Az AI helyettesíti a fejlesztőket” . . . 88
7.8.2 Tévhit #2: “Az AI által generált kód rossz minőségű” . . . 88
7.8.3 Tévhit #3: “Csak programozóknak hasznos” . . . 88
7.8.4 Tévhit #4: “Drága és jó AI eszközök kellenek, olcsóbb nem használni” . . . 88
7.8.5 Tévhit #5: “Nem kell hozzá technikai tudás” . . . 88
7.8.6 Tévhit #6: “Az AI kód biztonságilag veszélyes” . . . 88
7.9 Ami NEM fog eltűnni . . . 89
7.10 Képességek, amiket egy PM-nek most érdemes fejleszteni . . . 90
7.10.1 1. Specifikáció-írás (Spec Writing) . . . 90
7.10.2 2. AI műveltség (AI Literacy) . . . 90
7.10.3 3. Prompt engineering alapok . . . 90
7.10.4 4. Minőségi kapu (Quality Gate) definiálás . . . 90
7.10.5 5. Változás-menedzsment (Change Management) . . . 91
7.11 Akcióterv: hogyan kezdj hozzá? . . . 91
7.11.1 1. hét: Ismerkedés (0 kockázat) . . . 91
7.11.2 2. hét: Első kísérlet (alacsony kockázat) . . . 91
7.11.3 3. hét: Spec-driven próbálkozás (közepes kockázat) . . . 91
7.11.4 4. hét: Értékelés és következő lépések . . . 91
7.11.5 Folyamatos tevékenységek (1. hónaptól) . . . 92
7.12 Az átmenet kezelése: csapat menedzsment az AI-korszakban . . . 92
7.12.1 “A fejlesztők félnek, hogy az AI elveszi a munkájukat” . . . 92
7.12.2 “Egyes fejlesztők lelkesek, mások szkeptikusak” . . . 92
7.12.3 “A review folyamat még nem alkalmazkodott” . . . 92
7.12.4 “Nem tudjuk hogyan becsüljünk” . . . 92
7.12.5 “A vezetőség túl nagy elvárásokat támaszt” . . . 93
7.13 Összefoglalás: mit tegyen most egy PM? . . . 93

8 Függelék 94
8.1 A. Szószedet . . . 94
8.2 B. Linkgyűjtemény . . . 100
8.2.1 Anthropic és Claude Code . . . 100
8.2.2 MCP (Model Context Protocol) . . . 101
8.2.3 Kutatás és benchmarkok . . . 101
8.2.4 Iparági eszközök . . . 101
8.2.5 Blogok és cikkek . . . 101
8.2.6 YouTube csatornák . . . 101
8.2.7 Podcastok . . . 102
8.2.8 Közösségek és tanulás . . . 102
8.2.9 Szabványok és szabályozás . . . 102
8.2.10 Iparági jelentések és kutatások . . . 102
8.3 C. “Kipróbálom” – Quick Start . . . 103
8.3.1 Ha nem akarsz semmit telepíteni . . . 103
8.3.2 Ha szeretnéd kipróbálni a Claude Code-ot . . . 103
8.3.2.1 1. lépés: Telepítés (2 perc) . . . 103
8.3.2.2 2. lépés: Bejelentkezés . . . 103
8.3.2.3 3. lépés: Próbáld ki egy projekten . . . 103
8.3.2.4 4. lépés: Első parancsok . . . 104
8.3.2.5 5. lépés: Fedezd fel . . . 104
8.3.3 Hibaelhárítás . . . 104
8.3.4 Mi legyen a következő lépés? . . . 104
8.3.5 Árazás (2025. február) . . . 105
8.4 D. Ajánlott olvasmányok . . . 105

1. fejezet

Mi történt 2024 végén?

1.1 Az AI, ami kódot ír

Képzeld el, hogy van egy új kolléga a csapatodban. Nem kell neki kávészünet, nem megy szabadságra, és ha éjjel kettőkor eszedbe jut egy ötlet, üzenetet írhatsz neki, és reggelre kész lesz az alapja. Ez a kolléga nem ember — ez egy AI ágens, ami szoftverfejlesztésre lett kiképezve.

2024 végén valami alapvetően megváltozott az AI világában. Az AI modellek elértek egy szintet, ahol nem csak kérdésekre válaszolnak, hanem **önállóan cselekedni** is tudnak: fájlokat olvasnak, kódot írnak, teszteket futtatnak, és ha valami nem működik, megpróbálják kijavítani. Ez a fordulópont az „AI-asszisztenstől" az „AI-ágensig" vezető út mérföldköve.

Ez a könyv arról szól, hogyan változtatja meg ez a fejlődés a szoftverfejlesztést — és mit jelent ez számodra, mint projekt menedzser.

> **Ajánlott olvasmány**: Az Anthropic cég (a Claude fejlesztője) honlapján részletes kutatási anyagokat találsz arról, hogyan jutottak el idáig: anthropic.com/research

1.2 A fejlődés kronológiája

Az AI nem egyik napról a másikra lett ilyen okos. Egy évtizedes kutatás eredménye, ami 2022-ben gyorsult fel drámaian. Íme a főbb mérföldkövek:

Időpont	Esemény	Jelentősége
2020 jún.	GPT-3 megjelenik	Az első nagy nyelvi modell ami összefüggő szöveget ír
2022 nov.	ChatGPT elindul	AI mindenkinek – 100 millió felhasználó 2 hónap alatt
2023 márc.	GPT-4	Programozási feladatokban már jobb, mint sok ember
2023 júl.	Claude 2	Az Anthropic belép a versenybe
2024 márc.	Claude 3 / Opus	Első modell ami komplex gondolkodásra képes
2024 jún.	Claude 3.5 Sonnet	Áttörés a kód-értésben és generálásban
2024 okt.	SWE-bench rekord	Claude valós GitHub hibák 49%-át önállóan megoldja
2024 nov.	MCP szabvány	Egységes protokoll az AI és külső eszközök között

Időpont	Esemény	Jelentősége
2025 feb.	Claude Code megjelenés	Teljes fejlesztőkörnyezetben dolgozó AI ágens
2025 máj.	Claude 4.0 / Opus	Komplex, többlépéses feladatokban is megbízható
2025 ősz	Multi-ágens rendszerek	Több AI ágens dolgozik párhuzamosan, koordináltan
2026 feb.	Claude 4.6 / Opus	Még pontosabb, gyorsabb, jobb kontextus-kezelés

A legfontosabb pillanat ebben az idővonalon **2024 ősze** volt. Ez volt az a pont, amikor az AI túllépett a „kérdezz és válaszolok" szinten, és elkezdte **önállóan megoldani** a problémákat.

> **Háttér**: Ha érdekel a nagy nyelvi modellek működése közérthetően, a 3Blue1Brown YouTube csatorna kiváló vizuális magyarázatot ad: [youtube.com/@3blue1brown](https://www.youtube.com/@3blue1brown) – keresd a „GPT" és „Transformer" videókat.

1.3 A SWE-bench — amikor az AI levizsgázott

A szoftverfejlesztő közösségnek szüksége volt egy objektív mércére: tényleg tud-e az AI igazi programozói feladatokat megoldani, vagy csak trükközik? Erre hozták létre a **SWE-bench** (Software Engineering Benchmark) nevű tesztet.

A SWE-bench nem tankönyvi feladatokat ad az AI-nak. Ehelyett **valódi hibajelentéseket** vesz elő népszerű nyílt forráskódú projektekből (mint a Django, Flask, vagy a Scikit-learn), és megkéri az AI-t, hogy javítsa ki őket. A feladatok között van egyszerű és rendkívül bonyolult is — pont úgy, ahogy a való életben.

Hogyan néz ki egy SWE-bench feladat? Például:

> *„A Django ORM `QuerySet.union()` metódusa hibásan kezeli az `order_by()` hívást unió után. A felhasználók azt jelentik, hogy az eredmények nem a várt sorrendben jönnek vissza. A hiba a `django/db/models/sql/query.py` fájlban van."*

Az AI-nak meg kell találnia a releváns fájlokat egy több ezer fájlos projektben, megértenie a hibát, kijavítania, és biztosítania hogy a meglévő tesztek továbbra is átmenjenek.

Az eredmény meglepő volt: az Anthropic SWE-bench Verified tesztjén a Claude 3.5 Sonnet modell **49%-os eredményt** ért el. Ez azt jelenti, hogy a valós, éles hibajelentések közel felét önállóan meg tudta oldani.

> **Forrás**: Anthropic SWE-bench kutatás – a teljes kutatási anyag az Anthropic honlapján olvasható.
>
> **Ajánlott olvasmány**: A SWE-bench projekt honlapja részletesen bemutatja a módszertant: swebench.com

Miért fontos ez? Mert a korábbi rekord 45% volt, és a kutatók azt gondolták, hogy még évekbe telik mire az AI eléri az 50%-ot. Ehelyett hónapok alatt történt meg. 2025 végére a legjobb eredmények már 60% felett járnak.

1.4 A „tool use" paradigma — az igazi áttörés

A SWE-bench eredmények mögött nem egyszerűen „okosabb" modellek állnak. Az igazi áttörés az volt, hogy az AI megtanult **eszközöket használni** (tool use). Ez azt jelenti, hogy az AI nem csak szöveget generál — hanem képes:

1. **Fájlokat megnyitni és olvasni** a számítógépen
2. **Parancsokat futtatni** a terminálban (tesztek, build, stb.)

3. **Fájlokat szerkeszteni** és új fájlokat létrehozni
4. **Eredményeket értelmezni** és további lépéseket tervezni

Képzeld el a különbséget:

```
RÉGI MÓD (ChatGPT, 2023):
────────────────────────

Te: "Mi a hiba ebben a kódban?" [másolod a kódot]
AI: "A hiba a 42. sorban van, cseréld ki erre: ..."
Te: [manuálisan átírod]
Te: "Most ez a hiba jön" [másolod az új hibát]
AI: "Próbáld ezt: ..."
... és így tovább, oda-vissza

ÚJ MÓD (Claude Code, 2025):
──────────────────────────

Te: "Javítsd ki a login hibát"
AI: [megnyitja a fájlokat]
    [megtalálja a hibát]
    [kijavítja]
    [lefuttatja a teszteket]
    [ha kell, tovább javít]
    "Kész, itt a megoldás. Commitoljam?"
```

Ez a különbség alapvető, mert az AI így **komplett feladatokat** tud megoldani, nem csak tanácsot adni.

> **Ajánlott cikk**: Az Anthropic blogja részletesen bemutatja a tool use koncepciót: anthropic.com/news/tool-use-ga

1.5 Mit tud ma az AI — és mit nem?

Fontos tisztán látni, mire képes ma egy AI fejlesztő ágens, és hol vannak a határai. Nézzünk konkrét példákat mindkét oldalra.

1.5.1 Amit már jól csinál

- **Kód olvasás és megértés**: Egy teljes projekt kódbázisát képes áttekinteni és megérteni a struktúráját. Odatalál a releváns fájlokhoz, érti az összefüggéseket. *Példa: „Olvasd be a teljes auth modult és magyarázd el hogyan működik a session kezelés" – és 30 másodperc alatt átfogó összefoglalót ad.*
- **Hibakeresés és javítás**: Hibaüzenetből kiindulva megkeresi a problémás kódrészt, megérti a hibát, és javítást készít. *Példa: Beillesztesz egy stack trace-t, és az AI nemcsak megtalálja a hiba gyökerét, hanem az összefüggő fájlokat is javítja.*
- **Tesztek írása és futtatása**: Automatikus teszteket ír, lefuttatja őket, és ha valami elbukik, megpróbálja kijavítani a kódot. *Példa: „Írj teszteket az email modul összes publikus függvényéhez" – és 5 perc alatt 20 teszt eset készül, lefut, és zöld.*
- **Rutin feladatok**: Kód refaktorálás, kódstílus javítás, dokumentáció generálás, dependency frissítés — ezeket gyorsan és megbízhatóan végzi. *Példa: „Alakítsd át az összes JavaScript fájlt TypeScript-re a src/ könyvtárban" – és egy óra alatt 50 fájlt migrál, típusokkal.*
- **Több fájl együttes kezelése**: Nem csak egy fájlt szerkeszt, hanem érti, hogy egy változtatás más fájlokra is hathat, és azokat is módosítja. *Példa: Ha átnevez egy függvényt, megkeresi és frissíti az összes*

helyet ahol használva van.

1.5.2 Ahol még gyenge

- **Komplex architektúrális döntések**: Az AI nem érti, hogy miért választottál mikroszervizeket monolitikus alkalmazás helyett. Nem ismeri a csapatod képességeit, a budget korlátait, vagy a hosszú távú üzleti stratégiát. *Példa: Nem fogja magától mondani, hogy „ne építsünk saját auth rendszert, használjunk Auth0-t, mert a csapatunk kicsi és nem tudunk biztonsági auditot csinálni."*
- **Üzleti logika megértése**: Ha az alkalmazásod speciális üzleti szabályokat követ, az AI nem fogja ezeket kitalálni magától. *Példa: „A nagykereskedelmi ügyfelek 30 napos fizetési haladékot kapnak, kivéve Q4-ben" – ezt valakinek el kell mondania, az AI nem fogja kitalálni.*
- **Kontextus korlát**: Minden AI modellnek van egy „kontextus ablaka" — az a mennyiségű szöveg, amit egyszerre kezelni tud. Ha ez megtelik, az AI kezd elfelejteni dolgokat az eleje felől. Erről bővebben a III. fejezetben.
- **Hallucináció**: Néha az AI magabiztosan állít olyasmit, ami nem igaz. Kitalál API-kat amik nem léteznek, vagy hivatkozik funkciókra amik nincsenek a projektben. *Példa: „Használd a `request.user.getPermissions()` metódust" – ami nem létezik a projektben, de az AI úgy írja, mintha ott lenne.*
- **Csapatdinamika**: Nem érti a cég politikáját, nem tudja, hogy Pista utálja a TypeScript-et, vagy hogy a design csapat már 3 hete dolgozik az új layouton.

1.5.3 Az arany középút

```
Amit az AI jól csinál          Ami emberi döntés marad
____________________           ______________________

Kód írás és javítás       --   Architektúra tervezés
Tesztek generálása        --   Üzleti követelmények
Rutin refaktorálás        --   Prioritás meghatározás
Dokumentáció              --   Csapat koordináció
Dependency kezelés        --   Költség/haszon elemzés
Hibakeresés               --   Biztonsági döntések
Kód review                --   Release stratégia
```

1.6 Miért fontos ez egy PM-nek?

Talán azt gondolod: „Ez mind szép és jó, de én nem fejlesztő vagyok. Miért kellene ezzel foglalkoznom?"

Azért, mert az AI ágensek alapvetően megváltoztatják a projekt menedzsment három alappillérét:

1. Idő és becslések

Ha egy fejlesztő eddig 3 napot mondott egy feladatra, az AI ágenssel lehet, hogy 3 óra alatt elkészül az alapja. De ez nem jelenti, hogy 3 óra alatt kész — a review, a finomhangolás, és az integrálás továbbra is időbe telik. A becslési módszereidet újra kell kalibrálnod.

Gyakorlati példa: Egy közepes méretű feature (mondjuk egy új keresési funkció) hagyományosan így néz ki: - Tervezés: 1 nap - Implementáció: 3 nap - Tesztelés: 1 nap - Review + javítás: 1 nap - **Összesen: ~6 nap**

AI ágenssel: - Tervezés (spec írás): 2 óra - AI implementáció: 3-4 óra - Emberi review: 2 óra - Javítások: 1 óra - **Összesen: ~1 nap**

2. Kapacitás és párhuzamosság

Eddig ha 3 fejlesztőd volt, 3 feladaton tudtatok dolgozni. AI ágensekkel egyetlen fejlesztő 5-10 feladatot futtathat párhuzamosan. Ez új mennyiségű munkát tesz lehetővé — de új típusú koordinációt is igényel.

Gyakorlati példa: Egy fejlesztő reggel indít 5 AI ágenst különböző feature-ökön. Ebéd előtt mindegyik kész az alappal. Délután review-olja az eredményeket, finomhangol, és a nap végére 5 feature megy PR-be. Hagyományosan ez 5 fejlesztő 1 hetes munkája lett volna.

3. Minőségbiztosítás

Az AI gyorsan ír kódot, de nem mindig jó kódot. A PM szerepe egyre inkább az lesz, hogy meghatározza a minőségi elvárásokat, és biztosítsa hogy ezek be legyenek tartva — nem az, hogy a fejlesztési folyamatot mikro-menedzselje.

Gyakorlati példa: A PM definiálja: „Minden új feature-höz kell legalább 80%-os teszt lefedettség, és a kódnak át kell mennie a lint ellenőrzésen." Az AI ágens automatikusan betartja ezeket — de csak ha valaki előre megmondta.

Erről a három változásról szól ez a könyv: hogyan változik az idő, a kapacitás, és a minőség — és hogyan alkalmazkodhat hozzá egy PM.

> **Ajánlott cikk**: A Harvard Business Review 2025-ös cikke az AI hatásáról a projekt menedzsmentre: hbr.org – keresd az „AI project management" témát.
>
> **Videó**: Az Anthropic YouTube csatornáján találsz demókat a Claude képességeiről: [youtube.com/@AnthropicAI](https://www.youtube.com/@AnthropicAI)

Kulcs üzenet PM-nek

Az AI nem fogja kiváltani a fejlesztőket vagy a projekt menedzsereket. De azok a fejlesztők és PM-ek, akik tudják hogyan kell AI ágensekkel dolgozni, ki fogják váltani azokat, akik nem. A különbség nem a technikai tudásban van, hanem abban, hogy megérted-e az új munkafolyamatot.

2. fejezet

Claude Code – Az új fejlesztői eszköz

2.1 Mi az a Claude Code?

A Claude Code az Anthropic cég által fejlesztett **AI kódolási eszköz**, ami közvetlenül a fejlesztő munkakörnyezetében dolgozik. Nem egy webes chatablak, ahová be kell másolni a kódot és ki kell másolni a választ – ehelyett egy teljes jogú résztvevő a fejlesztési folyamatban.

Gondolj rá úgy, mint egy rendkívül gyors gyakornokra, akinek:

- Hozzáférése van a projekt összes fájljához
- Képes parancsokat futtatni a számítógépen
- Érti a projekt szerkezetét és összefüggéseit
- Teszteket tud futtatni és az eredmények alapján javítani
- Git-ben tud commitolni és pull requestet nyitni

De fontos megérteni, miben különbözik ez a ChatGPT-től, amit valószínűleg már használsz. A ChatGPT (és hasonló chatbotok) egy **beszélgetési felület**: te kérdezel, az AI válaszol szöveggel. Ha kódot kérsz, kapsz egy választ amit másolgatnod kell. A Claude Code ezzel szemben nem beszélget a kódról – **dolgozik** a kóddal. Közvetlenül hozzáfér a fájlokhoz, futtatja a parancsokat, ellenőrzi az eredményt. Ez a különbség olyan, mint a különbség aközött, hogy valaki **leírja** hogyan kell megjavítani a csapot, és aközött, hogy **megjavítja** a csapot.

> *Hivatalos dokumentáció: Claude Code overview – az Anthropic részletes leírása az eszközről, annak képességeiről és használatáról.*

2.2 Hol fut?

A Claude Code több környezetben is elérhető – a fejlesztő választja ki, melyik illik a munkafolyamatához:

```
+-----------------------------------------------------+
|                  Claude Code                        |
|                                                     |
| +----------+ +----------+ +----------+ +--------+  |
| | Terminál | |  VS Code | | Desktop  | |  Web   |  |
| |  (CLI)   | |  (IDE)   | |  (app)   | |(böng.) |  |
| +----------+ +----------+ +----------+ +--------+  |
|                                                     |
| Mind ugyanaz a motor -- csak a felület más          |
```

```
+-----------------------------------------------------+
```

- **Terminál (CLI)**: A legteljesebb élmény. A fejlesztő a parancssort nyitja meg, beírja hogy `claude`, és beszélgethet az AI-val miközben az fájlokat szerkeszt és parancsokat futtat. Ez a legtöbbet használt mód professzionális fejlesztőknél.
- **VS Code / JetBrains**: IDE kiegészítőként – a kódszerkesztőbe beágyazva, a kód kontextusában. A fejlesztő az editorban marad, és az AI ott szerkeszt mellette. Részletek: IDE integration.
- **Desktop app**: Önálló alkalmazás több session párhuzamos kezelésére. Hasznos, ha a fejlesztő több projekten dolgozik egyszerre.
- **Web**: Böngészőben, telepítés nélkül – hosszú feladatokhoz, amikhez nem kell helyben futtatni. Hasznos, ha a fejlesztő nincs a gépe előtt, de el akar indítani egy feladatot.

A PM szempontjából mindegy, melyiket használják a fejlesztők – a lényeg, hogy **ugyanaz az AI motor** fut mindegyik mögött. Ha valaki a terminálból dolgozik, ugyanazt az eredményt kapja, mint aki a VS Code-ból.

> *IDE-specifikus részletek és telepítési útmutató: Claude Code IDE integration*

2.3 Az agentic loop – hogyan gondolkodik

A hagyományos AI chatbotnál te kérdezel, az AI válaszol, aztán te cselekszel. A Claude Code másképp működik. Ő is **cselekszik**: fájlokat olvas, kódot szerkeszt, parancsokat futtat, és az eredmények alapján tovább dolgozik.

Ezt hívjuk **agentic loop**-nak (ágentikus ciklusnak):

```
+---------------------------------------------+
|                                             |
|    +----------+                              |
|    |  GONDOL   |  "Milyen fájlokat kell      |
|    | (tervez)  |   módosítani?"               |
|    +-----+-----+                              |
|          |                                    |
|          v                                    |
|    +----------+                              |
|    | CSELEKSZIK|  Fájlt olvas, kódot ír,      |
|    |(végrehajt)|  parancsot futtat             |
|    +-----+-----+                              |
|          |                                    |
|          v                                    |
|    +----------+                              |
|    | ELLENŐRIZ |  Teszt fut? Hiba van?        |
|    |(verifikál)|  Build rendben?               |
|    +-----+-----+                              |
|          |                                    |
|          | ha kész ------------> EREDMÉNY     |
|          |                                    |
|          | ha nem kész                        |
|          +------------------+                 |
|                             |                 |
+-----------------------------+                 |
            ismétli amíg                        |
            kész nem lesz                       |
```

Ez a ciklus az, ami az AI-t **ágenssé** teszi, nem csak chatbottá. A Claude Code nem áll meg az első válasznál – addig dolgozik, amíg a feladat ténylegesen készen nincs. Ha az első javítás nem jó, megpróbálja másképp. Ha a teszt elbukik, megnézi miért, és újra javít.

Az Anthropic ezt a működést részletesen leírja a Claude Code best practices oldalon, ahol gyakorlati tanácsokat is adnak, hogyan érdemes a leghatékonyabban használni.

Részletes példa: Lássuk lépésről lépésre, hogyan dolgozik az AI, amikor egy valós feladatot kap. Mondjuk a fejlesztő azt mondja: *"A felhasználók azt jelentik, hogy a profil oldal lassan tölt be. Derítsd ki miért, és javítsd ki."*

1. **Gondol**: "Teljesítmény probléma a profil oldalon. Keresem a releváns fájlokat. A profil oldal valószínűleg a `pages/profile` vagy `routes/profile` könyvtárban lesz."
2. **Cselekszik**: A Grep eszközzel megkeresi a "profile" szót a routing fájlokban. Találat: `src/pages/ProfilePage.tsx`.
3. **Cselekszik**: Beolvassa a `ProfilePage.tsx` fájlt. Látja, hogy az oldal betöltéskor 3 külön API hívást indít: felhasználói adatok, bejegyzések, és követők listája.
4. **Gondol**: "3 szekvenciális API hívás – ez lassítja a betöltést. De előbb nézzük meg, mennyi idő telik el. Futtatom a teszteket, hogy lássam van-e már teljesítmény teszt."
5. **Cselekszik**: Bash eszközzel lefuttatja: `npm test -- --grep "profile"`. Találat: van egy teszt, de nem mér teljesítményt.
6. **Gondol**: "A 3 API hívást párhuzamosítani kell (`Promise.all`). Így a három hívás egyidejűleg indul, és a leglassabb határozza meg a teljes időt, nem összeadódnak."
7. **Cselekszik**: Módosítja a `ProfilePage.tsx` fájlt – a 3 szekvenciális `await` hívást egyetlen `Promise.all` blokkba fogja.
8. **Cselekszik**: Ír egy teljesítmény tesztet: `tests/pages/profile-performance.test.ts` – ami ellenőrzi, hogy az oldal betöltése 2 másodpercen belül megtörténik.
9. **Ellenőriz**: Lefuttatja az összes tesztet. 1 teszt elbukik – de nem a saját új tesztje, hanem egy másik, ami a profil oldal renderelését teszteli, és elvárta a régi szekvenciális viselkedést.
10. **Gondol**: "A meglévő teszt a régi API hívás sorrendet várta el. Frissítenem kell, hogy az új párhuzamos hívást tükrözze."
11. **Cselekszik**: Módosítja a másik tesztfájlt is.
12. **Ellenőriz**: Újra futtatja a teszteket. 24/24 átment.
13. **Eredmény**: "Kész. Két fájlt módosítottam, egy új tesztet írtam. A profil oldal betöltési ideje kb. harmadára csökken, mert a három API hívás most párhuzamosan fut. Commitoljam?"

Ez a példa jól mutatja a ciklus lényegét: az AI nem akar mindent egyetlen lépésben megoldani. Lépésről lépésre halad, minden lépésnél ellenőrzi az eredményt, és ha valami nem stimmel, visszalép és javít. Pontosan úgy, ahogy egy jó fejlesztő dolgozik – csak sokkal gyorsabban.

2.4 Az eszközök – mivel dolgozik

A Claude Code nem varázsol. Konkrét, meghatározott eszközöket használ, pont úgy, mint egy emberi fejlesztő. Minden egyes lépésnél az AI kiválasztja a megfelelő eszközt, használja, és az eredmény alapján dönt a következő lépésről.

Eszköz	Mit csinál	Emberi párhuzam
Read	Fájlokat olvas	Megnyitja a fájlt a szerkesztőben
Edit	Fájl egy részét módosítja	Átír egy kódrészt
Write	Új fájlt hoz létre	Új fájlt ment
Bash	Parancssori parancsokat futtat	Terminálba gépel
Grep	Szöveget keres a fájlokban	"Hol van ez a függvény?"
Glob	Fájlokat keres minta alapján	"Hol vannak a teszt fájlok?"

Eszköz	Mit csinál	Emberi párhuzam
WebFetch	Weboldalt olvas el	Böngészőt nyit
WebSearch	Interneten keres	Google-t használ

Lássunk pár gyakorlati példát, hogy ezek az eszközök hogyan működnek együtt:

1. példa – Hibakeresés (Read + Grep + Edit + Bash)

A fejlesztő azt mondja: *"A regisztrációs form nem menti el a telefonszámot."*

- Az AI **Grep**-pel megkeresi a "telefonszám" vagy "phone" szót a kódban
- **Read**-del beolvassa a találatokat: megtalálja a regisztrációs formot és az API endpoint-ot
- Észreveszi, hogy a form elküldi a `phone` mezőt, de az API endpoint nem menti el (hiányzó mező az adatbázis műveletnél)
- **Edit**-tel hozzáadja a hiányzó mezőt az API-hoz
- **Bash**-sel futtatja a teszteket: `npm test`
- Ha a teszt átment, jelenti az eredményt

2. példa – Kutatás (WebSearch + WebFetch + Read)

A fejlesztő azt mondja: *"A Stripe API v3-ra kell migrálni. Derítsd ki mit kell változtatni."*

- Az AI **WebSearch**-csel megkeresi a Stripe v3 migration guide-ot
- **WebFetch**-csel beolvassa a hivatalos migrációs dokumentációt
- **Grep**-pel megkeresi a projektben az összes Stripe API hívást
- **Read**-del beolvassa az érintett fájlokat
- Összefoglalót ad: "12 fájlban van Stripe hívás, ebből 4 igényel változtatást a v3-hoz"

Ezek az eszközök teszik lehetővé, hogy a Claude Code ne csak "mondja" mit kellene csinálni, hanem **meg is csinálja**. Az Anthropic részletesen dokumentálja az összes elérhető eszközt: Claude Code settings – tools.

> *Az Anthropic blogján olvashatod, hogyan működik az "eszközhasználat" (tool use) koncepciója: Tool use GA*

2.5 Az engedélyezési rendszer – ki mit engedhet

Egy fontos kérdés, ami PM-ként biztosan felmerül: **mennyire biztonságos?** Ki kontrollálja, mit csinálhat az AI?

A Claude Code-nak van egy részletes engedélyezési rendszere (permission model), ami a fejlesztő kezében tartja az irányítást. Két fő szintje van:

1. Olvasási eszközök (mindig engedélyezettek)

Az AI bármikor olvashat fájlokat, kereshet a kódban, és megnézhet dolgokat. Ez nem változtat semmin, tehát nem kockázatos.

2. Írási eszközök (engedélyt kell kérni)

Amikor az AI fájlt akar szerkeszteni, parancsot akar futtatni, vagy bármit akar változtatni, **engedélyt kér a fejlesztőtől**. A fejlesztő háromféleképpen reagálhat:

```
AI: Szeretném módosítani a src/auth/login.ts fájlt.
    [Megmutatja a tervezett változtatást]

Fejlesztő választhat:
+-----------------------------------------+
```

```
|  [y] Igen, engedélyezem                    |
|  [n] Nem, ne csináld                       |
|  [a] Igen, és innentől mindig engedélyezd  |
|      az ilyen típusú műveletet             |
+-------------------------------------------+
```

A fejlesztő tehát létrehozhat **szabályokat**: "A fájl szerkesztés mindig engedélyezett a `src/` könyvtárban, de a `config/` könyvtárban mindig kérdezzen." Vagy: "A `npm test` parancsot mindig futtathatja, de a `rm` (törlés) parancsot soha."

Ezek a szabályok a Claude Code settings fájlban tárolódnak, és a csapat szintjén is beállíthatóak – tehát a PM vagy a tech lead meghatározhatja, hogy az egész csapatnak mit szabad és mit nem.

PM szemmel: Az engedélyezési rendszer az, ami biztosítja, hogy az AI nem csinál semmi váratlan dolgot. A fejlesztő dönti el, mennyire "engedi szabadjára" az AI-t – és bármikor vissza tudja vonni az engedélyeket. Ez olyan, mint amikor a gyakornoknak szólsz: "A kis feladatokat csináld meg egyedül, de a fontos dolgoknál kérj engedélyt."

2.6 CLAUDE.md – a projekt memóriája

Minden projektben lehet egy `CLAUDE.md` nevű fájl. Ez egy egyszerű szöveges fájl, amit a Claude Code **minden beszélgetés elején elolvas**. Ide kerülnek a projekt sajátos tudnivalói.

Gondolj rá úgy: amikor egy új kolléga érkezik a csapatba, az első héten átolvassa a belső dokumentációt, megismeri a konvenciókat, és megtanulja, hogyan működnek a dolgok. A `CLAUDE.md` pontosan ezt a "belső dokumentációt" biztosítja az AI számára – de az AI **garantáltan el is olvassa**, minden egyes alkalommal.

A CLAUDE.md fájl több szinten is létezhet (a részletes szabályokat az Anthropic dokumentációja írja le: CLAUDE.md):

```
Projekt gyökér/
|
+-- CLAUDE.md              <-- A fő projekt szintű utasítások
|
+-- src/
|   +-- CLAUDE.md          <-- Az src könyvtár sajátos szabályai
|   +-- auth/
|   |   +-- CLAUDE.md      <-- Az auth modul specifikus szabályai
|   +-- api/
|       +-- CLAUDE.md      <-- Az API modul specifikus szabályai
|
+-- tests/
    +-- CLAUDE.md          <-- Teszt írási konvenciók
```

Ez a hierarchikus rendszer lehetővé teszi, hogy különböző könyvtárakban különböző szabályok legyenek – anélkül, hogy egyetlen hatalmas fájl lenne.

Lássunk egy realisztikus példát, hogyan néz ki egy valós projekt `CLAUDE.md` fájlja:

```
+-------------- CLAUDE.md --------------------------------+
|                                                         |
|  # Projekt: WebShop Backend                             |
|                                                         |
|  ## Kód stílus                                         |
```

```
|  - TypeScript strict mode-ban írunk                      |
|  - Változókhoz camelCase-t használunk                    |
|  - Függvényekhez egyértelmű, angol neveket adunk         |
|  - Minden publikus függvényhez JSDoc kommentet írunk     |
|                                                          |
|  ## Architektúra                                        |
|  - Backend: Express.js + TypeORM                         |
|  - Adatbázis: PostgreSQL                                 |
|  - Cache: Redis                                          |
|  - A src/services/ réteg tartalmazza az üzleti logikát   |
|  - A src/controllers/ csak a HTTP kezelést végzi         |
|                                                          |
|  ## Tesztek                                              |
|  - Teszt framework: Jest                                 |
|  - Futtatás: npm test                                    |
|  - Minimum lefedettség: 80%                              |
|  - Minden új feature-höz kell unit és integration teszt |
|  - Mock-ot használunk külső API hívásokhoz               |
|                                                          |
|  ## Git konvenciók                                       |
|  - Branch név: feature/JIRA-123-rövid-leírás             |
|  - Commit üzenet: "feat: leírás" vagy "fix: leírás"    |
|  - A main branch-re TILOS közvetlenül pusholni           |
|  - Minden PR-hez kell legalább 1 approver                |
|                                                          |
|  ## Fontos szabályok                                     |
|  - A .env fájlokat SOHA ne commitold                     |
|  - Az adatbázis migrációkat kézzel kell futtatni         |
|  - A /admin/* endpointokhoz mindig kell admin role       |
|    ellenőrzés                                            |
|  - Érzékeny adatokat (jelszó, token) SOHA ne logold      |
|                                                          |
|  ## Gyakori parancsok                                    |
|  - Build: npm run build                                  |
|  - Teszt: npm test                                       |
|  - Lint: npm run lint                                    |
|  - Dev szerver: npm run dev                              |
|  - Migráció: npm run db:migrate                          |
|                                                          |
+----------------------------------------------------------+
```

PM szemmel: A CLAUDE.md hasonlít egy belső wiki oldalhoz, amit a fejlesztőknek írsz. Csak éppen nem embereknek szól, hanem az AI-nak. Ha eddig frusztráltad, hogy az új fejlesztő nem olvassa el a projekt konvenciókat – az AI garantáltan elolvassa, minden egyes alkalommal.

Még egy fontos szempont: a CLAUDE.md **verziócontrollba** kerül (Git-ben tárolódik). Tehát a PM látja, ki mikor változtatta meg a szabályokat, és visszakövethető, hogyan fejlődött a projekt konfigurációja.

Bővebben: CLAUDE.md dokumentáció – példa fájlok, hierarchia, és advanced tippek.

2.7 Hookrendszer – automatikus minőségbiztosítás

A hookrendszer (kampórendszer) lehetővé teszi, hogy bizonyos eseményekre automatikus műveleteket kössünk. Gondolj rá úgy, mint szabályokra, amik **mindig lefutnak**, emberi feledékenységtől függetlenül.

Például:

- **Fájl szerkesztés után** –> automatikusan lefut a kódformázó (linter)
- **Commit előtt** –> automatikusan lefutnak a tesztek
- **Bash parancs futtatásakor** –> ellenőrzi, hogy nem töröl-e fontos fájlokat
- **Session elején** –> betölti a korábbi tapasztalatokat (memória)
- **Session végén** –> elmenti a tanulságokat a következő session-höz

```
Esemény                    Hook (automatikus akció)
--------                   ------------------------
AI fájlt szerkeszt    -->  Kódformázó lefut
AI commitolni akar    -->  Tesztek lefutnak
AI bash-t használ     -->  Biztonsági ellenőrzés
Session indul         -->  Korábbi emlékek betöltése
Session véget ér      -->  Tanulságok mentése
```

Lássunk két konkrét példát, hogy érthetőbb legyen, hogyan is működik ez a gyakorlatban:

1. példa – A kódformázó hook

A csapatban megállapodás, hogy a kód mindig egységes formátumú legyen (pl. behúzások, sortörések, zárójelek). Amikor az AI módosít egy fájlt, automatikusan lefut a kódformázó:

```
AI módosítja: src/auth/login.ts
     |
     v
Hook aktiválódik: "prettier --write src/auth/login.ts"
     |
     v
A fájl automatikusan formázva lesz
     |
     v
AI látja az eredményt és tovább dolgozik
```

Ez biztosítja, hogy az AI által írt kód **mindig megfelel** a csapat formázási szabályainak – anélkül, hogy bárki szólna neki.

2. példa – A biztonsági hook (amikor megakadályoz valamit)

Tegyük fel, hogy a hookrendszerben van egy szabály: "A `rm -rf` parancsot soha ne futtasd a projekt gyökerében." Mi történik, ha az AI mégis megpróbálja?

```
AI futtatni akarja: "rm -rf ./node_modules && npm install"
     |
     v
Hook ellenőrzi a parancsot
     |
     v
BLOKKOL! "A 'rm -rf' parancs nem engedélyezett
          a projekt gyökerében. Használd a
```

```
            'npm ci' parancsot ehelyett."
      |
      v
AI megkapja az üzenetet, és másképp oldja meg:
"npm ci"  (ami ugyanazt éri el, biztonságosan)
```

A hook tehát nem csak figyelmeztet – **meg is akadályozza** a veszélyes műveletet, és általában alternatívát is javasol. Az AI megkapja a hook üzenetet, alkalmazkodik, és másképp oldja meg a feladatot.

PM szemmel: A hookrendszer az, ami biztosítja, hogy az AI betartsa a csapat szabályait. Nem kell bízni abban, hogy "emlékezni fog" – a hookrendszer kikényszeríti. Ez olyan, mint egy automatikus minőség-ellenőrző sor egy gyárban: minden termék (változtatás) átmegy rajta, és ami nem felel meg, az nem megy tovább.

Részletek és konfigurációs példák: Hooks guide

2.8 MCP – az "USB-C" az AI alkalmazásoknak

Az MCP (Model Context Protocol – Modell Kontextus Protokoll) egy nyílt szabvány, ami lehetővé teszi, hogy az AI eszközök **külső rendszerekhez** csatlakozzanak.

Gondolj rá úgy, mint az USB-C-re: ahogy az USB-C szabványos csatlakozást biztosít különböző elektronikai eszközök között, az MCP szabványos csatlakozást biztosít az AI és a külvilág között. Az MCP-t maga az Anthropic fejlesztette ki, és nyílt szabványként adta ki – tehát bárki fejleszthet MCP szervereket, és bármely AI eszköz csatlakozhat hozzájuk. A teljes architektúra leírása elérhető: MCP architecture.

```
+----------+     MCP      +------------------+
|          |------------->| Google Drive      |
|          |              +------------------+
|          |     MCP      +------------------+
|  Claude  |------------->| Jira / Notion    |
|  Code    |              +------------------+
|          |     MCP      +------------------+
|          |------------->| Adatbázis        |
|          |              +------------------+
|          |     MCP      +------------------+
|          |------------->| Figma / Design   |
|          |              +------------------+
|          |     MCP      +------------------+
|          |------------->| Slack / Teams    |
+----------+              +------------------+
```

Az MCP nélkül az AI csak a helyi fájlrendszert és a parancssort látja. MCP-vel viszont:

- Elolvashatja a Google Drive-on lévő követelményeket
- Megnézheti a Jira ticketeket
- Lekérdezheti az adatbázist
- Behúzhatja a Figma design-okat
- Üzenetet küldhet Slack-en

Lássunk két konkrét példát, hogyan néz ki ez a gyakorlatban:

1. példa – Jira integráció

A fejlesztő azt mondja: *"Nézd meg a PROJ-456 Jira ticket-et és implementáld amit kér."*

MCP nélkül az AI nem tudná mit csinálni – nincs hozzáférése a Jira-hoz. MCP-vel viszont:

```
Fejlesztő: "Nézd meg a PROJ-456 ticketet és csináld meg"
     |
     v
AI használja az MCP Jira eszközt:
jira.get_issue("PROJ-456")
     |
     v
Visszakapja:
- Cím: "Kereső mező hozzáadása a termék listához"
- Leírás: "Szöveges kereső mező kell a termék lista
  tetejére, ami név és kategória szerint szűr.
  Elfogadási kritérium: a keresés debounce-olt legyen
  (300ms), és üres állapotban minden terméket mutasson."
     |
     v
AI implementálja a keresőt a specifikáció alapján
     |
     v
AI frissíti a Jira ticketet: állapot -> "In Review"
```

2. példa – Adatbázis lekérdezés

A fejlesztő hibát keres: *"A /users endpoint üres listát ad vissza. Nézd meg az adatbázisban, van-e egyáltalán felhasználó."*

```
AI használja az MCP adatbázis eszközt:
db.query("SELECT COUNT(*) FROM users")
     |
     v
Eredmény: 15,234 felhasználó van az adatbázisban
     |
     v
AI: "Az adatbázisban vannak felhasználók. A probléma
     nem az adatban van, hanem a lekérdezésben.
     Megvizsgálom az API kódot..."
```

PM szemmel: Az MCP teszi lehetővé, hogy az AI ne csak kódot írjon, hanem a teljes fejlesztési ökoszisztémában mozogjon. Ha a követelmények Google Docsban vannak, az AI onnan is el tudja olvasni. Ha a feladatok Jira-ban vannak, az AI közvetlenül onnan dolgozik. Ez csökkenti a "kontextus váltást" – az AI nem kéri, hogy másold be a Jira ticketet, hanem maga olvassa el.

MCP bevezető és koncepció: Model Context Protocol – Introduction

2.9 Subágensek – párhuzamos munkavégzés

A Claude Code képes **több kisebb ágenst** (subágenst) indítani, amik párhuzamosan dolgoznak. Ez olyan, mintha a gyakornokod szólna 3 másik gyakornoknak, hogy "te nézd meg az adatbázist, te a frontend-et, te pedig futtasd a teszteket" – és utána összefoglalná az eredményt.

```
+-------------------------------------------+
|            Fő Claude Code session          |
```

```
|                                            |
|    "Vizsgáld meg az auth, a db, és az API   |
|     modult párhuzamosan"                    |
|                                            |
|    +-----------+ +----------+ +----------+  |
|    | Subágens A| |Subágens B| |Subágens C|  |
|    |  Auth     | |  DB      | |  API     |  |
|    | vizsgálat | | vizsgálat| | vizsgálat|  |
|    +-----+-----+ +----+-----+ +----+-----+  |
|          |            |            |         |
|          +-------------+------------+         |
|                       |                      |
|                  Összefoglalás               |
|                                            |
+--------------------------------------------+
```

Lássunk egy részletes példát arra, hogyan működik ez a gyakorlatban.

Példa: Biztonsági audit

A fejlesztő azt mondja: *"Vizsgáld át a projekt biztonságát: az authentikációt, az SQL lekérdezéseket, és a környezeti változók kezelését."*

A fő ágens három subágenst indít:

```
Fő ágens: "Biztonsági audit -- 3 területet kell átvizsgálni"
     |
     +---> Subágens A: Autentikáció vizsgálat
     |     - Beolvassa az auth/ könyvtár összes fájlját
     |     - Ellenőrzi a jelszó hash-elést (bcrypt? salt?)
     |     - Megvizsgálja a session kezelést
     |     - Keresi a hard-coded tokeneket
     |     - Eredmény: "A jelszó hash-elés rendben (bcrypt),
     |       de a session token nem jár le soha -- javítandó."
     |
     +---> Subágens B: SQL injection vizsgálat
     |     - Megkeresi az összes adatbázis lekérdezést
     |     - Ellenőrzi, hogy mindenhol paraméterezett
     |       lekérdezéseket használnak-e (nem string concat.)
     |     - Eredmény: "2 helyen raw SQL van paraméterezés
     |       nélkül: src/reports/export.ts és
     |       src/admin/search.ts -- kritikus."
     |
     +---> Subágens C: Környezeti változók vizsgálat
           - Ellenőrzi a .env.example fájlt
           - Keresi a hard-coded jelszavakat és API kulcsokat
           - Megvizsgálja, hogy a .env a .gitignore-ban van-e
           - Eredmény: "A .env a .gitignore-ban van (rendben),
             de a src/config/stripe.ts-ben hard-coded test
             API kulcs van -- javítandó."

Fő ágens összefoglal:
```

```
"Biztonsági audit eredménye:
 - KRITIKUS: 2 SQL injection sebezhetőség
 - FONTOS: Session token nem jár le
 - FONTOS: Hard-coded Stripe API kulcs
 - RENDBEN: Jelszó hash-elés, .env kezelés
 Javítsam a kritikus problémákat?"
```

Miért fontos a subágensek használata?

- **Sebesség**: 3 párhuzamos vizsgálat gyorsabb, mint 3 egymás utáni. A fenti audit 3 perc alatt lefut, nem 9 perc alatt.
- **Kontextus védelem**: Minden subágens a saját ablakában dolgozik, nem terheli a fő beszélgetést. A fő ágens kontextusa tiszta marad az összefoglaláshoz.
- **Specializáció**: Minden subágens más feladatra optimalizálható – más utasításokat kap, más fájlokat lát.
- **Megbízhatóság**: Ha egy subágens elakad, a többi tovább dolgozik.

Bővebben: Sub-agents dokumentáció – architektúra, konfiguráció, és advanced használati minták.

2.10 GitHub Actions – AI a CI/CD folyamatban

Az eddigi példákban a Claude Code mindig a fejlesztő gépén futott, a fejlesztő felügyelete mellett. De mi van, ha az AI-t be akarjuk építeni az **automatizált fejlesztési folyamatba** – hogy emberi beavatkozás nélkül is dolgozzon?

Erre való a Claude Code GitHub Actions integráció. Ez lehetővé teszi, hogy a Claude Code **automatikusan reagáljon** GitHub eseményekre:

```
GitHub esemény              Claude Code reakció
----------------           ---------------------
Új issue nyílik        -->  AI elemzi és címkézi a hibajegyet
PR nyílik              -->  AI code review-t végez
PR-re komment érkezik  -->  AI válaszol / javít
Label hozzáadva        -->  AI implementálja a feature-t
```

Példa: Automatikus code review

Képzelj el egy munkafolyamatot, ahol minden pull request automatikusan kap egy AI review-t:

```
1. Fejlesztő megnyitja a PR-t: "Add search functionality"
     |
     v
2. GitHub Actions elindítja a Claude Code-ot
     |
     v
3. AI áttekinti a PR-ben lévő összes változtatást
     |
     v
4. AI kommentet ír a PR-re:

   "Code Review Összefoglalás:

   Általános: A keresési funkcionalitás jól strukturált.
```

```
Észrevételeim:
- src/search.ts:42 -- A keresési lekérdezésben nincs
  index a 'name' mezőn. Nagy adatbázisnál ez lassú
  lesz. Javaslom egy index létrehozását.
- src/search.ts:67 -- A felhasználói input nincs
  sanitizálva -- XSS kockázat.
- tests/search.test.ts -- Hiányoznak az üres input
  és a speciális karakterek tesztesetei."
```

Példa: Issue-ből automatikus implementáció

Még érdekesebb: a Claude Code képes **GitHub issue-ból automatikusan PR-t csinálni**:

```
1. PM létrehoz egy issue-t: "A láblécből hiányzik a
   copyright évszám"
     |
     v
2. Valaki hozzáadja a "claude" labelt
     |
     v
3. GitHub Actions elindítja a Claude Code-ot
     |
     v
4. AI elolvassa az issue-t, megkeresi a releváns fájlokat,
   elvégzi a változtatást, és megnyit egy PR-t:
   "fix: add copyright year to footer"
     |
     v
5. A PR-t egy ember review-olja és mergeli
```

PM szemmel: A GitHub Actions integráció az, ami lehetővé teszi, hogy az AI ne csak "kérésre" dolgozzon, hanem **automatikusan reagáljon** a fejlesztési folyamat eseményeire. Ez különösen hasznos rutinfeladatoknál: code review, egyszerűbb bugfixek, dokumentáció frissítés. Az ember pedig a fontos döntésekre összpontosíthat.

2.11 Egy konkrét példa: bugfix a gyakorlatban

Lássunk egy valós forgatókönyvet, lépésről lépésre, részletesen. A fejlesztő reggel megnyitja a terminált:

```
Fejlesztő:  "A felhasználók azt jelentik, hogy a jelszó-
             visszaállítás nem működik timeout hibával.
             Javítsd ki és írj rá tesztet."
```

Most követjük, mit csinál az AI, minden egyes lépést:

```
[1. lépés -- Keresés]
AI gondolkodik: "Jelszó visszaállítás... Keresem a releváns
                 fájlokat."
AI használja: Grep("password-reset" VAGY "passwordReset")

Találatok:
  src/auth/password-reset.ts
  src/auth/password-reset.service.ts
  src/email/templates/password-reset.html
```

```
  tests/auth/password-reset.test.ts

[2. lépés -- Olvasás]
AI használja: Read(src/auth/password-reset.ts)

Látja a kódot:
  async function resetPassword(email: string) {
    const user = await findUser(email);
    const token = generateToken();
    await sendEmail(email, token, { timeout: 5000 });
    //                              ^^^^^^^^^^^^^^^^
    //   AI észreveszi: 5000ms = 5 másodperc timeout
    await saveToken(user.id, token);
  }

[3. lépés -- Elemzés]
AI gondolkodik: "5 másodperces timeout az email küldésre.
                 Az SMTP szerverek általában 10-30 másodperc
                 alatt válaszolnak. Ez túl rövid. De nézzük
                 meg a service fájlt is, hátha ott is van
                 hasonló."

AI használja: Read(src/auth/password-reset.service.ts)

Találat: itt is van egy 5000-es timeout az email
verifikálásnál.

[4. lépés -- Javítás]
AI használja: Edit(src/auth/password-reset.ts)
  timeout: 5000  -->  timeout: 30000

AI használja: Edit(src/auth/password-reset.service.ts)
  timeout: 5000  -->  timeout: 30000

[5. lépés -- Teszt írás]
AI használja: Write(tests/auth/password-reset-timeout.test.ts)

  describe('Password Reset Timeout', () => {
    it('should handle slow SMTP server', async () => {
      // Szimulál egy lassú SMTP szervert (15 másodperc)
      mockSmtp.setDelay(15000);
      const result = await resetPassword('user@test.com');
      expect(result.success).toBe(true);
    });

    it('should timeout after 30 seconds', async () => {
      // Szimulál egy nagyon lassú szervert (35 másodperc)
      mockSmtp.setDelay(35000);
      await expect(resetPassword('user@test.com'))
```

```
        .rejects.toThrow('timeout');
    });
  });

[6. lépés -- Teszt futtatás]
AI használja: Bash("npm test -- --grep password-reset")

Eredmény:
  PASS tests/auth/password-reset.test.ts (8 teszt)
  PASS tests/auth/password-reset-timeout.test.ts (2 teszt)
  FAIL tests/email/email-service.test.ts
    > "Expected timeout to be 5000 but got 30000"

[7. lépés -- Újabb javítás]
AI gondolkodik: "Van egy teszt ami az 5 másodperces
                 timeout-ot várta. Frissítenem kell."

AI használja: Read(tests/email/email-service.test.ts)
AI használja: Edit(tests/email/email-service.test.ts)
  expect(config.timeout).toBe(5000)
  -->
  expect(config.timeout).toBe(30000)

[8. lépés -- Végső ellenőrzés]
AI használja: Bash("npm test")

Eredmény:
  47 passing (12s)
  0 failing

[9. lépés -- Jelentés]
AI: "Kész. A javítás összefoglalása:

     Probléma: Az email küldés 5 másodperces timeout-ot
     használt, ami túl rövid volt az SMTP szervernek.

     Javítás:
     - src/auth/password-reset.ts: timeout 5s --> 30s
     - src/auth/password-reset.service.ts: timeout 5s --> 30s
     - tests/email/email-service.test.ts: timeout assert
       frissítése

     Új tesztek:
     - tests/auth/password-reset-timeout.test.ts:
       2 új teszt eset (lassú szerver + timeout)

     Összes teszt: 47/47 átment.
     Commitoljam?"
```

A fejlesztő átnézi a változtatásokat, jóváhagyja, és a Claude Code commitol. Az egész **3-5 perc** volt, amihez egy ember 30-60 percet ülne.

Fontos: a fejlesztő végig látta, mit csinál az AI. Nem kellett vakon bíznia benne – minden lépés látható, és bármikor közbe tudott volna lépni. Ez a "human-in-the-loop" elv: az AI dolgozik, de az ember felügyel.

2.12 Tippek a hatékony használathoz

Az Anthropic összegyűjtötte a Claude Code best practices oldalon a legfontosabb tippeket. Íme a PM számára relevánsak:

1. Pontos utasítások adása

Minél pontosabb az utasítás, annál jobb az eredmény:

```
Rossz:  "Javítsd meg a login-t"
Jó:     "A login oldalon a 'Password' mező nem fogad el
         speciális karaktereket (pl. @, #). Javítsd ki,
         és írj tesztet ami ellenőrzi a speciális
         karakterek működését."
```

2. A CLAUDE.md karbantartása

Ha a csapat új konvenciót vezet be, az kerüljön be a CLAUDE.md-be is. Különben az AI a régi szabályok szerint fog dolgozni.

3. Kis, jól definiált feladatok

Az AI jobban teljesít, ha egy feladatot kap, nem tízet egyszerre:

```
Rossz:  "Csináld meg az egész user management modult"
Jó:     "Csinálj egy GET /users endpoint-ot, ami
         paginált listát ad vissza, maximum 50 elem/oldal"
```

4. Az eredmény ellenőrzése

Bármennyire is jó az AI, az emberi review elengedhetetlen. A PM feladata biztosítani, hogy **minden AI által írt kód** átmenjen code review-n – pont úgy, mint egy emberi fejlesztő kódja.

> *További tippek: Claude Code best practices – kontextus kezelés, prompt stratégiák, és advanced technikák.*
>
> *Az Anthropic YouTube csatornáján demókat és tutorialokat találsz: [youtube.com/@AnthropicAI](https://www.youtube.c*

2.13 Összefoglalva

Fogalom	Mit jelent PM-nek
Claude Code	AI fejlesztő eszköz, ami fájlokat szerkeszt, tesztel, commitol
Agentic loop	Automatikus ciklus: gondol –> cselekszik –> ellenőriz
Eszközök (tools)	Konkrét képességek: olvasás, írás, keresés, futtatás
Engedélyezési rendszer	A fejlesztő kontrollálja mit csinálhat az AI
CLAUDE.md	Projekt szabályok amit az AI mindig betart
Hookrendszer	Automatikus minőség-ellenőrzés minden lépésnél
MCP	Az AI csatlakozhat Jira-hoz, Google Docs-hoz, stb.
Subágensek	Több AI dolgozik párhuzamosan, majd összegez
GitHub Actions	Az AI automatikusan reagál GitHub eseményekre

Kulcs üzenet PM-nek

A Claude Code nem ChatGPT, ahová kódot másolsz. Ez egy teljes fejlesztőkörnyezetben dolgozó ágens, ami fájlokat szerkeszt, teszteket futtat, és commitol – a fejlesztő felügyeletével. Mint PM, nem a kódot kell értened, hanem a munkafolyamatot: az AI önállóan dolgozik, de az ember felügyel és dönt. Az engedélyezési rendszer, a hookrendszer, és a CLAUDE.md együtt biztosítják, hogy az AI a csapat szabályai szerint dolgozzon. A GitHub Actions integráció pedig lehetővé teszi, hogy az AI a CI/CD folyamatban is részt vegyen – automatikus code review-tól az egyszerű bugfixekig.

3. fejezet

Vibe Coding — a csábítás és a csapda

3.1 Mi az a „vibe coding”?

2025 februárjában Andrej Karpathy — a világ egyik legismertebb AI kutatója, korábban a Tesla AI igazgatója, az OpenAI alapító tagja — közzétett egy bejegyzést az X-en, amiben leírta az új fejlesztési stílusát:

> *„Csak látok dolgokat, mondok dolgokat, futtatom a kódot, és bemásolom a dolgokat, és nagyjából működik.”*

Ezt nevezte el **vibe coding**-nak — szó szerinti fordításban „hangulat alapú kódolás”. A lényeg: nem kell értened a kódot, csak elmondod az AI-nak mit akarsz, és ő megcsinálja. Ha nem jó, mondod újra, más szavakkal. Mintha egy beszélgetésből születne a szoftver.

A kifejezést azóta az egész technológiai iparág átvette. A ThoughtWorks Technology Radar 2025-ös kiadásában már külön pontot kapott, és számos szakmai blogon — például Simon Willison AI coding blogjában — rendszeresen elemzik az előnyeit és a veszélyeit. A Fireship YouTube csatorna több milliós megtekintésű videókban magyarázza el a koncepciót rövidre fogva.

És ez működik. Prototípusokra, egyszerű alkalmazásokra, személyes projektekre tökéletesen alkalmas. Egy hétvége alatt működő weboldalt csinálhatsz anélkül, hogy egyetlen sort írnál.

De van egy probléma. És ez a probléma annál nagyobb, minél komolyabb a projekt.

3.2 A való világban: házépítés terv nélkül

Mielőtt belemerülünk a technikai részletekbe, gondoljunk egy egyszerű analógiára, amit minden PM ismer.

Képzeld el, hogy házat építesz. Két lehetőséged van:

A) Terv nélkül („vibe building”)

Odamész az építkezésre és azt mondod: „Csináljatok egy szép házat.” Az építőmester elkezd téglát rakni. Szép lesz az első szoba. Aztán szólsz: „Legyen még egy emelet.” Na, de a fundamentum ehhez gyenge. Szóval: át kell építeni az alapot. Ami azt jelenti, hogy az első szoba fele lerombolódik. Újrakezdés. Aztán jön a villanyszerelő: „Hol menjenek a kábelek?” Senki nem tudja, mert nincs tervrajz. Minden fal felbontása újabb meglepetés.

B) Tervrajzzal („spec-driven building”)

Először megrajzolja az építész a terveket. Az „elveszett” idő? Két hét. De utána minden szakember pontosan tudja, mit hol kell csinálni. A PM (építésvezető) megnézi a tervet és mondja: „Oké, de a konyha legyen nagyobb.”

Átrajzolják a papírt — nem kell falat bontani. Az építés után a tervrajz megmarad: ha tíz év múlva felújítasz, pontosan látod, hol mennek a csövek.

Ez az analógia tökéletesen lefordítható szoftverre:

```
Házépítés                        Szoftverfejlesztés
─────────                        ──────────────────

Tervrajz                         Specifikáció
Fundamentum                      Architektúra
Falépítés                        Kód írás
Villanyszerelés                  Integrációk (API, DB)
Lakás-átadáskor ellenőrzés       Tesztelés és review
Építésvezető (PM)                Projektmenedzser
Tulajdonos                       Stakeholder
```

A vibe coding olyan, mint terv nélkül építeni: az első szoba gyorsan kész, de ahogy nő a ház, egyre több mindent kell újracsinálni.

3.3 A csábítás

A vibe coding azért csábító, mert **azonnali eredményt ad**. Nincs tervezés, nincs specifikáció, nincs design review. Csak te és az AI, beszélgettek, és 10 perc múlva van valami ami működik.

Addy Osmani, a Google senior mérnöke, aki sokat ír a fejlesztői produktivitásról, sokszor rámutatott arra, hogy a „gyors eredmény” érzete megtévesztő lehet. Az azonnali feedback loop — látod hogy valami működik — dopamint ad, és azt az illúziót kelti, hogy a projekt jól halad. De a „működik a képernyőmön” és a „kész a production-re” között hatalmas a különbség.

```
Vibe Coding munkafolyamat:

"Csinálj egy webshopot"
         |
         v
AI generál valamit
         |
         v
"Ez nem jó, legyen kék"
         |
         v
AI módosít
         |
         v
"A kosár nem működik"
         |
         v
AI javít... vagy ront
         |
         v
"Már a login sem megy!"
         |
         v
```

```
???
```

Ismerős? Ez a „javítsd meg A-t, elromlik B” spirál a vibe coding természetes következménye. Nincs terv, nincs struktúra — minden változtatás potenciálisan elront valami mást.

Ez nem is meglepő, ha belegondolunk. Martin Fowler, a szoftverfejlesztés egyik legnagyobb gondolkodója, már évtizedek óta írja, hogy a szoftver entrópiája (rendezetlensége) természetesen nő, hacsak nincs tudatos erőkifejtés a struktúra megtartására. Terv nélkül az entrópia még gyorsabban nő — és az AI ezt csak felgyorsítja, mert hihetetlenül gyorsan generál újabb és újabb kódot, amit senki nem gondolt át.

3.4 Vibe coding kudarcok a valóságban

Mielőtt valaki azt gondolná, hogy ez elméleti probléma, lássunk néhány tipikus forgatókönyvet, amiket PM-ként felismerhetsz:

1. Az „eltűnő funkciók” esete

Egy csapat vibe coding-gal épít egy ügyfélportált. Készül a regisztráció, a bejelentkezés, a profil oldal. Minden szépen működik. Aztán jön egy új kérés: „Legyen kereső funkció is.” Az AI implementálja a keresést — de közben észrevétlenül felülírja a bejelentkezési logika egy részét. A bejelentkezés még működik az egyszerű esetben, de a „maradj bejelentkezve” funkció csendben eltűnik. Ezt csak három héttel később veszi észre egy ügyfél.

Miért történik ez? Mert a vibe coding-ban nincs specifikáció, ami leírná, hogy minek kell működnie. Nincs tesztlista, amit le lehetne futtatni. Nincs mód automatikusan ellenőrizni, hogy a régit nem rontottuk el.

2. A „már senki nem érti” szindróma

Egy fejlesztő három hónapon át vibe coding-gal épít egy belső eszközt. Működik, mindenki használja. Aztán a fejlesztő elmegy szabadságra, és valaki másnak kell változtatnia. Az új fejlesztő megnyitja a kódot: semmi dokumentáció, semmi specifikáció, semmi magyarázat arról, hogy miért úgy van megoldva valami, ahogy. A kód „működik”, de senki nem tudja biztonságosan módosítani, mert senki nem érti.

3. A „kicsiben jó, nagyban katasztrófa” projekt

Egy startup MVP-t épít (Minimum Viable Product — a legkisebb működő termék). Vibe coding-gal két hét alatt készül egy demo. Az investorok lelkesek, jön a finanszírozás. Most jön a valós fejlesztés: több felhasználó, több funkció, biztonság, skálázás. De a vibe coding-gal épített alapok nem bírják: az egész alkalmazást újra kell írni. Az „előnyt” amit a gyorsaság adott, azonnal elveszíti az újraírás.

Mindhárom eset közös tanulsága: a vibe coding rejtett költségeket halmoz fel. Ezeket a szoftverfejlesztésben **technikai adósságnak** hívjuk (erről bővebben később).

3.5 A három alapvető probléma

3.5.1 1. A kontextus probléma

Minden AI modellnek van egy **kontextus ablaka** — ez az a mennyiségű szöveg (kód, beszélgetés, fájl tartalom), amit egyszerre kezelni tud. Gondolj rá úgy, mint az AI „munkamemóriájára”.

Hogy értsd a méretrendet: a Claude modell kontextus ablaka kb. 200 000 token. Ez elsőre hatalmasnak tűnik, de nézzük meg, mit jelent a gyakorlatban:

- 1 token kb. 4 karakter (angol szövegben; magyar szövegben kicsit kevesebb)
- 200 000 token kb. 150 000 sornyi kód
- Egy átlagos forrásfájl 100-300 sor
- Tehát a kontextus ablak kb. **500-1500 fájlnyi** kódot képes befogadni

De ez félrevezető, mert a kontextus ablakban nem csak a kód van:

- A beszélgetés előzménye (minden kérdés és válasz)
- Az AI „gondolatai” (a válaszgenerálás közbeni lépések)
- A rendszer utasítások (CLAUDE.md, projekt szabályok)
- A fájlok tartalma, amiket az AI megnyitott

A gyakorlatban tehát a használható kontextus **jóval kevesebb** — egy tipikus hosszabb munka session-ben az AI effektíve talán 20-50 fájlt képes átlátni, miközben a beszélgetés-történetet is tartja. Bővebben erről az Anthropic kontextus kezelési útmutatójában olvashatsz.

```
A kontextus ablak telítődése:


+──────────────────────────────────────────────────+
| ##........................................... | 10% -- friss,
|                                              |        mindent lát
| ##############............................... | 30% -- jól dolgozik
|                                              |
| ##############################................ | 60% -- lassul,
|                                              |        néha felejt
| ##############################################.. | 90% -- hibázik,
|                                              |        korábbi
| ################################################ | 100%   utasításokat
|                                              |        elfelejti
+──────────────────────────────────────────────────+
```

Vibe coding során ez a kontextus gyorsan megtelik:

- Elküldöd az első kérést –> az AI elolvassa a fájlokat –> 20% foglalt
- Javítást kérsz –> újabb fájlok, előző beszélgetés –> 40%
- Megint javítás –> még több kontextus –> 60%
- A 10. javításnál –> a kontextus szinte tele van –> az AI „elfelejti” az első kéréseidet

Eredmény: Az AI javítja a legutóbbi hibát, de közben elrontja amit 5 lépéssel korábban csinált. Mert már nem „emlékszik” rá.

Spec-driven megközelítésben ez a probléma kezelhető: az AI nem egy végtelen chat-ből dolgozik, hanem egy rövid, strukturált specifikációból. Minden session elején elolvassa a specifikációt és a feladatlistát — nem kell „emlékezni” a korábbi beszélgetésekre. Ez az, amit az Anthropic hivatalosan is ajánlott gyakorlatként ismertet a Claude Code best practices dokumentációban.

3.5.2 2. A minőség probléma

A vibe coding legnagyobb csapdája: **nincs mit ellenőrizni**. Ha nincs specifikáció, honnan tudod, hogy a szoftver azt csinálja, amit kell?

```
Vibe Coding:                  Spec-Driven:


"Csinálj login oldalt"        Specifikáció:
      |                       - Email + jelszó mező
      v                       - "Elfelejtett jelszó" link
AI csinál valamit             - 3 sikertelen próba után zárolás
      |                       - Google OAuth opció
      v                       - Hibás jelszó: "Helytelen adatok"
```

```
"Kész van?"                          |
"...szerintem igen"                  v
                                AI implementál a spec alapján
                                     |
                                     v
                                Ellenőrizhető:
                                [x] Van email mező?
                                [x] Van "Elfelejtett jelszó"?
                                [x] Zárol 3 próba után?
                                [x] Google OAuth működik?
```

A spec-driven megközelítésnél minden követelmény le van írva, és minden követelményhez tartozik egy ellenőrizhető feltétel. A PM pontosan tudja, mi van kész és mi nincs.

Vibe coding-nál a „kész" definíciója: „úgy tűnik, működik". Ami production-ben gyakran azt jelenti: „az alap eset működik, de a szélső esetek nem".

Kent Beck, az agilis szoftverfejlesztés egyik megalkotója, a Tidy First? című könyvében írja, hogy a szoftver minőség nem véletlenül történik — tudatos, inkrementális lépések során épül fel. A vibe coding pont az ellenkezőjét csinálja: véletlenszerű lépéseket tesz, és reménykedik, hogy a végeredmény jó lesz.

PM-ként gondolj erre így: ha egy beszállító azt mondja neked, „szerintem kész van", az nem elfogadható. Ami elfogadható: „itt a követelménylista, minden pontot kipipáltam, és itt a bizonyíték (teszteredmények) hogy működik." A spec-driven megközelítés ezt adja.

3.5.3 3. A skálázás probléma

A vibe coding egyetlen chat session-ben zajlik. Ez a session:

- **Egy ember** beszélget **egy AI-val**
- **Egy szál** gondolkodás — nem párhuzamosítható
- **Egy kontextus ablak** — korlátozott kapacitás
- **Nincs memória** — a következő session-ben minden előlről

```
Vibe Coding skálázása:

Projekt mérete:    Fájlok:      Vibe Coding hatékonysága:
_______________    ______       _________________________

Kis script         1-5          ############ Kiváló
Kis app            5-20         ########.... Jó
Közepes app        20-50        #####....... Nehézkes
Nagy app           50-200       ##.......... Alig működik
Enterprise         200+         ............ Lehetetlen
```

Egy 200 fájlos projektben az AI nem tudja egyszerre a fejében tartani az összeset. A vibe coding egyszerűen nem skálázódik komplex projektekre.

Miért fontos ez PM-ként? Mert a legtöbb valóban értéket termelő szoftver — amiért ügyfelek fizetnek, ami a céged működteti — nem 5 fájlból áll. Egy tipikus webes alkalmazás 50-500 fájl. Egy nagyobb vállalati rendszer ezernél is több. Ezeknél a méreteknél a vibe coding nem lassul — összeomlik.

A spec-driven megközelítés azért skálázódik, mert **lebontja a munkát**: a nagy feladat kicsi, független darabokra bomlik, és minden darabnál az AI csak a releváns fájlokkal dolgozik. Több AI ágens párhuzamosan dolgozhat, mindegyik a saját kontextus ablakában, a saját részfeladatán. Erről bővebben a következő fejezetben lesz szó.

3.6 Részletes összehasonlítás: ugyanaz a feature, két megközelítéssel

Lássuk egy konkrét példán, hogyan különbözik a két megközelítés a gyakorlatban. A feladat: **felhasználói értesítési rendszer** egy meglévő alkalmazáshoz. A felhasználók email és push értesítést kapjanak, ha valami fontos történik a fiókjukban.

3.6.1 A) Vibe coding megközelítés

Idő	Tevékenység	Állapot
0:00	"Csinálj értesítési rendszert"	Indul
0:10	AI generál email küldő kódot	Működni tűnik
0:20	"Push értesítést is kérek"	AI hozzáad
0:35	Az email küldés elromlik	Javítás
0:50	"A felhasználó nem tudja kikapcsolni"	AI hozzáad UI-t
1:10	A kikapcsolás elrontja az email kódot	Újra javítás
1:30	"Legyen logolva, ki mit kapott"	AI nekiáll
1:50	A logolás lassítja az egészet	Teljesítményhiba
2:15	Újabb javítás, újabb mellékhatás	Spirál
2:45	"Kezdjük elölről..."	Újrakezdés
3:30	Második nekifutás, ugyanazok a hibák	Frusztráció
4:00+	Valami "működik", de senki nem biztos benne, hogy minden eset le van fedve	"Kész"(?)

Összesen: 4+ óra, bizonytalan eredmény, nulla dokumentáció.

3.6.2 B) Spec-driven megközelítés

Idő	Tevékenység	Állapot
0:00	PM ír proposalt: miért kell, kinek, milyen értesítések kellenek	Tervezés
0:20	AI generál specifikációt: - Email értesítés (5 típus) - Push értesítés (3 típus) - Felhasználói beállítások - Audit log	Spec kész
0:30	PM review: "A 'rendszer karbantartás' értesítést is add hozzá"	Review
0:35	AI frissíti a spec-et	Spec végleges
0:40	AI generál design-t és feladatlistát	Tasks kész (12 task)
0:45	AI implementál: Task 1-3 (DB séma)	3/12 kész
1:00	AI implementál: Task 4-6 (email)	6/12 kész
1:15	AI implementál: Task 7-9 (push)	9/12 kész
1:30	AI implementál: Task 10-12 (UI, log)	12/12 kész
1:40	Tesztek futnak: mind zöld	Ellenőrizve
1:50	PM review: spec vs. eredmény	KÉSZ

Összesen: ~2 óra, ellenőrzött eredmény, teljes dokumentáció.

A különbség vizuálisan:

```
Hatékonyság az idő függvényében:

Produktivitás
^
|   Vibe ___
|        /   \
|       /     \___         ___
|      /          \       /   \___    (hullámzó, kiszámíthatatlan)
|     /            \     /
|    /              \___/
|   /
|  /
|──────────────────────────────────────> Idő
|
|                     Spec  ___________
|                          /
|                    _____/
|                   /                    (lassú indulás, egyenletes haladás)
|             _____/
|            /
|   ________/
|  /
|──────────────────────────────────────> Idő
```

A vibe coding gyorsan indul, de ahogy növekszik a komplexitás, hullámzik: vannak „jó” időszakok és „újraírás” időszakok. A spec-driven lassan indul (tervezés), de utána egyenletesen, kiszámíthatóan halad.

3.7 A technikai adósság — amit minden PM-nek értenie kell

A szoftverfejlesztésben van egy fontos fogalom, amit PM-ként érdemes ismerni: a **technikai adósság** (angolul: technical debt). Ezt Martin Fowler részletesen tárgyalja a blogján, de röviden:

Technikai adósság az, amikor „gyors megoldást” választunk „jó megoldás” helyett, és később ennek az árát megfizetjük.

Gondolj rá úgy, mint egy hitelre. Ha hitelt veszel fel (gyors megoldás), most több pénzed van, de később kamatot is fizetsz. Ha eléggé eladósodszol, a kamat többe kerül, mint amennyi pénzt kaptál.

Szoftverre lefordítva:

```
Technikai adósság hasonlat:

Gyors megoldás (vibe coding)     =  Hitel felvétel
Karbantartási költség            =  Kamat
Újraírási kényszer               =  Csőd (nem tudod fizetni a kamatot)

Jó megoldás (spec-driven)        =  Önerőből építés
Alacsony karbantartási költség   =  Nincs kamat
Könnyű bővítés                   =  Tartalék a jövőre
```

A vibe coding közvetlenül technikai adósságot termel, mert:

1. **Nincs dokumentáció**: senki nem tudja, miért úgy van megoldva valami, ahogy

2. **Nincs teszt**: senki nem tudja biztosan, hogy egy változtatás nem ront-e el mást
3. **Nincs struktúra**: a kód „nőtt, nem tervezték” — ezért minden változtatás költséges
4. **Nincs konzisztencia**: különböző stílusok, megközelítések, megoldások keverednek

A spec-driven megközelítés nem eliminálja a technikai adósságot — az minden szoftverben felgyülemlik valamilyen mértékben. De **kontrollálja**: a specifikáció és a design dokumentum biztosítja, hogy a döntések tudatosak legyenek, és a későbbi karbantartás ne legyen rejtvény.

Miért érdekli ez a PM-et? Mert a technikai adósság közvetlenül hat a szállítási sebességre. Egy eladósodott kódbázison minden új feature-t lassabban lehet megcsinálni. Az első feature 1 nap. A második 2 nap. A tizedik 2 hét. A PM látja, hogy „egyre lassabbak vagyunk” — de nem érti, miért. Az ok gyakran a felhalmozott technikai adósság.

3.8 A kontextus ablak probléma konkrét számokkal

Hogy jobban értsd a kontextus ablak korlátait, nézzünk egy konkrét példát. Tegyük fel, hogy van egy közepes méretű webes alkalmazás:

```
Tipikus közepes alkalmazás:

Fájltípus                    Darabszám    Átlag sorok    Össz. sorok
──────────────────────────   ─────────    ───────────    ───────────
Backend kód (Python/JS)       40            200           8 000
Frontend komponensek          30            150           4 500
Tesztek                       25            100           2 500
Konfigurációs fájlok          10             50             500
Adatbázis migrációk            8             80             640
Dokumentáció                   5            200           1 000
──────────────────────────   ─────────    ───────────    ───────────
Összesen                      118                        17 140 sor
```

A Claude kontextus ablaka (200K token) elvileg befogadná mind a ~17 000 sort. De a gyakorlatban:

- A beszélgetés előzménye: ~30% kontextus
- Rendszer utasítások (CLAUDE.md, stb.): ~10% kontextus
- Az AI gondolkodása: ~20% kontextus
- **Kód számára megmaradt hely: ~40%, azaz kb. 6 000 sor**

Ez azt jelenti, hogy az AI egyszerre kb. **30-40 fájlt** lát. Egy 118 fájlos projektnek alig az egyharmada. A többi 80 fájlról egyszerűen „nem tud” abban a pillanatban.

Vibe coding-ban ez katasztrófa: ahogy haladunk előre, az AI egyre kevesebb fájlt lát, és egyre több módosítás történik „vakon” — olyan fájlokra hatva, amiket az AI abban a pillanatban nem látott.

Spec-driven megközelítésben **pont ezért bontjuk szét a munkát kis feladatokra**: minden feladatnál az AI csak a releváns 5-10 fájlt látja, és pontosan tudja mit kell tennie (mert ott a spec). Nem kell „mindent a fejében tartani”, elegendő az aktuális feladatot és annak környezetét látnia.

Az Anthropic kutatási anyagai részletesen tárgyalják a nagyméretű kontextus ablakok használatát és korlátait.

3.9 Mikor JÓ a vibe coding?

Fontos hangsúlyozni: a vibe coding nem rossz eszköz — rossz helyen használva az. Vannak helyzetek, ahol tökéletesen megfelel:

Helyzet	Miért jó itt a vibe coding
Prototípus	Gyors validáció, nem kell tartós minőség
Hackathon	Sebesség számít, nem karbantarthatóság
Személyes projekt	Nincs csapat, nincs PM, nincs production
Tanulás	Az AI magyaráz miközben generálja a kódot
Egyszerű script	1-2 fájl, jól körülhatárolt feladat
Ötlet validáció	„Működne ez egyáltalán?” kérdésre válasz
Belső eszközöcske	Amit csak te használsz, és ha elromlik, nem baj
Adatelemzés	Egyszeri lekérdezés, vizualizáció, riport

A kulcs: ha a következő mondatok igazak, a vibe coding rendben van:

1. „Ha holnap törölnöm kell az egészet, nem baj.”
2. „Csak én használom.”
3. „Nincs benne szenzitív adat.”
4. „Nem kell másnak karbantartania.”

Ha bármelyikre „nem” a válasz, érdemes spec-driven megközelítésre váltani.

3.10 Mikor ROSSZ a vibe coding?

Helyzet	Miért rossz itt
Production kód	Nincs spec –> nincs mit tesztelni –> hibák
Csapatmunka	A többiek nem látják mi történt a chat-ben
Komplex rendszer	A kontextus ablak nem elég
Hosszú projekt	Nincs memória, minden session újraindul
Szabályozott iparág	Nincs audit trail, nincs nyomon követhetőség
PM felügyelete alatt	Nem tudod ellenőrizni, mert nincs mérce
Több fejlesztő	Nincs közös referencia, mindenki mást ért
Biztonságkritikus	Nem látod a „szélső esetek” kezelését

3.11 Az összehasonlítás

Szempont	Vibe Coding	Spec-Driven
Indulási sebesség	Azonnali	10-30 perc tervezés
Teljes idő (kicsi projekt)	Gyors	Hasonló
Teljes idő (nagy projekt)	Lassú (újracsinálás)	Gyorsabb
Kód minőség	Változó, kiszámíthatatlan	Konzisztens
Nyomon követhetőség	Nincs (chat history)	Teljes (artifact-ok)
PM rálátás	„Kész van?” „Igen”	Spec –> Tasks –> Progress
Skálázhatóság	1 ember, 1 chat	Több ágens, párhuzamos
Újrafelhasználhatóság	Semmi	Spec újra felhasználható
Review-olhatóság	Chat log átolvasása	Strukturált artifact-ok
Technikai adósság	Gyorsan nő, láthatatlanul	Kontrollált, tudatos
Onboarding	Új ember: „mi ez a kód?”	Új ember: olvassa a spec-et
Karbantartás 6 hónap múlva	„Ki írta ezt és miért?”	Spec + design + tasks

3.12 Miért számít a PM láthatóság?

Egy PM számára a legfontosabb kérdés: **„Hol tartunk?”** És a második: **„Mikorra lesz kész?”**

Vibe coding-ban ezekre a kérdésekre a válasz mindig bizonytalan:

- „Hol tartunk?” –> „Haladunk, most a login-on dolgozunk” (de ez mit jelent pontosan?)
- „Mikorra lesz kész?” –> „Talán péntekre” (de ez becslésen alapul, nem mérésen)

Spec-driven megközelítésben:

- „Hol tartunk?” –> „A tasks.md szerint 18/25 feladat kész, azaz 72%”
- „Mikorra lesz kész?” –> „Napi 5 feladat tempóval ~1.5 nap van hátra”

```
PM láthatóság összehasonlítás:

Vibe Coding:                      Spec-Driven:

PM: "Hogy állsz?"                 PM megnyitja a tasks.md-t:
Dev: "Jó, haladok."              - [x] 1.1 DB séma
PM: "De pontosan hol?"            - [x] 1.2 API endpoint
Dev: "Hát... a login              - [x] 2.1 Email küldés
     nagyjából kész..."           - [x] 2.2 Push értesítés
PM: "És mi van hátra?"            - [ ] 2.3 Felhasználói UI    <-- itt tart
Dev: "Még pár dolog."             - [ ] 3.1 Tesztek
PM: *sóhajt*                      - [ ] 3.2 Dokumentáció

                                  Haladás: 4/7 (57%)
                                  PM: *pontosan látja a helyzetet*
```

Ez nem csak kényelem kérdése. A PM feladata, hogy kommunikáljon a stakeholderek felé: az ügyfélnek, a vezetőségnek, a többi csapatnak. Ha nem látod pontosan, hol tart a projekt, nem tudsz megbízható információt adni. És a megbízható információ a PM legfontosabb eszköze.

3.13 Mikor válik a vibe coding spec-driven-né — a természetes fejlődés

Az érdekesség az, hogy sok csapatnál a vibe coding **természetesen** fejlődik spec-driven megközelítéssé. Ez nem egy bináris váltás („tegnapig vibe, mától spec”), hanem egy fokozatos átmenet:

```
1. fázis:  Vibe coding        "Wow, ez működik!"
              |
              |  Első kudarcok, újraírások
              v
2. fázis:  Vibe + review      "Na jó, de nézze meg valaki"
              |
              |  "Miért ne írjuk le először, mit akarunk?"
              v
3. fázis:  Plan -> vibe       "Előbb gondolkodjunk, aztán kódoljunk"
              |
              |  "Ha már leírjuk, írjuk le rendesen"
              v
4. fázis:  Spec-driven        "Specifikáció -> design -> implementáció"
              |
              |  "Miért nem dolgozik egyszerre több AI ágens?"
              v
5. fázis:  Orchestrated       "Több ágens, párhuzamosan, automatizáltan"
```

Az 1. fázis az, ahol a legtöbb ember ma tart. Felteszik a ChatGPT-t vagy Claude-ot, beszélgetnek vele, és örülnek az eredménynek.

A 2. fázis akkor jön, amikor először történik komoly baj: elveszett munka, rejtett hiba production-ben, vagy egyszerűen a projekt „összeomlik" a komplexitás súlya alatt. Ilyenkor jönnek be az első biztonsági hálók: valaki megnézi a kódot mielőtt production-be kerül.

A 3. fázis az, amikor a csapat rájön, hogy a tervezésre fordított 20 perc 2 óra javítást takarít meg. Már nem „csak úgy" kódolnak, hanem először átbeszélik, mit akarnak. De még nincs formális spec.

A 4. fázis formalizálja azt, amit a 3. fázisban már informálisan csináltak. Innentől van proposal, spec, design, és feladatlista. Az AI nem „beszélgetésből" dolgozik, hanem strukturált dokumentumokból.

Az 5. fázis az, ami ebben a könyvben a következő fejezet témája: több AI ágens dolgozik párhuzamosan, mindegyik a saját részfeladatán, központi koordinációval.

A legtöbb csapat számára a 3-as fázis már hatalmas előrelépés a vibe coding-hoz képest. Nem kell rögtön az 5-ös fázisba ugrani — a lényeg, hogy **a fejlődés iránya egyértelmű**: a struktúra felé.

3.14 Hogyan segít mindez a cégednek?

Ha PM vagy és azt gondolod, „oké, de engem a kódolás technikai részletei nem érdekelnek" — teljesen jogos. De íme, miért érdekel téged is ez a különbség:

1. **Kiszámítható szállítás**: Spec-driven megközelítéssel pontosabb becslést tudsz adni a stakeholdereknek, mert látható a haladás.
2. **Alacsonyabb kockázat**: A specifikáció és a tesztek csökkentik annak az esélyét, hogy production-ben derül ki egy hiba.
3. **Jobb kommunikáció**: A spec, a design, és a tasks.md olyan dokumentumok, amiket nem-technikai emberek is olvashatnak. Nem kell „lefordítani" a fejlesztő szavait.
4. **Onboarding**: Ha új fejlesztő jön a csapatba, olvassa a specifikációt — nem kell három hetet a régi kódban böngészni.
5. **Audit és compliance**: Szabályozott iparágakban (pénzügy, egészségügy) a specifikáció és a design dokumentum audit trail-ként is szolgál.
6. **Hosszú távú költség**: A spec-driven megközelítéssel épített szoftvert olcsóbb karbantartani, mert strukturált és dokumentált.

3.15 Következő lépések

A következő fejezetben a 4. és 5. fázist nézzük meg részletesen: hogyan működik a spec-driven fejlesztés a gyakorlatban (OpenSpec), és hogyan skálázható több ágens párhuzamos munkájával (orkesztráció).

Kulcs üzenet PM-nek

A vibe coding a szoftverfejlesztés fast food-ja: gyors, olcsó, és alkalmi használatra remek. De ha egy céget akarsz belőle felépíteni — szükséged van igazi konyhára, igazi receptekre, és igazi minőségellenőrzésre. A spec-driven fejlesztés adja ezt a struktúrát. Ahogy az építkezésnél sem a téglarakással kezdjük, hanem a tervrajzzal — a szoftverfejlesztésben sem a kóddal kell kezdeni, hanem a specifikációval. A PM feladata nem az, hogy kódot írjon vagy értsen — hanem az, hogy láthatóságot, struktúrát, és kiszámíthatóságot biztosítson. A spec-driven megközelítés pontosan ezt teszi lehetővé.

4. fejezet

OpenSpec – Specifikacio-vezerelt fejlesztes

4.1 A problema ujra

Az elozo fejezetben lattuk, miert nem eleg a vibe coding. De ha nem "chattelunk az AI-val", akkor hogyan mondjuk el neki, mit csinaljon?

A valasz: **ugyanugy, ahogy egy jo PM elmondja egy fejlesztocsapatnak** – specifikacioval, tervvel, es feladatlistaval. Csak eppen nem embereknek, hanem AI agenseknek.

Ez az OpenSpec lenyege: egy strukturalt munkafolyamat, ahol **minden lepes dokumentalt, ellenorizheto, es visszakovetheto**.

Ez nem uj gondolat. Az Agile Manifesto 2001-ben fogalmazta meg, hogy a "mukodo szoftver" fontosabb, mint a "reszletes dokumentacio". Ez igaz – de az AI agensek vilagaban a mukodo szoftver eppen a jo dokumentaciobol szuletik. Az OpenSpec ezt az egyensulyt keresi: **annyi dokumentacio, amennyi kell – se tobb, se kevesebb**.

4.2 Az OpenSpec es az ismert PM eszkozok – forditorozsetta

Mielott belemegyunk a reszletekbe, erdemes megerteni, hogyan viszonyulnak az OpenSpec fogalmak ahhoz, amit mar ismersz. Ha dolgoztol Jira-val, Confluence-szel, vagy barmilyen PM eszkozzel (Linear, Notion, Asana), a kovetkezo tablazat segit osszekotni a ket vilagot:

OpenSpec artifact	Ismert PM megfeleloje	Peldak
Proposal	PRD (Product Requirements Document), Product Brief	Amit a PM vagy stakeholder ir: miert kell ez, kinek, mi valtozik
Spec	User Story-k + Acceptance Criteria, BDD scenariok	"Felhasznalokent szeretnem visszaallitani a jelszavam, hogy…"
Design	Technical Design Document, ADR (Architecture Decision Record)	Technikai dontesek es indoklasaik
Tasks	Sprint Backlog, Jira Sub-tasks	Checkboxos lista, amit az AI "pipalgat"
Implementation	A sprint maga – a fejlesztes	Az AI ir kodot, commitol, tesztel

A kulonbseg: a hagyomanyos eszkozoknel ezek **kulonallo rendszerekben** elnek (Jira-ban a ticket, Confluence-ben a spec, Slack-en a dontesek, a fejleszto fejeben a terv). Az OpenSpec-ben **minden egy helyen van, egymast epiti, es a kod mellett el a git-ben**.

Ha eddig Atlassian eszkozokkel dolgoztol, gondolj ugy az OpenSpec-re, mint egy "all-in-one" megkozelitesre, ahol a Jira ticket, a Confluence oldal, es a fejlesztes egybeforrva letezik.

4.3 A pipeline – az otlettol a kodig

Az OpenSpec egy lancot epit fel, ahol minden lepes a kovetkezo alapja:

```
+-------------------------------------------------------------+
|                    OpenSpec Pipeline                        |
|                                                             |
|   +----------+                                              |
|   | PROPOSAL |  <-- "Mit es miert?"                         |
|   |          |    PM/stakeholder nyelven                    |
|   +----+-----+                                              |
|        |                                                    |
|        | .......... PM review gate ..........               |
|        v                                                    |
|   +----------+                                              |
|   |  SPECS   |  <-- "Pontosan mit kell tudnia?"             |
|   |          |    Merheto kovetelmenyek                     |
|   +----+-----+                                              |
|        |                                                    |
|        | .......... PM review gate ..........               |
|        v                                                    |
|   +----------+                                              |
|   |  DESIGN  |  <-- "Hogyan epitjuk meg?"                   |
|   |          |    Technikai dontesek                        |
|   +----+-----+                                              |
|        |                                                    |
|        | .......... Dev review gate .........               |
|        v                                                    |
|   +----------+                                              |
|   |  TASKS   |  <-- "Mi a teendo lista?"                    |
|   |          |    Checkboxos feladat lista                  |
|   +----+-----+                                              |
|        |                                                    |
|        v                                                    |
|   +--------------+                                          |
|   |IMPLEMENTATION|  <-- Az AI itt kodol                     |
|   |              |    Feladatrol feladatra halad            |
|   +--------------+                                          |
|                                                             |
|   Minden lepes: markdown fajl, git-ben tarolva,             |
|   barki altal olvashato es review-olhato                    |
+-------------------------------------------------------------+
```

Figyeld meg a "review gate"-eket a diagramon. Ezek az a pontok, ahol **ember – te, a PM – ellenorzi es jovahagyja** a kovetkezo lepes elindulasat. Errol bovebben lentebb.

Nezzuk meg egyenkent, mit tartalmaz minden lepes.

4.4 1. Proposal – "Mit es miert?"

A proposal (javaslat) az egyetlen dokumentum, amit idealis esetben **ember ir** – te, a PM, vagy a stakeholder. Ez mondja el, miert van szukseg erre a valtozasra.

A proposal a klasszikus PRD (Product Requirements Document) egyszeru, tomor megfeleloje. Nem kell 20 oldalas dokumentumot irnod – 1-2 oldal eleg. A lenyeg: **kontextust adsz az AI-nak**, hogy megertse a problema hatteret.

4.4.1 Miert fontos a "miert?"

Martin Fowler, a szoftverfejlesztes egyik legismertebb gondolkodoja irja: a legjobb specifikaciok nem azzal kezdodnek, *mit* kell epiteni, hanem azzal, *miert*. Ha az AI (vagy egy fejleszto) erti a miert-et, jobb donteseket hoz a reszletekben.

Pelda: Tegyuk fel, hogy felhasznaloi visszajelzesek alapjan kell egy jelszo-visszaallitasi funkcio. Ime egy reszletes proposal:

```
+-------------- proposal.md ----------------------+
|                                                 |
|  # Jelszo-visszaallitas funkcio                 |
|                                                 |
|  ## Hatter es motivacio                         |
|                                                 |
|  A felhasznalok 15%-a hetente elfelejti         |
|  a jelszavat. Jelenleg nincs lehetoseg          |
|  onkiszolgalo visszaallitasra -- a support      |
|  csapat manualisan reseteli. Ez heti 40         |
|  support ticket, ami evi ~2000 ticket,          |
|  atlagosan 15 perc/ticket = 500 ora/ev          |
|  support koltseg.                               |
|                                                 |
|  ## Celcsoport                                  |
|                                                 |
|  Minden regisztralt felhasznalo (jelenleg       |
|  ~12.000 aktiv felhasznalo).                    |
|                                                 |
|  ## Sikerkritérium                              |
|                                                 |
|  - Support ticketek szama 80%-kal csokken       |
|    (heti 40 -> heti 8)                          |
|  - A felhasznalok 90%-a sikeres onkiszolgalo    |
|    visszaallitast vegez                         |
|  - A rendszer 99.9% rendelkezesre allassal      |
|    mukodik                                      |
|                                                 |
```

```
|  ## Mi valtozik                                  |
|                                                  |
|  - Email alapu jelszo-visszaallitas              |
|  - Token alapu biztonsagi link                   |
|  - Jelszo erosség ellenorzes                     |
|  - Audit log minden visszaallitasrol             |
|                                                  |
|  ## Erintett teruletek                           |
|                                                  |
|  - Auth modul                                    |
|  - Email kuldes                                  |
|  - Felhasznaloi felulet                          |
|  - Admin dashboard (uj riport)                   |
|                                                  |
|  ## Amit NEM tartalmaz (scope hatarok)           |
|                                                  |
|  - Kétfaktoros hitelesites (kulon projekt)       |
|  - SMS alapu visszaallitas (kovetkezo fazis)     |
|  - Social login (kulon proposal)                 |
|                                                  |
+--------------------------------------------------+
```

PM szemmel: A proposal az, amit eddig is irtal – egy rovid leiras a valtozasrol, uzleti indoklassal. Semmi technikai zsargon. De figyeld meg a ket uj szekciót: a **Sikerkritérium** (honnan tudjuk, hogy sikeres volt?) es az **Amit NEM tartalmaz** (mi az, amihez nem nyulunk). Ez a ket szekció rengeteg felreertest eloz meg – mind az AI, mind a csapat fele.

Tipp: A Shape Up metodologia – amit a Basecamp fejlesztett ki – ugyanezt javasolja: minden projekt elinditasa elott határozd meg, mi az "appetite" (mennyi idot akarsz raszanni) es mi a "no-go" (amihez biztosan nem nyulsz). A proposalban a scope hatarok pontosan ezt a ceelt szolgaljak.

4.5 2. Specs – "Pontosan mit kell tudnia?"

A specifikacio a proposal-bol **merheto kovetelményeket** csinal. Ezt altalaban az AI generalja a proposal alapjan, de a PM review-olja.

A specifikacio formatuma a BDD (Behaviour-Driven Development) szcenariokra hasonlit: "HA ez tortenik, AKKOR ez legyen az eredmeny". Ez a formatum azert hasznos, mert **minden forgatokonyv kozvetlenul tesztelhetove valik** – az AI teszt kodot tud generalni belole.

```
+-------------- spec.md --------------------------------+
|                                                       |
|  # Jelszo-visszaallitas -- Specifikacio               |
|                                                       |
|  ## 1. Kovetelmeny: Visszaallitas keres               |
|                                                       |
|  A felhasznalo a bejelentkezo oldalon kepes           |
|  jelszó-visszaallitast kezdemenyezni az email         |
|  cime megadasaval.                                    |
|                                                       |
|  ### 1.1 Forgatokonyv: Sikeres keres                  |
```

```
|  - HA a felhasznalo megadja az email cimet              |
|  - ES az email cim letezik a rendszerben                |
|  - AKKOR a rendszer kuld egy linket                     |
|    ami 24 oraig ervenyes                                |
|  - ES a felhasznalo sikeres visszajelzest kap           |
|    ("Ellenorizd az emailjeidet")                        |
|                                                         |
|  ### 1.2 Forgatokonyv: Nem letezo email                 |
|  - HA az email cim nincs a rendszerben                  |
|  - AKKOR a rendszer UGYANAZT a visszajelzest            |
|    mutatja (biztonsagi okokbol)                         |
|  - ES NEM kuld emailt                                   |
|  - ES NEM jelzi, hogy a cim nem letezik                 |
|                                                         |
|  ### 1.3 Forgatokonyv: Tul sok keres                    |
|  - HA a felhasznalo 5-nel tobb kerest kuld              |
|    1 oran belul                                         |
|  - AKKOR a rendszer 1 orara blokkolja                   |
|    a további kereséseket                                |
|  - ES naplozza a gyanús tevékenysget                    |
|                                                         |
|  ## 2. Kovetelmeny: Token hasznalat                      |
|                                                         |
|  ### 2.1 Forgatokonyv: Ervenyes token                   |
|  - HA a felhasznalo a kapott linkre kattint             |
|  - ES a token 24 oran belul van                         |
|  - AKKOR uj jelszo megadasa oldal jelenik meg           |
|                                                         |
|  ### 2.2 Forgatokonyv: Lejart token                     |
|  - HA a felhasznalo 24 ora utan kattint                 |
|  - AKKOR hibauzenet es uj keres lehetoseg              |
|                                                         |
|  ### 2.3 Forgatokonyv: Mar hasznalt token               |
|  - HA a token mar egyszer felhasznalasra kerult         |
|  - AKKOR hibauzenet: "Ez a link mar felhasznalt"        |
|                                                         |
|  ## 3. Kovetelmeny: Uj jelszo beallitas                  |
|                                                         |
|  ### 3.1 Forgatokonyv: Ervenyes uj jelszo               |
|  - HA az uj jelszo megfelel a szabalyoknak              |
|    (min. 8 karakter, szam, nagybetu)                    |
|  - AKKOR a jelszo megvaltozik                           |
|  - ES minden aktiv session kijelentkezik                |
|  - ES a felhasznalo ertesito emailt kap                 |
|                                                         |
|  ### 3.2 Forgatokonyv: Gyenge jelszo                    |
|  - HA az uj jelszo nem felel meg a szabalyoknak         |
|  - AKKOR real-time visszajelzes mutatja                 |
|    melyik szabaly nem teljesul                          |
```

```
|                                                          |
+----------------------------------------------------------+
```

PM szemmel: A spec az, amit eddig a fejlesztovel egyeztettel szoban – de most le van irva, visszakeresheto, es az AI is erti. Minden "forgatokonyv" egy potencialis teszt eset. Amikor review-olod, a kovetkezo kerdeseket tedd fel magadnak:

1. **Teljes-e?** – Van-e olyan felhasznaloi szituacio, ami kimaradt?
2. **Ertelmes-e?** – Ertened-e mint felhasznalo, mit csinal a rendszer?
3. **Tesztelheto-e?** – Minden forgatokonyvnek van egyertelmu "AKKOR" resze?
4. **Konzisztens-e?** – Nincsenek-e ellentmondasok a forgatokonyvek kozott?

A specifikacio-review a PM legfontosabb minoseg-ellenorzo eszkoze. Ha a spec jo, a kod jo lesz. Ha a spec hibas, a kod is hibas lesz – de legalabb tudjuk, *miert*.

4.6 3. Design – "Hogyan epitjuk meg?"

A design dokumentum a technikai donteseket rogziti. Ezt az AI kesziti, de a senior fejleszto review-olja.

A design az ugynevezett ADR (Architecture Decision Record) formátumot koveti – ez a ThoughtWorks Tech Radar altal is javasolt bevalt gyakorlat. Minden technikai dontes egy onallo bejegyzés, ami tartalmazza a dontest, az alternativakat, es az indoklast.

```
+-------------- design.md ----------------------------------+
|                                                           |
|  # Jelszo-visszaallitas -- Design                         |
|                                                           |
|  ## Architekturalis dontesek                              |
|                                                           |
|  ### ADR-001: Token generalas                             |
|  Statusz: Elfogadva                                       |
|  Dontes: Veletlenszeru 256-bites token                    |
|  Kontextus: Biztonsagos, egyedi tokent kell               |
|    generalni minden visszaallitasi kereshez.              |
|  Alternativa 1: JWT token -- elvetettuk, mert             |
|    nem vonhato vissza (stateless).                        |
|  Alternativa 2: Szamlalo alapu ID -- elvetettuk,          |
|    mert kitalalhato (brute force tamadas).                |
|  Kovetkezmeny: Token tablat kell letrehozni               |
|    az adatbazisban, lejarat mezovel.                      |
|                                                           |
|  ### ADR-002: Email szolgaltatas                           |
|  Statusz: Elfogadva                                       |
|  Dontes: A meglevo SendGrid integraciot                   |
|    hasznaljuk.                                            |
|  Kontextus: Mar van SendGrid fiokunk, nincs               |
|    szukseg uj szolgaltatasra.                             |
|  Alternativa: Sajat SMTP -- tul komplex                   |
|    ehhez a feladathoz.                                    |
|  Kovetkezmeny: A meglevo email modulra epitunk.           |
|                                                           |
```

```
|  ### ADR-003: Session kezelés                                |
|  Statusz: Elfogadva                                         |
|  Dontes: Jelszo valtoztatas utan minden                     |
|    aktiv session lezarasra kerul.                            |
|  Kontextus: Ha valaki jogosulatlanul hasznalta              |
|    a fiokot, a jelszo valtoztatas utan sem                  |
|    maradhat bejelentkezve.                                  |
|  Alternativa: Csak az aktualis session marad --             |
|    biztonsagi kockazat.                                     |
|                                                             |
|  ## Kockazatok                                              |
|                                                             |
|  - [Kockazat] Email deliverability                          |
|    -> Spam filterek blokkolhatjak                           |
|    -> Megoldas: SPF/DKIM beallitas                          |
|    -> Felelős: DevOps csapat                                |
|                                                             |
|  - [Kockazat] Token brute force                             |
|    -> Rosszindulatu felhasznalo probalgathatja              |
|       a tokeneket                                           |
|    -> Megoldas: Rate limiting + 256-bites                   |
|       token (gyakorlatilag kitalalhatatlan)                 |
|                                                             |
|  - [Kockazat] Email keses                                   |
|    -> A felhasznalo nem kapja meg ideben                    |
|    -> Megoldas: "Nem kaptad meg?" ujrakuldo                 |
|       gomb + 2 perc varakozasi ido utana                    |
|                                                             |
+-------------------------------------------------------------+
```

PM szemmel: Nem kell ertened a technikai reszleteket. De ket dolgot erdemes figyelni: - **Dontesek**: Vannak alternativak feltuntetve? (Ha igen, atgondolt dontes volt. Ha nem, kerd meg az AI-t, hogy mutassa meg az alternativakat.) - **Kockazatok**: Vannak megoldasi javaslatok mellettuk? (Ha igen, felkeszultek. Ha nincs megoldasi javaslat, az piros flag – kerdezz ra.)

Az ADR formatum egyik nagy elonye, hogy **a dontesek visszakovethetek**. Ha 3 honap mulva felmerul a kerdes "miert nem JWT-t hasznalunk?", nem kell senki fejeben turkalni – ott van a design.md-ben, indoklassal egyutt.

A GitLab fejlesztoi kezikonyve hasonlo megkozelitest kovet: minden architekturalis dontest dokumentalnak, es a dokumentum a kod mellett el. Ez nem burokracia – ez **intezmenyi memoria**.

4.7 4. Tasks – "Mi a teendo lista?"

A tasks.md a feladatok checkboxos listaja. Ez az AI munkalapja – feladatrol feladatra halad, es bejeloli ami kesz.

```
+-------------- tasks.md --------------------------------+
|                                                         |
|  ## 1. Adatbazis                                        |
|                                                         |
```

```
|  - [x] 1.1 Token tabla letrehozasa                      |
|  - [x] 1.2 Token lejarat mezo hozzaadasa                |
|                                                         |
|  ## 2. Backend                                          |
|                                                         |
|  - [x] 2.1 Reset endpoint implementalas                 |
|  - [x] 2.2 Token generalas logika                       |
|  - [ ] 2.3 Email kuldes integracio            <-- itt   |
|  - [ ] 2.4 Rate limiting (5 keres/ora)                  |
|                                                         |
|  ## 3. Frontend                                         |
|                                                         |
|  - [ ] 3.1 Reset keres form                             |
|  - [ ] 3.2 Uj jelszo form                               |
|  - [ ] 3.3 Jelszo-erosség indikator                     |
|                                                         |
|  ## 4. Biztonsag                                        |
|                                                         |
|  - [ ] 4.1 Session invalidalas jelszo valtoztatáskor     |
|  - [ ] 4.2 Audit log implementalas                      |
|                                                         |
|  ## 5. Teszteles                                        |
|                                                         |
|  - [ ] 5.1 Unit tesztek                                 |
|  - [ ] 5.2 E2E tesztek                                  |
|  - [ ] 5.3 Biztonsagi tesztek (brute force)             |
|                                                         |
|  Haladas: 4/13 kesz (31%)                               |
|                                                         |
+---------------------------------------------------------+
```

PM szemmel: Ez a tasks.md a projektmenedzsment aranybanyaja. Egyetlen fajl, ami megmondja: - **Mi van kesz** es **mi nincs** – pontosan, valos idoben - **Mennyi maradt** – szazalekos haladas - **Mi a kovetkezo** – az elso bepipálatlan feladat - **Milyen kategoriak vannak** – adatbazis, backend, frontend, biztonsag, teszteles

Nem kell a fejlesztot kerdezned: "Hogy allsz?" Megnyitod a tasks.md fajlt, es latod.

Ha megszoktad a Linear vagy Jira sprint board-ot, gondolj a tasks.md-re ugy, mint annak a szoveges valtozatara – csak ezt nem kell manuálisan frissíteni, mert az AI automatikusan bepipalja a kesz feladatokat.

4.8 A review gate – a PM mint minosegkapu

Az OpenSpec pipeline legfontosabb eleme nem maga a dokumentum, hanem a **review gate-ek** – azok a pontok, ahol ember ellenorzi es jovahagyja a kovetkezo lepes elindulasat.

```
+---------------------------------------------------------+
|                  Review Gate-ek                          |
|                                                         |
|  PROPOSAL (ember irja)                                  |
|       |                                                 |
|       v                                                 |
```

```
|  [=== GATE 1: PM review ===]                              |
|  Kerdesek:                                                |
|  - Teljes-e a hatter es motivacio?                        |
|  - Van sikerkritérium?                                    |
|  - Vilagosak a scope hatarok?                             |
|       |                                                   |
|       v                                                   |
|  SPEC (AI generalja)                                      |
|       |                                                   |
|       v                                                   |
|  [=== GATE 2: PM review ===]                              |
|  Kerdesek:                                                |
|  - Minden felhasznaloi szituacio le van fedve?            |
|  - A forgatokonyvek tesztelhetoek?                        |
|  - Nincs ellentmondas?                                    |
|       |                                                   |
|       v                                                   |
|  DESIGN (AI generalja)                                    |
|       |                                                   |
|       v                                                   |
|  [=== GATE 3: Dev review ===]                             |
|  Kerdesek:                                                |
|  - Van alternativa minden dontesnel?                      |
|  - A kockazatok kezelve vannak?                           |
|  - A PM: van-e uzleti kockazat?                           |
|       |                                                   |
|       v                                                   |
|  TASKS -> IMPLEMENTATION                                  |
|       |                                                   |
|       v                                                   |
|  [=== GATE 4: Vegso review ===]                           |
|  Kerdesek:                                                |
|  - Minden task bepipálva?                                 |
|  - Tesztek futnak?                                        |
|  - A spec-nek megfelel az eredmeny?                       |
|                                                           |
+-----------------------------------------------------------+
```

Minden gate-nel **a PM donti el, hogy tovabbmegy-e a folyamat**. Ez nem lassitja le a munkat – egy spec review 10-20 perc. De megelozi, hogy az AI rossz iranyba induljon el, es orak munkaját kelljen kidobni.

Ez a megkozelftes hasonlit a Shape Up metodologia "betting table" koncepciojahoz: a csapat nem indit el mindent, ami felmerul, hanem **tudatosan dont, mi az, ami erdemes a befektett idore**. A review gate ugyanez mikro-szinten: minden fazis elott tudatos dontes szuletik.

4.9 A PM szerepe az OpenSpec munkafolyamatban

A legfontosabb kerdes: hol vagy te ebben a folyamatban?

```
+-------------------------------------------------------+
```

```
|                                                        |
|  PM ir --------> PROPOSAL ------> AI generalja ----> |
|                  PM review                             |
|                                                        |
|  AI generalja --> SPECS --> PM review --------------> |
|                   "Igen, ezt akarom"                   |
|                    vagy                                |
|                   "Nem, ez hiányzik: ..."              |
|                                                        |
|  AI generalja --> DESIGN --> Dev review -------------> |
|                   PM: dontesek es kockazatok OK?       |
|                                                        |
|  AI generalja --> TASKS ---> AI implemental ---------> |
|                   PM: haladas nyomon kovetese          |
|                                                        |
|  AI implemental --> KOD ---> Dev + PM review -------> |
|                    Tesztek futnak?                      |
|                    Spec-nek megfelel?                   |
|                                                        |
+--------------------------------------------------------+
```

Osszefoglalva a PM teendoit:

Fazis	PM feladata	Idoigeny
Proposal	Megirja	15-30 perc
Specs	Review-olja: teljes-e, pontos-e?	10-20 perc
Design	Atfutja: vannak-e kockazatok?	5-10 perc
Tasks	Nyomon koveti a haladast	Folyamatos, passziv
Implementation	Vegeredmenyt ellenorzi	Review-nkent 10-20 perc

A teljes PM idoráforditas egy feature-re: 1-2 ora, szemben a korabbi napokkal amit egyeztetesre, status meetingekre, es kerdesek megvalaszolasara forditott.

4.10 Egy PM napja az OpenSpec-kel – gyakorlati peldak

Hogyan nez ki egy tipikus munkanap, ha OpenSpec-et hasznalsz? Ime egy valos szcenario:

```
+------------------------------------------------------------+
|  PM napirend -- OpenSpec munkafolyamat                     |
|                                                            |
|  09:00  Reggeli attekintes                                 |
|         - Megnyitod a futo change-ek tasks.md fajljait     |
|         - Megnezed: mi keszult el tegnapota                |
|         - Halvanyodik: 60% -> 78% (jo utemben halad)       |
|                                                            |
|  09:30  Uj proposal iras                                   |
|         - A stakeholder tegnap kerte az export funkciót    |
|         - 20 percben megírod a proposal.md-t               |
|         - Elinditod: az AI generalja a specifikaciot       |
|                                                            |
|  10:00  Spec review                                        |
```

```
|         - Az AI elkeszitette a spec.md-t az export         |
|           funkciohoz                                       |
|         - Vegigolvasod: 8 forgatokonyv, 3 kovetelmeny      |
|         - Eszrevetel: "hiányzik a CSV export, csak         |
|           XLSX van" -> visszakuldod javitasra               |
|         - 15 perc munka                                    |
|                                                            |
|  11:00  Stakeholder meeting                                |
|         - A jelszó-visszaallitas feature 78%-ban kesz      |
|         - Megmutatod a tasks.md-t: "13-bol 10 task kesz,   |
|           a teszteles van hatra"                           |
|         - Nem "erzesre" mondod az allapotat -- szamok      |
|           vannak                                           |
|                                                            |
|  14:00  Design review                                      |
|         - Az export funkcio design.md-je elkeszult          |
|         - Atfutod a donteseket: az AI streaming CSV-t      |
|           javasol nagy fajlokhoz -- jol hangzik             |
|         - Kockazat: memoriahaszanlat nagy exportnal         |
|           -> van megoldas mellette -> OK                    |
|         - 10 perc munka                                    |
|                                                            |
|  15:00  Vegso review                                       |
|         - A jelszó-visszaallitas 100%: minden task kesz    |
|         - Vegnezed az eredmenyt: a spec forgatokonyvei     |
|           alapjan ellenorzod                               |
|         - "Lejart token -> hibauzenet" -- OK               |
|         - "Rate limiting -> blokkol 5 keres utan" -- OK    |
|         - Jovahagyod a merge-et                            |
|                                                            |
|  16:00  Riportolas                                         |
|         - A nap vegen osszegzed: 1 feature lezarva,        |
|           1 uj feature specifikacios fazisban               |
|         - A kovetkezo napi terv: spec review az            |
|           export funkciohoz                                |
|                                                            |
+------------------------------------------------------------+
```

Figyeld meg: a PM napjanak nagy resze **olvasas es dontes**, nem koordinacio es statuszkerdezgetes. Ez az OpenSpec legnagyobb elonye a PM szamara.

4.11 Hogyan nez ki ez a gyakorlatban?

Lassunk egy valos peldat a wt-tools projektbol. Egy uj funkciot akartunk hozzaadni: "az AI agensek kuldjenek uzeneteket egymasnak".

1. lepes: Proposal megirasa (5 perc) > *"A parhuzamosan dolgozo AI agensek nem tudnak kommunikalni egymassal. Ha az egyik agens felfedez valamit, ami a masik munkajat erinti, nincs modja szolni. Megoldas: egyszeru uzenetküldo rendszer."*

2. lepes: Az AI geneval specifikaciot (automatikus) > *Kovetelmenyek: uzenet kuldes agens -> agens, inbox lekerdezes, broadcast uzenet mindenkinek, uzenetek szinkronizálasa gepek kozott.*

3. lepes: PM review – "A broadcast jo otlet, viszont legyen timestamp is" -> specifikacio frissul

4. lepes: Design generalas (automatikus) > *Dontes: fajl alapu rendszer, nincs szerver szukseges. Uzenetek JSON sornonkent egy fajlban, a meglevo git sync viszi gepek kozott.*

5. lepes: Tasks generalas -> 8 feladat, checkboxos lista

6. lepes: Implementacio -> az AI dolgozik, PM nezi a haladast a tasks.md-ben

Eredmeny: 3 ora alatt kesz egy funkcio, ami hagyomanyos modon 2-3 nap lett volna. Es minden lepes dokumentalva van, visszakeresheto.

4.12 Visszakovethetos – a proposal-tol a kodig

Az OpenSpec egyik legfontosabb tulajdonsaga a **teljes visszakovethetoseg** (traceability). Minden sor kod visszavezetheto egy feladatra, minden feladat egy specifikaciohoz, minden specifikacio egy proposal-hoz.

```
+-------------------------------------------------------------+
|  Visszakovethetosegi lanc                                   |
|                                                             |
|  Proposal: "A felhasznalok 15%-a elfelejti a jelszavat"    |
|       |                                                     |
|       +---> Spec 2.3: "Mar hasznalt token -> hibauzenet"   |
|                 |                                           |
|                 +---> Task 4.2: "Audit log implementalas"   |
|                           |                                 |
|                           +---> Commit: "feat: add          |
|                                 password reset audit log"   |
|                                    |                        |
|                                    +---> Fajl:              |
|                                    src/auth/audit.ts        |
|                                                             |
|  Ha barki megkerdezi: "Miert van ez a kod itt?"             |
|  A valasz: vegigkoveted a lancot a proposal-ig.             |
+-------------------------------------------------------------+
```

Ez kulonosen fontos szabalyozott iparagakban (penzugy, egeszsegugy, kormányzat), ahol auditalhatosag szukseges. De akar egy startup-nal is hasznos: 6 honap mulva nem kell talalgatnod, miert van egy adott funkcio a rendszerben.

A hagyomanyos fejlesztesben ez a lanc altalaban **fejekben el** – es amikor valaki tavozik a csapatbol, a tudas elveszik. Az OpenSpec-ben a lanc **fajlokban el**, a kod mellett, a git-ben.

4.13 Mi van, ha a spec hibas? – Iteraciok es frissitesek

A valosag az, hogy **a legjobb specifikacio sem tokeletes**. Fejlesztes kozben derulhetnek ki uj szempontok, a stakeholder meggondolhatja magat, vagy a technikai korlatokkal szembesulve modositani kell.

Az OpenSpec erre is felkeszult:

```
+-------------------------------------------------------------+
```

```
|  Spec iteracio munkafolyamat                                |
|                                                             |
|  Eredeti spec                                               |
|       |                                                     |
|       v                                                     |
|  Implementacio soran felfedezunk valamit:                   |
|  "A SendGrid API nem tamogatja a sablon                     |
|   nyelvet, amit terveztunk"                                 |
|       |                                                     |
|       v                                                     |
|  Lehetosegek:                                               |
|                                                             |
|  A) Spec frissites -- a kovetelmeny modosul                 |
|     -> spec.md-ben uj verzio, git commit-ban lathato        |
|     -> tasks.md frissul (uj/modositott feladatok)           |
|     -> PM review: "OK, ez elfogadhato kompromisszum"        |
|                                                             |
|  B) Design frissites -- a megvalositas modosul              |
|     -> design.md-ben uj ADR bejegyzes                       |
|     -> a spec valtozatlan marad                             |
|     -> Dev review: "OK, ez jobb megoldas"                   |
|                                                             |
|  C) Proposal frissites -- az scope modosul                  |
|     -> proposal.md-ben uj "Amit NEM tartalmaz" bejegyzes    |
|     -> PM + stakeholder review                              |
|     -> Dontes: "Ezt kiemeljuk kulon feature-be"             |
|                                                             |
+-------------------------------------------------------------+
```

A lenyeg: **minden valtozas dokumentalt**. A git tortenete megmutatja, mi volt az eredeti spec, es hogyan modosult. Nem veszik el semmi – a korabbi verziok mindig visszanezhetoek.

Ez kulonbozik a hagyomanyos megkozelitestol, ahol a specifikacio-valtozasok gyakran szobeli megallapodasok ("egyeztettunk Marcieval, hogy ez nem kell"). Az OpenSpec-ben minden modositas egy commit, ami lathato, review-olhato, es visszavonhato.

4.14 Osszehasonlitas: Waterfall vs Agile vs OpenSpec

Ha ismered a klasszikus projekt-modszertanokat, erdemes latni, hogyan viszonyul hozzajuk az OpenSpec:

```
+-------------------------------------------------------------+
|  Harom megkozelites osszehasonlitasa                        |
|                                                             |
|  WATERFALL (hagyományos)                                    |
|  +------+------+------+------+------+                       |
|  | Kove-| Terv | Impl.| Teszt| Atad.|                       |
|  | telm.|      |      |      |      |                       |
|  +------+------+------+------+------+                       |
|  |----- 6-12 honap ------------------>                      |
|  Jellemzo: Minden elore megtervezett.                       |
```

```
|  Hatrany: Keson derul ki, ha rossz iranyba ment.          |
|                                                           |
|  AGILE / SCRUM                                            |
|  +----+ +----+ +----+ +----+ +----+                       |
|  | S1 | | S2 | | S3 | | S4 | | S5 |                      |
|  +----+ +----+ +----+ +----+ +----+                       |
|  |-2hét-|-2hét-|-2hét-|-2hét-|-2hét->                    |
|  Jellemzo: Kis iteraciok, folyamatos visszajelzes.        |
|  Hatrany: Gyakran hiányzik a dokumentacio,                |
|  a tudas a fejleszto fejeben marad.                       |
|                                                           |
|  OPENSPEC                                                 |
|  +--------+    +--------+    +--------+                  |
|  |Proposal|    |Proposal|    |Proposal|                  |
|  |Spec    |    |Spec    |    |Spec    |                  |
|  |Design  |    |Design  |    |Design  |                  |
|  |Tasks   |    |Tasks   |    |Tasks   |                  |
|  |Impl.   |    |Impl.   |    |Impl.   |                  |
|  +--------+    +--------+    +--------+                  |
|  |--orak--| -> |--orak--| -> |--orak--|-->               |
|  Jellemzo: Teljesen dokumentalt, AI vegrehajtja,          |
|  orak alatt kesz ami regen hetekbe telt.                  |
|  Hatrany: A proposal minosege meghatarozo.                |
|                                                           |
+-----------------------------------------------------------+
```

Szempont	Waterfall	Agile/Scrum	OpenSpec
Tervezes	Mindent elore	Sprint elejen	Feature elejen
Dokumentacio	Tulzott	Minimalis	Pont eleg
Visszajelzes	Keso (honapok)	Gyors (2 het)	Azonnali (orak)
Ki implemental	Fejleszto	Fejleszto	AI agens
PM lathatos	Merfoldko riport	Standup + board	tasks.md (valos ideju)
Valtozas kezelese	Nehezkes (CR)	Sprint hataron	Spec update + ujrafutas
Szallitas	Egy nagy release	Sprint vegeken	Folyamatosan

Az Agile Manifesto negy alapelve kozul harom kozvetlenul vonatkozik az OpenSpec-re:

1. **"Mukodo szoftver a reszletes dokumentacio helyett"** – Az OpenSpec pontosan annyi dokumentaciot keszit, amennyit a mukodo szoftver eloallitasa igenyel, se tobbet, se kevesebbet.
2. **"Egyuttmukodes az ugyfellel a szerzodeses targyalas helyett"** – A PM folyamatosan review-olja es igazitja az iranyt, nem egyszer ad meg mindent elore.
3. **"Valtozasra valaszolas a terv kovetese helyett"** – A spec barmiktor frissitheto, es az AI az uj spec alapjan dolgozik tovabb.

4.15 Az OpenSpec es a Shape Up metodologia

A Shape Up – a Basecamp altal kifejlesztett termekfejlesztesi modszertan – sok szempontbol az OpenSpec szellemi elode. Erdemes latni a parhuzamokat:

Shape Up fogalom	OpenSpec megfeleloje
Shaping (a problema "formaba ontese")	Proposal iras

Shape Up fogalom	OpenSpec megfeleloje
Appetite (mennyi idot szanunk ra)	A proposal scope hatarai
Pitch (rovid javaslat, amit elfogadnak/elutasitanak)	A proposal maga
Betting table (dontes, mi az, amire "fogadunk")	PM review gate
Hill chart (haladas vizualizalasa)	tasks.md szazalekos haladas
Cool down (pihenoszunet ciklusok kozott)	Review es iteracios idoszak

A Shape Up legfontosabb tanítása: **a projekt sikeret a shaping (formázás) fazis minosege hatarozza meg**. Ha jol "formázod" a problemat, a megvalositás egyszeru. Ha rosszul, semmilyen mennyisegu fejlesztoi munka nem menti meg.

Az OpenSpec-ben ugyanez igaz: **a proposal minosege meghatarozo**. Egy jo proposal-bol jo spec szuletik, abbol jo design, abbol jo kod. Egy rossz proposal-bol kaotikus specifikacio lesz, es az AI rossz iranyba halad.

4.16 Az OpenSpec es a hagyomanyos PM eszkozok

Talan felmerul: "Miben mas ez, mint a Jira?"

Szempont	Jira / hagyomanyos	OpenSpec
Ki irja a ticketet?	PM	PM (proposal)
Ki bontja feladatokra?	Dev + PM meeting	AI automatikusan
Hol van a spec?	Confluence (talaan)	A kod mellett, git-ben
Hol van a haladas?	Jira board	tasks.md (valos ideju)
Ki frissiti?	Dev (ha emlekszik)	AI automatikusan
Review-olhato?	Board szintjen	Artifact szintjen
Visszakovetheto?	Ticket linkek (ha vannak)	Teljes lanc (proposal -> kod)
Valtozas tortenete?	Ticket kommentek	Git history (minden verzio)

Az OpenSpec nem helyettesiti a Jirat. De kiegesziti: a Jira ticket-bol lesz egy proposal, abbol specifikacio, abbol feladatlista, abbol kod. Minden osszefugg, minden visszakovetheto.

A gyakorlatban sok csapat megtartja a Jira-t (vagy a Linear-t, Notion-t) a magasabb szintu portfolio menedzsmenthez es stakeholder riportolashoz, mig az OpenSpec a **fejlesztesi vegrehajtast** strukturalja. A ketto egyutt mukodik: a Jira ticket hivatkozik az OpenSpec proposal-ra, a tasks.md haladas visszatukrozodik a Jira status-ban.

4.17 Tovabbolvasas

Ha tobbnet szeretnel tudni az itt emlitett koncepciekrol, ime a legfontosabb forrasok:

- **Agile Manifesto** – Az agilis fejlesztes alapelvei, amelyekre az OpenSpec is epit.
- **Shape Up (Basecamp)** – A Basecamp termekfejlesztesi modszertana, az OpenSpec szellemi elode. Ingyenesen olvashato online.
- **Atlassian Agile Guides** – Gyakorlati utmutatok specifikaciok es kovetelmenykezeles temakon.
- **Martin Fowler blogja** – A szoftverfejlesztes egyik legelismertebb gondolkodoja, akinek irásai a specifikaciok es tervezes temaiban kulonosen relevansak.
- **GitLab Engineering Handbook** – Peldaerteku nyilt fejlesztesi folyamat, architekturalis dontesek dokumentalasaval.
- **Claude Code dokumentacio** – Az Anthropic hivatalos dokumentacioja az AI-vezérelt fejleszteshez.
- **ThoughtWorks Tech Radar** – Technologiai trendek es bevalt gyakorlatok, beleertve az ADR formátumot.

Kulcs üzenet PM-nek

Az OpenSpec munkafolyamatban a PM vegre nem a fejlesztot kerdezgeti, hanem olvashato dokumentumokat review-ol. Az AI a tervbol dolgozik, nem a levegobe beszeltek. A haladas valos ideju, a minoseg merheto, es ha valami nem stimmel, visszamehetsz a specifikaciohoz es ramutathatsz ra: ”ezt kertem, ezt kaptam – mi a kulonbseg?” A review gate-ek biztositjak, hogy a PM megmarad donthozonak – nem az AI donti el, mit epit, hanem te. Az AI vegrehajtja, amit a specifikaciod eloir.

5. fejezet

Párhuzamos AI ágensek – Orchestráció

5.1 Az ötlet

Az előző fejezetben láttuk, hogyan dolgozik egyetlen AI ágens egy feladaton: specifikáció –> design –> tasks –> implementáció. De mi van, ha nem egy feladatod van, hanem tíz? Vagy húsz?

Gondolj egy valós projektre: egy webshop modernizálása. Kell bele:

- Felhasználói regisztráció átdolgozás
- Fizetési rendszer frissítés
- Email értesítések
- Admin felület
- API átírás
- Keresés javítás

Hagyományos módszerrel ez 6 fejlesztő, 6 hétnyi munka. **De mi lenne, ha 6 AI ágens dolgozna rajta egyszerre?**

Ez az orchestráció lényege.

A gondolat nem új: a szoftverfejlesztés történetében mindig keresték a módszert, hogyan lehet több ember munkáját párhuzamosítani. Fred Brooks 1975-ös klasszikusa, a *The Mythical Man-Month*, már rámutatott, hogy több ember nem jelent lineáris gyorsulást – mert nő a koordinációs költség. Az AI ágensekkel ez megváltozik: az ágenseknek nem kell meetingelniük, nem kell email-ezniük, és nem sértődnek meg. A koordinációt egy szoftver végzi, automatikusan.

5.2 Git worktree-k – a párhuzamos munkaterületek

Mielőtt az orchestrációba merülnénk, meg kell érteni egy egyszerű fogalmat: **hogyan dolgozik több ágens egyszerre ugyanazon a projekten anélkül, hogy egymást zavarnák?**

A válasz: **git worktree-k** (munkafák).

5.2.1 Az iroda hasonlat

Képzeld el, hogy a projekted egy nagy iroda. A fő iroda az, ahol az éles, működő szoftver van – ez a "master" ág. Amikor új munkára van szükség, nem a fő irodában dolgozol (mert az zárolhat lenne), hanem nyitsz egy **különálló szobát**. Ebben a szobában a projekt egy teljes másolata van – az összes fájl, az egész történelem – de amit itt változtatsz, az nem befolyásolja a fő irodát, amíg te úgy nem döntesz.

```
Képzeld el, hogy van egy irodád (a projekt).
De az irodának 5 különálló szobája van,
mindben egy-egy fejlesztő dolgozik.

+-------------------------------------------+
|                 Projekt                   |
|                                           |
|  +----------+  "szoba" = worktree         |
|  | Master   |  A fő verzió, ahova az      |
|  | (előtér) |  elkészült munka kerül      |
|  +----------+                             |
|                                           |
|  +----------+  +----------+  +----------+ |
|  | Szoba 1  |  | Szoba 2  |  | Szoba 3  | |
|  | Ágens A  |  | Ágens B  |  | Ágens C  | |
|  | regisztr.|  | fizetés  |  | email    | |
|  +----------+  +----------+  +----------+ |
|                                           |
|  +----------+  +----------+               |
|  | Szoba 4  |  | Szoba 5  |               |
|  | Ágens D  |  | Ágens E  |               |
|  | admin    |  | keresés  |               |
|  +----------+  +----------+               |
|                                           |
|  Minden szobában a projekt egy másolata,  |
|  de mindenki a saját változtatásain dolgozik.|
|  Amikor kész, a változtatások beolvadnak  |
|  a fő verzióba.                           |
+-------------------------------------------+
```

Ez nem egyszerűen metafora – pontosan így működik technikai szinten is. A git worktree funkció (bővebben lásd: Git Worktree dokumentáció) lehetővé teszi, hogy egyetlen repository-ból több munkakönyvtárat hozz létre. Minden munkakönyvtár egy különálló "branch"-en (ágon) dolgozik, de mindegyik ugyanahhoz a központi tárhelyhez tartozik.

5.2.2 Miért jobb ez, mint a "színpadi" párhuzamosság?

Ha valaha dolgoztál úgy, hogy 5 fejlesztő ugyanazon a Git repo-n dolgozott, tudod: állandóan ütköznek. Az egyik megváltoztat egy fájlt, a másik is, és jön a "merge conflict" – az összeolvasztási ütközés. A worktree-kkel ez ritkább, mert:

1. **Teljes izoláció**: Minden ágens a saját könyvtárában dolgozik, nem látja a másik munkáját
2. **Nem kell várni**: Az ágens nem várja meg, amíg a másik "push-ol" (felküldi a munkáját)
3. **Atomi beolvasztás**: Amikor kész, az egész munka egyben olvad be – nem darabonként

PM szemmel: Mintha 5 fejlesztő 5 külön gépen dolgozna, és a munkájukat a végén összefésülik. Annyi a különbség, hogy itt AI ágensek dolgoznak, automatikusan, és az "összefésülés" is automatikus. A worktree-k a háttértechnológia – neked PM-ként nem kell ezzel foglalkoznod, de jó tudni, hogy ez garantálja a biztonságos párhuzamosságot. Többet a branching stratégiákról az Atlassian Git Workflows oldalán olvashatsz.

5.3 A Ralph Loop – az autonóm munkaciklus

Minden worktree-ben egy **Ralph Loop** nevű ciklus fut. Ez adja az ágens önállóságát: nem kell egy emberi felügyelőnek minden lépésnél bólintania.

5.3.1 Miért "loop"?

Mert az ágens nem egyszer próbálja meg a feladatot, hanem ismételten: kiválasztja a következő feladatot, megpróbálja megoldani, futtatja a teszteket, és ha nem sikerült, újra próbálja. Ez az iteratív megközelítés sokkal robusztusabb, mint egy egyszeri próbálkozás – ahogy egy emberi fejlesztő is "debugol" (hibát keres), az AI ágens is iterálhat.

```
+---------------- Ralph Loop ----------------+
|                                            |
|   +-------------+                          |
|   | Feladat     |  Olvas: tasks.md         |
|   | kiválasztása |  Találja: első [ ] elem |
|   +------+------+                          |
|          |                                 |
|          v                                 |
|   +-------------+                          |
|   | Claude Code  |  Implementálja a feladat|
|   | fut          |  (fájlok, tesztek, comm.)|
|   +------+------+                          |
|          |                                 |
|          v                                 |
|   +-------------+                          |
|   | Ellenőrzés   |  Minden teszt átment?   |
|   |              |  Van még feladat?        |
|   +------+------+                          |
|          |                                 |
|          +-- Van még feladat -->  Következő |
|          |                                 |
|          +-- Minden kész --> Beolvasztás    |
|          |                                 |
|          +-- Elakadt --> Leáll, jelent      |
|                                            |
|   Biztonsági korlátok:                     |
|   - Max. iteráció szám (alapért. 10)       |
|   - Max. futási idő (alapért. 45 perc/iter.)|
|   - Elakadás detekció (2 iteráció haladás  |
|     nélkül --> leáll)                      |
+--------------------------------------------+
```

PM szemmel: A Ralph Loop az, ami garantálja, hogy az ágens nem áll le az első hiba után, de nem is fut a végtelenségig. Ha elakad, jelzi és vár. Ha kész, jelzi és megáll.

5.3.2 Ralph Loop részletesen – iterációról iterációra

Lássunk egy konkrét példát: az ágens a "felhasználói regisztráció" change-en dolgozik. A tasks.md-ben 6 feladat van.

1. iteráció: - Az ágens megnyitja a tasks.md-t, látja az első bepipálatlan feladatot: "1.1 Regisztrációs form komponens létrehozása" - Elindítja a Claude Code-ot a feladattal - Claude Code létrehozza a fájlt, megírja a kódot, futtatja a teszteket - A tesztek átmennek – bejelöli: [x] 1.1 Regisztrációs form - Commitol (menti a munkáját) - Ellenőrzi: van még feladat? Igen –> következő iteráció

2. iteráció: - Következő feladat: "1.2 Email validáció hozzáadása" - Claude Code implementálja - Egy teszt eltörik ("invalid email" esetet nem kezeli) - Claude Code automatikusan javítja, újrafuttatja a tesztet - Most átment – bejelöli, commitol - Van még feladat? Igen –> következő iteráció

3. iteráció: - Következő feladat: "2.1 Backend endpoint a regisztrációhoz" - Claude Code megírja az endpoint-ot, adatbázis migrációs fájlt - Tesztek átmennek - Bejelölés, commit

4. iteráció: - Következő feladat: "2.2 Jelszó hash-elés és biztonság" - Ez bonyolultabb – Claude Code első nekifutásra nem találja el a tesztelvényt - Második próbálkozás: javítja a salt generation logikáját - Harmadik próbálkozás: most átmennek a tesztek - Bejelölés, commit - (Megjegyzés: ez az iteráción belüli újrapróbálkozás – a Ralph Loop iterációja "felülről" nézi a feladatokat, de egy iteráción belül Claude Code többször is próbálkozhat)

5. iteráció: - Hátramaradt 2 feladat: frontend összekötés és teszt kiírás - Mindkettőt megcsinálja (ha elég egyszerű, egy iterációban több feladat is lehet)

6. iteráció: - Az utolsó feladat: integrációs tesztek - Minden átment - Nincs több [ ] elem a tasks.md-ben - **A Ralph Loop leáll és jelenti: "change kész, kész a beolvasztásra"**

Az egész folyamat – 6 iteráció – jellemzően 20-40 perc alatt zajlik le. Emberi beavatkozás nulla.

5.3.3 Biztonsági mechanizmusok – mi van, ha valami rosszul sül?

Az AI ágensek nem tökéletesek. Néha elakadhatnak, végtelen ciklusba kerülhetnek, vagy rossz irányba indulhatnak. A rendszer többrétegű védelmet épít be:

1. Iterációs limit: Alapértelmezetten egy Ralph Loop maximum 10 iterációt futhat. Ha 10 iteráció után sincs kész, leáll és jelzi: "nem sikerült befejezni". Ez megakadályozza, hogy egy elakadt ágens órákig fusson feleslegesen (és fogyassza az API kreditet).

2. Időkorlát: Minden egyes iteráció maximum 45 percig futhat. Ha ennyi idő alatt sem fejezi be az aktuális feladatot, az iteráció leáll.

3. Elakadás-detektálás (stall detection): Ha az ágens 2 egymás utáni iterációban nem jelöl be új feladatot – azaz nem halad – a rendszer leállítja. Ez a leggyakoribb biztonsági esemény: az ágens próbálkozik, de nem képes megoldani a feladatot.

4. Azonnali leállítás: PM-ként vagy fejlesztőként bármikor leállíthatod az ágenst. A `wt-ralph stop <change-id>` parancs azonnal megállít egy adott ágenst. Nincs várni – a következő lehetőségnél leáll.

5. Visszagörgethetőség (rollback): Mivel minden ágens git-ben dolgozik, és minden lépés commit, bármikor visszamehetsz egy korábbi állapotra. Ha az ágens hibás kódot írt, a git történelem megmondja, melyik commit volt az utolsó jó, és egyszerűen visszaállítható. Ez nem tudományos-fantasztikus: ez a git alapfunkciója, amit évtizedek óta használnak fejlesztők.

6. Izoláció: Az ágens worktree-je teljesen különáll a fő ágtól. Ha egy ágens "megőrül" és rommá ír tele mindent, az a fő ágat nem érinti – egyszerűen töröljük a worktree-t és kész. Mintha egy részeg dolgozót kikísérsz az irodából: a fő iroda érintetlen marad.

PM szemmel: Nem kell félned, hogy az AI "eltöri" a projektet. A legrosszabb ami történhet: egy ágens nem végzi el a feladatát, és újra kell indítani. A fő projekt mindig védett.

5.4 Az orchestrátor – a karmester

Az orchestrátor a legmagasabb szintű automatizálás. Input: egy specifikáció dokumentum. Output: kész, beolvasztott, tesztelt kód.

Az orchestrátor szerepe hasonlít egy karmesterhez: nem ő játszik az egyes hangszereken, hanem koordinálja, hogy ki mikor játszik, és az egész együtt szépen szóljon. Ha tetszik, gondolhatsz rá úgy is, mint egy "szuper PM-re" – aki elolvassa a specifikációt, felbontja feladatokra, kiosztja őket a "fejlesztőknek" (AI ágenseknek), figyeli a haladást, és az eredményt összefésüli.

A Netflix mutatott rá először, hogy az ilyen fajta automatizált orchestráció – ahol sok kis szolgáltatás párhuzamosan dolgozik – nagyszabású rendszereknél is működik (lásd: Netflix Tech Blog). Az AI ágensek orchestrációja ennek a gondolatnak a közvetlen leszármazottja.

5.4.1 Hogyan működik?

```
+-------------------------------------------------------+
|                   ORCHESTRÁTOR                        |
|                                                       |
|  1. INPUT: Spec dokumentum (bármilyen markdown)       |
|     "Webshop modernizálás: regisztráció, fizetés,     |
|      email, admin, API, keresés"                      |
|            |                                          |
|            v                                          |
|  2. AI ELEMZÉS: Claude felbontja change-ekre          |
|     +---------------------------------+              |
|     |  Change 1: user-registration  [S] |             |
|     |  Change 2: payment-update     [M] |             |
|     |  Change 3: email-service      [S] |             |
|     |  Change 4: admin-panel        [L] |             |
|     |  Change 5: api-rewrite        [M] |             |
|     |  Change 6: search-upgrade     [S] |             |
|     +---------------------------------+              |
|            |                                          |
|            v                                          |
|  3. FÜGGŐSÉGI GRÁF: Mi függ mitől?                    |
|                                                       |
|     user-registration --+                             |
|                         +-->  api-rewrite             |
|     payment-update -----+        |                    |
|                                  v                    |
|     email-service -------->  admin-panel              |
|                                                       |
|     search-upgrade (független)                        |
|            |                                          |
|            v                                          |
|  4. PÁRHUZAMOS VÉGREHAJTÁS:                           |
|                                                       |
|     Kör 1:  [user-reg] [payment] [email] [search]     |
|              ---------- párhuzamosan ----------       |
```

```
|                   |         |                        |
|                   v         v                        |
|     Kör 2:     [api-rewrite]                         |
|                   |                                  |
|                   v                                  |
|     Kör 3:     [admin-panel]                         |
|           |                                          |
|           v                                          |
|  5. EREDMÉNY: Összefoglaló riport                    |
|     6/6 change kész, 0 hiba, 47 teszt átment         |
|                                                      |
+------------------------------------------------------+
```

5.4.2 A folyamat lépésről lépésre

1. lépés: Specifikáció beolvasása Adsz egy markdown dokumentumot (lehet akár egy product brief), és az orchestrátor Claude-al elemezteti. A Claude értelmezi, mit kell csinálni, és felbontja kisebb, kezelhető darabokra (change-ekre). Ez a lépés jellemzően 2-3 percet vesz igénybe – az AI elolvassa a specifikációt, azonosítja a független munkaegységeket, és javaslatot tesz a felosztásra.

2. lépés: Méretezés Minden change kap egy méretet: - **S (kicsi)**: 1-3 fájl módosítás, egyszerű feladat. Példa: "Kereső mező placeholder szövegének átírása" - **M (közepes)**: 5-10 fájl, komplex logika. Példa: "Fizetési rendszer új szolgáltatóval bővítése" - **L (nagy)**: Sok fájl, architektúrális hatás. Példa: "Admin felület teljes újratervezése"

A méret fontos a PM-nek, mert befolyásolja az idő- és költségbecslést. Egy S change jellemzően 10-20 perc, egy M 30-60 perc, egy L 1-2 óra.

3. lépés: Függőségi gráf (DAG) A Claude meghatározza, mely change-ek függenek egymástól. Például: az API átírás nem kezdődhet el, amíg a felhasználói regisztráció nincs kész (mert az API-nak ismernie kell az új felhasználói modellt).

Ezt a függőségi gráfot technikai nyelven **DAG**-nak hívják (Directed Acyclic Graph – irányított körmentes gráf). De ne ijedj meg a névtől. A következő hasonlat megmagyarázza.

4. lépés: Párhuzamos végrehajtás Ami független, az egyszerre indul. Minden change kap egy worktree-t, egy Ralph Loop-ot, és a teljes OpenSpec pipeline-t (proposal –> specs –> design –> tasks –> implementáció). Az orchestrátor figyeli mindegyiket.

5. lépés: Beolvasztás és tesztelés Amikor egy change kész, az orchestrátor beolvasztja a fő ágba, és lefuttatja a teszteket. Ha a teszt sikeres, feloldja a függő change-eket – azok elindulhatnak.

5.4.3 A függőségi gráf (DAG) – a főzési hasonlat

A DAG bonyolultan hangzik, de a mindennapi életben is használod. Képzeld el, hogy vacsorát főzöl: az étel hús, saláta, és desszert.

```
A vacsora "függőségi gráfja":


+------------+    +----------+    +----------+
| Hús sütése |    | Saláta   |    | Desszert |
| (30 perc)  |    | (10 perc)|    | (20 perc)|
+-----+------+    +-----+----+    +----+-----+
      |                 |              |
```

```
      |                 |             |
      v                 v             v
+------------+    +----------+    +----------+
| Szósz      |    | (kész)   |    | (kész)   |
| készítés   |    |          |    |          |
| (5 perc)   |    +----------+    +----------+
+-----+------+
      |
      v
+------------+
| Tálalás    |  <-- erre vár mindhárom
| (1 perc)   |
+------------+
```

- A **saláta** és a **desszert** függetlenek – ezeket párhuzamosan készítheted (vagy két különböző ember készítheti egyszerre)
- A **szósz** függ a **hústól** – nem tudod elkészíteni a szószt, amíg a hús nem sül meg (kell a szaftja!), tehát a szósz várakozik
- A **tálalás** minden komponenstől függ – amíg mindhárom nem kész, nem ehetsz

Pontosan így működik az orchestrátor is:

```
Szoftver projekt "DAG":


+------------------+   +-----------------+   +----------------+
| user-registration|   | email-service   |   | search-upgrade |
| (független)      |   | (független)     |   | (független)    |
+--------+---------+   +--------+--------+   +-------+--------+
         |                      |                    |
         v                      v                    v
+------------------+   +--------+--------+   +--------+--------+
| api-rewrite      |   | admin-panel     |   | (kész)          |
| (várja: user-reg)|   | (várja: email)  |   |                 |
+--------+---------+   +--------+--------+   +-----------------+
         |                      |
         v                      v
      [KÉSZ]                 [KÉSZ]
```

A függetleneket párhuzamosan indítja – ez a sebesség titka. A függőeket sorban, a helyes sorrendben. Az orchestrátor automatikusan kiszámítja az optimális sorrendet és ütemezést, ahogy egy jó séf is tudja, melyik fogást kell előbb elkezdeni, hogy minden egyszerre legyen kész.

Martin Fowler írásai a Continuous Integration-ról részletesen tárgyalják, miért fontos, hogy a párhuzamos munkaágak rendszeresen visszaolvadjanak a fő ágba – az orchestrátor pontosan ezt valósítja meg, automatikusan.

5.4.4 Valós példa: egy projekt az orchestrátorral, elejétől a végéig

Lássunk egy részletes példát. Tegyük fel, hogy egy e-learning platformot modernizálsz. A specifikációd így néz ki:

> *"Modernizáljuk az e-learning platformot: új kurzus-kereső, videó lejátszó upgrade, certifikát-generálás, és admin statisztikák."*

09:00 – Spec betöltése és elemzés Az orchestrátor elolvassa a specifikációt. Az AI 4 change-et azonosít: - `course-search` [S] – új kereső funkció - `video-player` [M] – videó lejátszó frissítés - `certificate-gen` [S] – certifikát generálás - `admin-stats` [M] – admin statisztikás dashboard

Függőség: az `admin-stats` függ a `course-search`-től (mert a keresési statisztikákat mutatja). A többi független.

09:03 – Párhuzamos indítás Az orchestrátor létrehoz 3 worktree-t és elindítja a Ralph Loop-ot a 3 független change-re: - Worktree 1: `course-search` – Ralph Loop indul - Worktree 2: `video-player` – Ralph Loop indul - Worktree 3: `certificate-gen` – Ralph Loop indul

Az `admin-stats` várakozik.

09:15 – Első eredmény A `course-search` (kis méretű) már kész! 3 iterációt használt, 12 percbe telt. Az orchestrátor beolvasztja a fő ágba, futtatja a teszteket – minden átment.

Mivel a `course-search` kész, az `admin-stats` függősége feloldódik. Az orchestrátor azonnal elindítja a 4. worktree-t: `admin-stats` Ralph Loop indul.

09:30 – Második eredmény A `certificate-gen` is kész: 2 iteráció, 14 perc. Beolvasztás, teszt, minden jó.

09:45 – Dashboard közbeni állapot A dashboard ilyesmit mutat:

```
Change           Státusz    Iteráció    Token
---------------  -------    --------    -----
course-search    [KÉSZ]     3/10        18k
certificate-gen  [KÉSZ]     2/10        12k
video-player     [FUT]      5/10        64k
admin-stats      [FUT]      2/10        22k
```

10:10 – video-player kész A közepes méretű change 7 iterációt használt – volt egy bonyolultabb rész, ahol a codec-támogatás tesztjeivel szenvedett az ágens. De végül minden átment.

10:35 – admin-stats kész Az utolsó change is befejeződött. Összesítés:

```
ORCHESTRÁTOR RIPORT
-------------------
Összesen:  4/4 change kész
Idő:       1 óra 35 perc
Token:     ~142k token
Költség:   ~\$8.50
Tesztek:   63 teszt, mind átment
Merge:     4/4 sikeresen beolvasztva
```

Egyetlen ember érintette a folyamatot: te, a PM, aki megírta a 10 soros specifikációt. A fejlesztő majd a code review-nál jön képbe.

5.5 Merge policy-k – mennyire legyen automatikus?

Az orchestrátor három szintű automatizálást kínál:

```
+-----------------------------------------------+
|  eager     |  Azonnal beolvasztja amikor kész    |
|            |  --> Leggyorsabb, de kockázatos     |
+------------+-------------------------------------+
| checkpoint |  N darab után megáll és vár         |
|            |  --> Egyensúly sebesség és kontroll |
```

```
+------------+-----------------------------------+
|  manual    |  Sorba áll, ember hagy jóvá       |
|            |  --> Leglassabb, de legbiztonságosab|
+------------+-----------------------------------+
```

PM szemmel: A merge policy a bizalom szintje. Új projekten érdemes `checkpoint`-tal indulni – az orchestrátor minden 2-3 kész change után megáll és vár, hogy átnézd. Amikor már bízol benne, átállhatsz `eager`-re.

5.5.1 A bizalmi gradiens – manual-tól eager-ig

Az orchestrátor használata során a legtöbb csapat egy természetes fejlődési utat jár be:

1. fázis: Manual (az első hét) Minden egyes change-et kézi review után olvasztasz be. Lassú, de sokat tanulsz arról, milyen minőségű kódot ír az AI. Megtanulod, mire figyel a rendszer, és mire nem. Ez az ismerkedési fázis.

Jellemző kihívások ebben a fázisban: - Nem bízol az AI döntéseiben – minden sort átnézel - Sokat tanulsz arról, hogyan gondolkodik az AI - A review idő magas: feladatonként 15-20 perc

2. fázis: Checkpoint (2-4. hét) Elkezded látni a mintákat: a kis (S) change-ek szinte mindig rendben vannak, a közepesek (M) is nagyrészt jók. Átállsz checkpoint üzemmódra: minden 2-3 change után nézed át a batch-et. Ha találsz hibát, javítod és tovább megy.

Jellemző tapasztalatok: - Az S change-ek 90%+ arányban review nélkül is jók - Az M change-eknél alkalmanként kell kézi javítás - A review idő csökkent: batch-enként 10-15 perc - A sebesség 2-3x gyorsabb, mint manual módban

3. fázis: Eager (5. hét~) A nagy change-eket még mindig kézi review-olod, de a kis és közepes méretű change-eket automatikusan engeded beolvadni. Ez már szinte teljes automatizálás – az orchestrátor ontja a kész kódot, te csak a napi összefoglalót nézed át, és a nagy, architektúrális change-eket review-olod.

Jellemző jellemzők: - A napi output megtöbbszöröződik - A PM szerepe inkább "minőségbiztosítás" és "irány-mutatás" - A fejlesztő review változatlanul fontos – ez nem marad el!

Fontos: A bizalmi gradiens nem a code review kihagyásáról szól. A fejlesztő review mindig megmarad. A kérdés az, hogy a **beolvasztás** automatikus-e, vagy várakozik-e emberi jóváhagyásra. Az eager módban a beolvasztás automatikus, de a fejlesztő utána nézi át a kódot – és ha valami nem jó, javítja. Ez hasonló a Continuous Integration elvéhez: először olvasszd be, aztán javítsd ha kell – a lényeg, hogy a kódágak ne távolodjanak el egymástól.

5.6 A GUI – valós idejű áttekintés

Az orchestrátor egy vizuális dashboard-ot biztosít, ahol minden ágens állapota valós időben látszik:

```
+-----------------------------------------------+
|  wt-tools Control Center                       |
|                                                |
|  Change               Státusz   Iteráció  Token  |
|  -----------------    -------   --------  -----  |
|  user-registration    * kész    3/10      45k    |
|  payment-update       > fut     5/10      82k    |
|  email-service        * kész    2/10      28k    |
|  search-upgrade       * kész    1/10      15k    |
|  api-rewrite          ~ vár    -         -      |
|  admin-panel          ~ vár    -         -      |
```

```
|                                                |
|  Összesen: 3/6 kész | 1 aktív | 2 várakozik     |
|  Eltelt idő: 2h 34m | API költség: \$12.50        |
|                                                |
+------------------------------------------------+
```

PM szemmel: Mint egy Jira board, de valós időben frissül, és nem emberektől függ hogy frissüljön. Nem kell standup meetinget tartanod – megnyitod a dashboardot és látod.

5.6.1 Milyen metrikákat figyel egy PM a dashboardon?

Nem minden szám egyformán fontos. Íme, amire érdemes figyelni:

1. Haladási arány (X/Y kész) A legfontosabb szám. Ha 6 change-ből 4 már kész, a projekt 67%-ánál jársz. Ez a "nagy kép" – ezt mondod a stakeholdernek.

2. Aktív ágensek száma Ha 3 ágens fut párhuzamosan, az a maximum kihasználtság. Ha 1 fut és 4 várakozik, az függőségi szűk keresztmetszet – érdemes megvizsgálni, miért.

3. Iterációszám / change Ha egy change már 8/10 iterációnál tart, az figyelmeztető jel: talán túl bonyolult, vagy a specifikáció nem elég pontos. Ilyenkor érdemes megnézni az ágens logját, és ha kell, segíteni (pontosítani a specifikációt, vagy a feladatot kisebbekre bontani).

4. Token-felhasználás Közvetlenül arányos a költséggel. Ha egy change szokatlanul sok tokent használ (például 200k egy S méretű change-hez), az valószínűleg azt jelenti, hogy az ágens körökben jár – elakadt, és újra meg újra próbálkozik ugyanazzal.

5. Eltelt idő A teljes projekt futási ideje. Általában 1-5 óra közepes projektre, a change-ek számától és bonyolultságától függően. Ha ez eltér a megszokottól (például 8 óra egy 4-change-es projektre), érdemes utánanézni.

A Google SRE Book részletesen tárgyalja, hogyan érdemes rendszer-metrikákat figyelni és reagálni anomáliákra – az elvek itt is alkalmazhatók.

5.7 Hibakezelés: mi történik, ha valami rosszul megy?

Bármilyen jó is az automatizálás, hibák előfordulnak. PM-ként fontos tudni, milyen típusú hibák vannak, és mi a teendő.

5.7.1 1. Merge conflict – összeolvasztási ütközés

Ez a leggyakoribb probléma. Kétfelé ágens dolgozik két különböző change-en, de történetesen mindkettő módosítja ugyanazt a fájlt. Amikor az orchestrátor megpróbálja beolvasztani a másodikat, **ütközés** keletkezik.

Mi történik automatikusan? - Az orchestrátor észleli az ütközést - Újraindítja az érintett ágenst a frissített fő ág alapján - Az ágens újra megcsinálja a munkáját, most már látva a másik ágens változásait - Ha ez sikerült, beolvasztja

Mi történik, ha az automatikus megoldás nem sikerül? - Az orchestrátor megjelöli a change-et "conflict" státusszal - Várakozik emberi beavatkozásra - A PM/fejlesztő kézi review-val oldja meg az ütközést

PM teendő: Ha a dashboardon "conflict" státuszt látsz, szólj a fejlesztőnek. Ez nem katasztrófa – a hagyományos fejlesztésben is naponta előfordul. Az orchestrátor az esetek 80-90%-ában maga megoldja; csak a maradék 10-20%-hoz kell ember.

5.7.2 2. Ágens elakadás (stall)

Az ágens nem képes megoldani egy feladatot. Például: a spec nem elég részletes, az ágens nem érti, mit kell csinálni, és körökben jár.

Jelei a dashboardon: - Magas iterációszám, alacsony bejelölési arány - Sok token-felhasználás, kevés commit

PM teendő: 1. Nézd meg az ágens logját – mit próbál csinálni? 2. Ha a spec nem elég pontos, pontosítsd és indítsd újra 3. Ha a feladat túl nagy, bontsd kisebbre (oszd az L-es change-et két M-esre)

5.7.3 3. Teszthiba a beolvasztás után

Néha egy change egyedül működik, de beolvasztás után eltöri a másik change valamelyik tesztjét. Ez az "integrációs hiba".

Mi történik automatikusan? - Az orchestrátor futtatja a teljes teszt-készletet beolvasztás után - Ha hiba van, visszagörgeti a beolvasztást (az előző jó állapotba) - Újraindítja az érintett ágenst a javításra

PM teendő: Általában semmi – az orchestrátor kezeli. Ha többször is előfordul, az a függőségi gráf hiányosságára utalhat: talán két change függne egymástól, de az AI nem ismerte fel a függőséget. Ilyenkor érdemes kézi függőséget beállítani.

5.7.4 4. API rate limit vagy költség-túllépés

Ha az AI API szolgáltató (például Anthropic) rate limitet alkalmaz, az ágensek lelassulnak vagy ideiglenesen leállnak.

PM teendő: A dashboardon látod, ha egy ágens "vár API válaszra". Érdemes a párhuzamos ágensek számát csökkenteni (például 5 helyett 3-ra), vagy az API terv szintjét növelni.

5.8 Valós számok

Egy közepméretű projekt modernizálása az orchestrátorral:

Mutató	Érték
Input	1 specifikáció dokumentum (2 oldal)
Change-ek száma	8
Párhuzamos ágensek	3 (egyszerre max.)
Teljes futási idő	~4-5 óra
Emberi beavatkozás	2x review (merge checkpoint)
API költség	~$15-25
Hagyományos becslés	3-5 fejlesztőnap

Bővebb kontextus a számokhoz:

- A **$15-25** API költség a Claude API használatra vonatkozik (tokenenként számolva). Egy S méretű change általában $1-3, egy M méretű $3-8, egy L méretű $8-15. Ezek 2026 eleji árak – az AI szolgáltatás árak évről évre csökkennek.
- A **4-5 óra futási idő** nem jelent 4-5 óra emberi munkát. A PM 15-30 percet tölt a specifikációval, aztán 2x10 percet a checkpoint review-val. A többi automatikus.
- A **3 párhuzamos ágens** az általános beállítás. Több ágens gyorsabb, de drágább és több merge conflictet okoz. Kevesebb ágens lassabb, de olcsóbb és biztonságosabb. A helyes szám projektről projektre változik.

5.8.1 Költségbecslés: hogyan tervezz büdzsét AI ágens használatra?

A leggyakoribb PM kérdés: "Mennyibe kerül ez?" Az AI ágensi munka költsége három fő tételből áll:

1. API tokenek (a fő költség) Az AI minden kérdésre és válaszra tokent használ. Egy token kb. egy angol szó-rész (magyar szöveggel kicsit több, mert a magyar szavak hosszabbak). Az Anthropic Claude árazása (2026 eleje): - Input: ~$3 / millió token - Output: ~$15 / millió token

Egy átlagos change 30-100k tokent használ. Egy 8-change-es projekt tehát 240-800k token, azaz **$5-25**.

2. Számítási erőforrások (elhanyagolható) A worktree-k és Ralph Loop-ok helyi gépen futnak. A számítási költség minimális – egy átlagos fejlesztői laptop elbír 3-5 párhuzamos ágenst.

3. Fejlesztői idő (a review) A fejlesztő review-ra fordított idő nem változik sokat – az AI által írt kódot ugyanúgy át kell nézni, mint az emberi kódot. De a **fejlesztési idő** jelentősen csökken.

Összevetés hagyományos költségekkel:

```
Hagyományos:
  5 fejlesztőnap x 8 óra x \$80/óra = \$3,200
  + PM idő: 2 nap x 8 óra x \$60/óra = \$960
  Összesen: ~\$4,160

Orchestrátorral:
  API költség: \$20
  PM idő: 1 óra x \$60/óra = \$60
  Fejlesztő review: 3 óra x \$80/óra = \$240
  Összesen: ~\$320

Megtakarítás: ~92%
```

Természetesen ez egy idealizált példa – a valós megtakarítás a feladat bonyolultságától és a szükséges emberi beavatkozás mértékétől függ. De a nagyságrend jellemzően 5-10x megtakarítás a hagyományos módszerhez képest.

Fontos megjegyzés: Az orchestrátor nem teszi feleslegessé az emberi review-t. Az AI által írt kódot egy senior fejlesztőnek végig kell néznie. Az orchestrátor az elkészülést gyorsítja, nem a review-t eliminálja.

5.9 Mikor érdemes orchestrálni?

Helyzet	Orchestrátor?
1 kis feature	Nem – egyetlen Claude Code session elég
2-3 összefüggő feature	Talán – ha vannak függetlenek
5+ feature, projekt szintű	Igen – ez az orchestrátor erőssége
Kritikus, biztonsági kód	Nem – emberi kézi munka kell
Gyors prototípus	Nem – vibe coding gyorsabb
Terv alapú modernizáció	Igen – pont erre készült
Legacy kód átírás	Igen – sok független modult lehet párhuzamosítani
Egyszerű szövegcsere 50 fájlban	Nem – egy script gyorsabb

5.9.1 Döntési fa PM-eknek

Ha bizonytalan vagy, használd ezt az egyszerű döntési fát:

```
Van legalább 3 független feladat?
```

```
|
+-- Nem --> Használj egyetlen Claude Code session-t
|
+-- Igen
      |
      Van specifikáció vagy product brief?
        |
        +-- Nem --> Először írd meg a specifikációt
        |
        +-- Igen
              |
              A feladatok biztonságkritikusak?
                |
                +-- Igen --> Emberi fejlesztés + AI asszisztens
                |
                +-- Nem --> ORCHESTRÁTOR (manual/checkpoint módban)
```

5.10 Összefoglalás: az orchestráció mint PM szupererő

Az orchestráció a PM számára azt jelenti, hogy a feladat-lebontás, kiosztás, nyomonkövetés és beolvasztás – amit eddig napokig tartott koordinálni – **automatikusan történik**. A PM szerepe megváltozik:

- **Régi modell**: PM lebontja a feladatokat –> kiosztja a fejlesztőknek –> naponta standup –> kérdezgeti "hogy állsz?" –> összefésüli az eredményt
- **Új modell**: PM megírja a specifikációt –> az orchestrátor mindent kioszt, futtat, és beolvaszt –> PM a dashboardot nézi és review-ol

Ez nem a PM feleslegessé tételét jelenti – éppen ellenkezőleg. A PM szerepe felértékelődik, mert a **specifikáció minősége** soha nem volt ennyire fontos. Ha a specifikáció pontos, az orchestrátor remek munkát végez. Ha a specifikáció homályos, az ágensek is homályos kódot írnak. A PM feladata tehát: pontos, mérhető specifikációkat írni, és a végeredményt ellenőrizni.

További olvasnivalók az AI ágensek és multi-agent rendszerek témájából: Anthropic Research, valamint DevOps és automatizálás témában a TechWorld with Nana YouTube csatorna kiváló bevezető videókat kínál.

Kulcs üzenet PM-nek

Az orchestráció lényege: egyetlen specifikációból több AI ágens dolgozik párhuzamosan, mindegyik a saját izolált munkaterületén, automatikus koordinálással. Mint PM, nem az ágenseket irányítod – a specifikációt írod, és a végeredményt review-olod. Közben a dashboard mutatja mi történik. Indulj manual módban, építsd fel a bizalmat checkpoint-on át, és végül engedd el eager módban – a költség töredéke a hagyományos fejlesztésnek, a sebesség többszöröse.

6. fejezet

Memória és tanulás

6.1 A probléma: az amnéziás kolléga

Képzeld el, hogy van egy rendkívül tehetséges gyakornokod. Minden nap bejön, brilliánsan dolgozik — de másnap reggel **mindenre elfelejtett**. Nem emlékszik mit csinált tegnap, milyen döntéseket hozott, milyen hibákba futott, vagy milyen konvenciókat tanult meg.

Minden reggel mindent elölről kell elmagyarázni.

Gondolj bele mélyebben, milyen lenne ez a gyakorlatban. Hétfőn elmagyarázod neki, hogy a csapat agilis módszertant használ, két hetes sprintekben gondolkodtok, a code review kötelező, és a teszteléshez pytest-et használtok. Egy teljes órát tölt el az onboarding. Kedden bejön — és fogalma sincs semmiről. Ismét el kell mondanod mindent. Szerdán megint. Csütörtökre már frusztrált vagy, péntekre pedig komolyan gondolkodsz, hogy nem lenne-e egyszerűbb magadnak megcsinálni a munkát.

Most képzeld el, hogy ez a gyakornok egyébként **zseniális**. Bármit kérsz tőle, 10 perc alatt megcsinálja, hibátlanul. Csak éppen minden reggel újra kell magyaráznod a kontextust. A tehetsége elvitathatatlan — de a memóriája nulla. Ez a szituáció nemcsak frusztráló, hanem gazdaságilag is pazarló: a te időd (amit magyarázásra fordítasz) sokkal drágább, mint amennyit megspórolsz azzal, hogy ő gyorsan dolgozik.

Ez pontosan az az élmény, amit az AI ágensekkel dolgozó fejlesztők éltek át 2024-ben és 2025 elején. Minden session (munkamenet) egy üres lappal indult. A Claude Code megnyílt, és az ágens semmit sem tudott a korábbi beszélgetésekről, döntésekről, vagy a projekt történetéről. Az Anthropic saját kutatásai is kiemelik, hogy a hatékony ágenseknek szükségük van kontextusra — a memória nem luxus, hanem alapkövetelmény.

Ez az úgynevezett **"kontextus ablak" probléma**: az AI egyszerre csak egy munkamenetnyi információt lát. Ami a korábbi beszélgetésekben történt, az számára nem létezik. Ahogy a Wikipédián az organizational memory szócikkben is olvashatod: egy szervezet (vagy egy AI rendszer) csak annyira hatékony, amennyire a felhalmozott tudást képes megőrizni és újra felhasználni.

6.2 A megoldás: Developer Memory

A Developer Memory rendszer megoldja ezt a problémát. Minden fontos döntést, tanulságot, és kontextust **automatikusan ment**, és a következő session-ben **automatikusan betölti** ami releváns.

Ez nem egy futurisztikus vízió — a gyakorlatban működő rendszer, ami a Claude Code hook rendszeren keresztül csatlakozik az ágenshez. A session elején automatikusan betöltődnek a releváns emlékek, a session végén automatikusan mentődnek az újak.

6.2.1 Előtte / utána

```
┌──────────────── MEMÓRIA NÉLKÜL ────────────────┐
│                                                 │
│  Session 1:                                     │
│  Fejlesztő: "A Redis timeout legyen 30 másodperc" │
│  AI: "Rendben, beállítom 30s-re." ✓             │
│                                                 │
│  Session 2 (másnap):                            │
│  Fejlesztő: "Mi a Redis timeout?"               │
│  AI: "Az alapértelmezett 5 másodperc."          │
│  Fejlesztő: "Nem, tegnap 30-ra állítottuk!"     │
│  AI: "Sajnálom, nincs információm erről."       │
│                                                 │
└─────────────────────────────────────────────────┘

┌──────────────── MEMÓRIÁVAL ────────────────────┐
│                                                 │
│  Session 1:                                     │
│  Fejlesztő: "A Redis timeout legyen 30 másodperc" │
│  AI: "Rendben, beállítom 30s-re." ✓             │
│  [Memória ment: "Redis timeout: 30s — döntés"]  │
│                                                 │
│  Session 2 (másnap):                            │
│  Fejlesztő: "Mi a Redis timeout?"               │
│  [Memória betölt: korábbi döntés a Redisről]    │
│  AI: "Korábbi döntés alapján: 30 másodperc.     │
│       Ezt az 1-es session-ben állítottuk be."   │
│                                                 │
└─────────────────────────────────────────────────┘
```

Még egy példa, ami a mindennapi munkában gyakran előjön:

```
┌──────────── MEMÓRIA NÉLKÜL (2. példa) ──────────┐
│                                                 │
│  Session 3:                                     │
│  Fejlesztő: "Futtasd le a teszteket"            │
│  AI: "npm test" — sikertelen, 5 hiba            │
│  AI: *20 percig keresi a hibát, végül rájön,    │
│      hogy a .env.test fájl kell hozzá*          │
│                                                 │
│  Session 8:                                     │
│  Fejlesztő: "Futtasd le a teszteket"            │
│  AI: "npm test" — megint sikertelen             │
│  AI: *Megint 20 percig keres, megint rájön      │
│      a .env.test-re...*                         │
│                                                 │
└─────────────────────────────────────────────────┘

┌──────────── MEMÓRIÁVAL (2. példa) ──────────────┐
```

```
│                                                   │
│  Session 8:                                       │
│  Fejlesztő: "Futtasd le a teszteket"             │
│  [Memória betölt: "Tesztek futtatása előtt:       │
│   cp .env.test.example .env.test"]                │
│  AI: "Előbb másolom a teszt környezeti fájlt,     │
│       utána futtatom a teszteket."                │
│  AI: "npm test" — 42/42 átment                    │
│  *2 perc az egész*                                │
│                                                   │
└───────────────────────────────────────────────────┘
```

6.3 Két réteg: CLAUDE.md (statikus) és Developer Memory (dinamikus)

Mielőtt mélyebbre merülnénk a memóriában, fontos megérteni, hogy a tudás két rétegben él. Ez olyan, mint egy cég működése: vannak **írott szabályzatok** (amik ritkán változnak), és van **szóbeli tudás** (ami napról napra gyarapodik).

```
┌───────────────────────────────────────────────────┐
│                                                   │
│  1. RÉTEG: CLAUDE.md (statikus)                   │
│  ═══════════════════════════                       │
│  Kézi, fejlesztő által írt fájl a projekt gyökerében │
│                                                   │
│  Tartalom:                                        │
│  - Kód stílusra vonatkozó szabályok               │
│  - Build és teszt parancsok                       │
│  - Projekt architektúra leírása                   │
│  - "Soha ne csináld" típusú tilalmak              │
│                                                   │
│  Jellemző:                                        │
│  - Emberek írják és tartják karban                │
│  - Git-ben verziózott, mindenki látja             │
│  - Ritkán változik (hetente, havonta)             │
│  - Mint egy alkalmazotti kézikönyv                │
│                                                   │
├───────────────────────────────────────────────────┤
│                                                   │
│  2. RÉTEG: Developer Memory (dinamikus)           │
│  ═══════════════════════════════                   │
│  Automatikus, AI által írt és olvasott memória    │
│                                                   │
│  Tartalom:                                        │
│  - Döntések és indoklásaik                        │
│  - Hibákból tanult leckék                         │
│  - Projekt-specifikus kontextus                   │
│  - Bevált munkamódszerek                          │
│                                                   │
```

```
│  Jellemző:                                          │
│  - Automatikusan íródik                             │
│  - Session-ről session-re gyarapodik                │
│  - Naponta változhat                                │
│  - Mint a tapasztalt kollégák szóbeli tudása        │
│                                                     │
└─────────────────────────────────────────────────────┘
```

A CLAUDE.md dokumentáció részletesen leírja, hogyan építsük fel ezt a statikus réteget. A Developer Memory erre építkezik: amit a CLAUDE.md nem tartalmaz (mert túl specifikus, túl friss, vagy túl sok lenne), azt a memória rendszer kezeli dinamikusan.

PM szemmel: A CLAUDE.md olyan, mint a céges szabályzat — azt írja le, hogyan "kell" dolgozni. A Developer Memory pedig olyan, mint a tapasztalt kollégák tudása — azt tartalmazza, mit tanultunk a gyakorlatból. Mindkettő kell: a szabályzat adja a keretet, a tapasztalat tölti meg tartalommal.

6.4 A három memória típus

A rendszer háromféle emléket különböztet meg:

```
┌─────────────────────────────────────────────────────┐
│                                                     │
│  DÖNTÉS (Decision)                                  │
│  "A és B közül B-t választottuk, mert..."           │
│                                                     │
│  1. példa: "SSE-t használunk WebSocket helyett,     │
│     mert a Cloudflare Workers nem támogatja         │
│     a tartós kapcsolatokat."                        │
│                                                     │
│  2. példa: "A PostgreSQL-t választottuk MongoDB     │
│     helyett, mert a lekérdezési mintáink relációsak │
│     és szükségünk van tranzakciókra."               │
│                                                     │
│  3. példa: "Az API verziózást URL path-ban          │
│     csináljuk (/v1/, /v2/) és nem headerben, mert   │
│     így a dokumentáció és a debugolás egyszerűbb."  │
│                                                     │
├─────────────────────────────────────────────────────┤
│                                                     │
│  TANULSÁG (Learning)                                │
│  "Ezt tapasztaltuk és ebből tanultunk."             │
│                                                     │
│  1. példa: "A PySide6 QTimer-t csak a fő szálból    │
│     szabad hívni, különben crash."                  │
│                                                     │
│  2. példa: "Ha a CI pipeline 'out of memory'        │
│     hibát dob, a NODE_OPTIONS=--max-old-space-      │
│     size=4096 megoldja. Háromszor előfordult már."  │
│                                                     │
│  3. példa: "A deploy script a staging környezetben  │
```

```
│     mindig a 'main' ágon fut, nem a feature         │
│     branch-en. Ha másikat próbálsz, csendben        │
│     ignorálja a változásokat."                      │
│                                                     │
├─────────────────────────────────────────────────────┤
│                                                     │
│  KONTEXTUS (Context)                                │
│  "Háttér információ a projektről."                  │
│                                                     │
│  1. példa: "Ez egy monorepo pnpm workspace-szel,    │
│     az API a packages/api/ könyvtárban van."        │
│                                                     │
│  2. példa: "A projekt két külső API-tól függ:       │
│     Stripe (fizetés) és SendGrid (email). Mindkettő │
│     sandbox üzemmódban van fejlesztéskor."          │
│                                                     │
│  3. példa: "A céges VPN-en keresztül érhető el      │
│     a staging szerver. Az AI ágens nem tudja        │
│     közvetlenül elérni – a fejlesztőt kell kérni."  │
│                                                     │
└─────────────────────────────────────────────────────┘
```

Ez a háromféle típus megfelel a szervezetfejlesztés ADR (Architectural Decision Records) gyakorlatának, amit a ThoughtWorks népszerűsített. Martin Fowler is írt arról, hogy az architekturális döntések dokumentálása kritikus a hosszú távú projekt egészségéhez. A Developer Memory lényegében **automatizálja ezt a dokumentációs folyamatot** — nem kell külön ADR-eket írni, mert a rendszer magától rögzíti a döntéseket.

PM szemmel: Gondolj rá úgy, mint három fiókra az irodában: - **Döntések fiókja**: Protokollok és megállapodások, amikre hivatkozni lehet. "Miért választottuk a Stripe-ot?" — nyisd ki a fiókot. - **Tanulságok fiókja**: Lessons learned, hogy ne kövessük el újra ugyanazt. "Mit csinálunk, ha a CI elszáll?" — nyisd ki a fiókot. - **Kontextus fiókja**: Az "onboarding dokumentum", ami az új embernek kell. "Hol van az API kód?" — nyisd ki a fiókot.

6.5 Az 5 réteg — hogyan dolgozik a memória

A memóriát egy ötlépéses automatikus rendszer kezeli. Nem kell az ágensnek mondani, hogy „jegyezd meg" — a rendszer automatikusan működik:

```
Session élete:

┌──────── Session indul ────────┐
│ 1. BETÖLTÉS                   │  Betölti amit tud a projektről:
│    (Session Start)            │  korábbi döntések, konvenciók,
│                               │  ismert hibák, bevált módszerek
└───────────────────────────────┘
                │
                ▼
┌──────── Munka közben ─────────┐
│ 2. FELIDÉZÉS                  │  Minden kérdésnél megkeresi a
│    (Kérdés beérkezik)         │  releváns emlékeket
```

```
│                         │  "Volt már hasonló probléma?"
│ 3. FELISMERÉS           │  Ha parancsot futtat, megnézi
│    (Eszköz használat)   │  volt-e már hasonló helyzet
│                         │
│ 4. HIBAEMLÉK            │  Ha hiba történik, megkeresi
│    (Hiba esetén)        │  a korábbi megoldásokat
│                         │  "Ezt a hibát már láttuk,
│                         │   a megoldás: ..."
└─────────────────────────┘
            │
            ▼
┌──── Session véget ér ──┐
│ 5. MENTÉS               │  Elmenti a fontos dolgokat:
│    (Session End)        │  új döntéseket, tanulságokat,
│                         │  felfedezett mintákat
└─────────────────────────┘
```

PM szemmel: Úgy kell elképzelni, mint egy pályakezdőt akinek van egy jegyzetfüzete: - Reggel megnézi a tegnapi jegyzeteit (1. betöltés) - Munka közben ha kérdése van, visszalapoz (2-4. felidézés) - Este leírja amit tanult (5. mentés)

A különbség: az AI ágens ezt automatikusan, következetesen, minden alkalommal megteszi. Ember ritkán ilyen fegyelmezett.

6.5.1 A szemantikus keresés — hogyan talál a memória

Egy fontos részlet, amit érdemes megérteni: a memória nem úgy keres, mint egy hagyományos kereső (Google, Ctrl+F). Nem pontos szavakat keres, hanem **jelentést**.

Ez az úgynevezett **szemantikus keresés** (semantic search). Egyszerűen elmagyarázva:

```
┌──────────────────────────────────────────────────┐
│  HAGYOMÁNYOS KERESÉS (kulcsszó alapú)            │
│                                                  │
│  Keresed: "Redis timeout"                        │
│  Találat: csak ami pontosan tartalmazza          │
│           a "Redis" ÉS "timeout" szavakat        │
│  Nem találja: "cache lejárat 30 másodperc"       │
│               (pedig ugyanarról szól!)           │
└──────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────┐
│  SZEMANTIKUS KERESÉS (jelentés alapú)            │
│                                                  │
│  Keresed: "Redis timeout"                        │
│  Találat: "Redis timeout: 30s"                   │
│           "cache lejárat 30 másodperc"           │
│           "a Redis kapcsolat bontása 30s után"   │
│  Mind releváns, bár más szavakkal fogalmazták!   │
└──────────────────────────────────────────────────┘
```

Technikailag ez úgy működik, hogy a rendszer minden emléket egy matematikai “térbe” helyez, ahol a hasonló

jelentésű dolgok közel kerülnek egymáshoz. Amikor keresel, nem szavakat hasonlít össze, hanem **jelentés-vektorokat**. Ezért találja meg a "cache lejárat" emléket akkor is, amikor "Redis timeout"-ra keresel — mert a jelentésük közel áll egymáshoz.

PM szemmel: Gondolj rá úgy, mint a különbség aközött, hogy megkérdezed egy kollégát "Tudod, mi volt az a Redis dolog?", és ő érti mire gondolsz — szemben azzal, hogy Ctrl+F-fel keresed a "Redis" szót a cég 500 oldalas dokumentációjában. Az első emberi, kontextus-alapú megértés. A szemantikus keresés ezt utánozza.

6.6 Egy teljes nap — mikor aktiválódik a memória

Lássuk végig egy fejlesztő teljes napját, és hogyan segít a memória minden lépésben:

```
09:00 — Reggeli session indítás
═══════════════════════════════

Fejlesztő: claude
[Memória betölt: 14 releváns emlék a projektről]
AI: "Jó reggelt! A tegnapi session-ben a user-service
     refaktoráláson dolgoztunk. 3 feladatból 2 kész,
     a harmadik: az email validáció átírása."

09:15 — Fejlesztő kérdez
═══════════════════════

Fejlesztő: "Hogyan futtatom a teszteket erre a modulra?"
[Memória felidézés: "user-service tesztek: npm test
 -- --scope=@app/user-service"]
AI: "Korábbi tapasztalat alapján:
     npm test -- --scope=@app/user-service"

10:30 — Hiba történik
═══════════════════

AI futtat: npm test
Hiba: "ECONNREFUSED localhost:5432"
[Memória hibaemléke betölt: "Teszt adatbázishoz kell:
 docker compose up test-db"]
AI: "Ismert probléma — a teszt DB nem fut.
     Először: docker compose up -d test-db
     Várunk 5 másodpercet, utána újra futtatom."

12:00 — Architekturális kérdés
═══════════════════════════════

Fejlesztő: "Az email küldés szinkron vagy aszinkron legyen?"
[Memória betölt: Korábbi döntés: "Minden külső API hívást
 aszinkron queue-n keresztül végzünk (BullMQ), hogy a
 felhasználói válasz ne függjön a külső szolgáltatástól."]
AI: "Korábbi döntés alapján: aszinkron, BullMQ-n keresztül.
     Ezt a fizetési modulnál is így csináltuk."

14:00 — Új döntés születik
═══════════════════════

Fejlesztő: "Legyen rate limiting az email API-n: max 10/perc."
```

```
AI: "Rendben, implementálom."
[Memória ment: "Email API rate limit: 10 kérés/perc/user.
 Döntés indoklása: spam védelem."]

16:00 — Session zárul
═══════════════════
[Memória mentés: 3 új emlék mentve:
 1. Email validáció kész (kontextus)
 2. Rate limit döntés (döntés)
 3. BullMQ worker hiba megoldása (tanulság)]
AI: "Mai munka összefoglalása: email validáció kész,
     rate limit implementálva, 1 bug javítva.
     Holnap a notification service jön."
```

Figyeld meg: a fejlesztőnek egyszer sem kellett mondania, hogy "jegyezd meg" vagy "ne felejtsd el". **Minden automatikusan történt.** A Claude Code hook rendszer biztosítja, hogy a session elején és végén a memória műveletek lefussanak.

6.7 Csapat memória — gépek között

A memória nem csak egyetlen gépen létezik. Ha több fejlesztő (vagy több gépen futó ágens) dolgozik ugyanazon a projekten, a memória **git-en keresztül szinkronizálódik**:

```
┌─────────┐                    ┌─────────┐
│ Gép A   │     git sync       │ Gép B   │
│         │ ◄────────────────► │         │
│ Ágens 1 │                    │ Ágens 2 │
│ Memória │                    │ Memória │
└─────────┘                    └─────────┘
     │                              │
     └──────────── Közös ───────────┘
            memória adatbázis
```

Ha az Ágens 1 megtanulta, hogy „a deployment scriptben a `--force` flag kell", ez az információ megjelenik az Ágens 2-nél is.

6.7.1 Hogyan működik ez egy 5 fős csapatnál?

```
┌──────────────────────────────────────────────────┐
│            CSAPAT MEMÓRIA SZINKRONIZÁCIÓ          │
│                                                  │
│  Anna (backend)     ──┐                          │
│  Béla (frontend)    ──┤                          │
│  Csaba (devops)     ──┼──> Git repo  <─── AI ágens │
│  Dóra (mobile)      ──┤    (memória)       (6. tag) │
│  Erik (tesztelő)    ──┘                          │
│                                                  │
│  Minden fejlesztő gépéről automatikusan          │
│  szinkronizálódik a memória a közös repóba.      │
│  Ha Anna döntést hoz, Erik ágense is tudni fogja. │
└──────────────────────────────────────────────────┘
```

A gyakorlatban ez így néz ki:

Hétfő: Anna döntést hoz — "A REST API-ban a pagination cursor-alapú lesz, nem offset-alapú, mert a nagy adathalmazoknál hatékonyabb." Az ágense menti a döntést.

Kedd reggel: Béla elkezd dolgozni a frontend-en. Az ágense a szinkronizált memóriából tudja, hogy cursor-alapú pagination van. Nem kérdez, nem implementálja rosszul — egyből a jó megközelítést használja.

Szerda: Erik teszteket ír. Az ágense tudja Anna döntéséből, hogy cursor-alapú pagination-t kell tesztelni, és Béla tanulságából, hogy a "hasNextPage" mező boolean típusú.

Csütörtök: Csaba a CI pipeline-on dolgozik. Az ágense tudja, hogy a tesztekhez kell Docker (tanulság), és hogy a pagination cursor-alapú (döntés) — tehát a teszt fixture-öket is ennek megfelelően generálja.

Péntek: Dóra a mobilalkalmazáson dolgozik. Az ágense a hét összes döntését és tanulságát ismeri — az API-t azonnal helyesen hívja, mert a memóriából tudja a pagination formátumot, a rate limiteket, és az autentikáció módját.

Ez az Atlassian által is leírt tudásmegosztási minta: a csapat tudása nem egyetlen ember fejében él, hanem egy közös, mindenki számára elérhető rendszerben. A különbség: itt ez a rendszer **automatikusan íródik és automatikusan használódik**.

PM szemmel: Mint egy közös tudásbázis (wiki), de ami automatikusan íródik és automatikusan használódik. Nem kell senkit emlékeztetned, hogy „írd be a wiki-be" — megtörténik. És ami még fontosabb: nem kell emlékeztetned, hogy „olvasd el a wiki-t" — az AI ágens **mindig** elolvassa.

6.8 Memória kontra hagyományos tudáskezelés

A legtöbb szervezetnek van már valamilyen tudáskezelési rendszere: Confluence, Notion, SharePoint, vagy egyszerű Google Docs. A probléma nem az eszközzel van, hanem az emberi természettel. Az Atlassian saját kutatásai is igazolják: a tudáskezelési rendszerek legnagyobb kihívása nem a technológia, hanem a következetes használat.

```
┌──────────────────────────────────────────────────┐
│  HAGYOMÁNYOS TUDÁSKEZELÉS                        │
│  (Confluence / Notion / Wiki)                    │
│                                                  │
│  [+] Létezik a platform                          │
│  [-] Valakinek meg kell írnia a dokumentációt    │
│  [-] Valakinek karban kell tartania              │
│  [-] Valakinek el kell olvasnia                  │
│  [-] Az emberek elfelejtik frissíteni            │
│  [-] Az információ elavul                        │
│  [-] Senki nem keresi ott először                │
│                                                  │
│  Eredmény: "Tudom, hogy valahol le van írva,     │
│  de gyorsabb megkérdezni Pistát."                │
└──────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────┐
│  DEVELOPER MEMORY                                │
│  (automatikus)                                   │
│                                                  │
```

```
│  [+] Automatikusan íródik (nem kell emberi írás)     │
│  [+] Automatikusan frissül (minden session-ben)      │
│  [+] Automatikusan használódik (AI mindig olvassa)   │
│  [+] Relevancia-alapú keresés (nem kell tudni hol)  │
│  [+] Nem avul el (új döntés felülírja a régit)      │
│  [+] Mindenki ágense hozzáfér (csapat szinkron)     │
│                                                      │
│  Eredmény: "Az ágens már tudja, nem kell kérdezni." │
└──────────────────────────────────────────────────────┘
```

Ez nem jelenti azt, hogy a hagyományos dokumentáció felesleges! A CLAUDE.md (statikus réteg) lényegében egy strukturált, emberi kezű dokumentáció. A Developer Memory (dinamikus réteg) pedig automatikusan kiegészíti azokkal az információkkal, amiket senki sem írna le külön, de mindenki számára fontosak.

PM szemmel: A Developer Memory nem váltja ki a Confluence-t — de megoldja a Confluence legnagyobb problémáját: hogy az emberek nem írják és nem olvassák. Az AI ágensnek nem kell "motiváció" a dokumentáláshoz. Automatikusan jegyzetel, és automatikusan visszaolvassa.

6.9 Memória higiénia — karbantartás

Ahogy egy irodában is néha ki kell rendezni a fiókokat, a memóriát is karban kell tartani. Nem sok munkát igényel, de fontos.

6.9.1 Mikor kell takarítani?

- **Elavult döntések**: Ha egy korábbi döntés megváltozott (pl. "PostgreSQL-ről áttértünk MongoDB-re"), a régi memóriát törölni kell vagy frissíteni, különben az ágens a régi döntést követi.
- **Téves tanulságok**: Ha egy tanulság kiderült, hogy hibás volt (pl. "a bug nem a Redis-ben volt, hanem a hálózatban"), javítani kell.
- **Duplikációk**: Ha ugyanaz az információ többször is elmentődött más-más megfogalmazásban, érdemes összevonni.
- **Lezárt projektek**: Ha egy projekt vagy modul már nem aktív, a hozzá tartozó specifikus memóriák eltávolíthatók, hogy ne zavarják az aktuális munkát.

6.9.2 Hogyan?

```
MEMÓRIA KARBANTARTÁS LÉPÉSEI

1. Időszakos áttekintés (havonta egyszer)
   "Mutasd a legutóbbi 30 nap memóriáit"
   ──> átnézed, van-e elavult vagy hibás

2. Helytelen memória javítása
   "Töröld a XYZ memóriát, mert már nem érvényes"
   vagy: "Frissítsd: a timeout most már 60s, nem 30s"

3. Duplikáció ellenőrzés
   A rendszer képes automatikusan észlelni a
   hasonló emlékeket és jelezni a duplikációkat.

4. Tematikus takarítás
```

```
Projekt lezárásakor: "Töröld az összes memóriát
ami a v1-es API-ra vonatkozik, mert már nincs
használatban."
```

PM szemmel: Ez olyan, mint a retrospektív meeting tudás-karbantartási része. Havonta egyszer 15 perc: átnézni, hogy ami a "projekt tudásbázisban" van, az még aktuális-e. A különbség: itt 15 perc ennyit ér, mert az AI ágens **tényleg használja** a tudásbázist, szemben a Confluence-szel, amit legutóbb hat hónapja frissített valaki.

6.10 Valós példák: hogyan segít a memória a gyakorlatban

6.10.1 1. példa: Az adatbázis hiba (időmegtakarítás)

Egy projekt 3. hetében az AI ágens egy furcsa hibába futott:

```
Session 15 (memória nélkül):
AI futtatja: npm test
Hiba: "ECONNREFUSED localhost:5432"
AI: "Úgy tűnik, az adatbázis nem fut."
AI: *15 percig debugol, mire rájön, hogy a teszt
    adatbázis külön Docker containerben fut*

Session 15 (memóriával):
AI futtatja: npm test
Hiba: "ECONNREFUSED localhost:5432"
[Memória betölt: "Teszt adatbázishoz kell:
 docker compose up test-db"]
AI: "Ismert probléma — a teszt adatbázis nem fut.
     Indítom: docker compose up test-db"
*30 másodperc és kész*
```

15 perc vs. 30 másodperc. És ez **minden alkalommal** megtörténik, amikor a teszt DB nincs elindítva. 10 alkalom után már 150 percet spóroltunk.

6.10.2 2. példa: A kód stílus konzisztencia (minőség)

```
Memória nélkül:
Session 5:  AI "camelCase" változóneveket használ
Session 8:  AI "snake_case" változóneveket használ
Session 12: AI megint "camelCase"-t használ
Eredmény: inkonzisztens kód, extra review idő

Memóriával:
[Memória: "Konvenció: mindenhol camelCase,
 kivéve az adatbázis oszlopokat (snake_case)"]
Minden session: konzisztens kód, kevesebb review
```

6.10.3 3. példa: Az új csapattag onboardingja (gyorsaság)

```
Hagyományos onboarding (új AI ágens, memória nélkül):
- Nem ismeri a projekt struktúrát  --> 30 perc keresgélés
```

```
- Nem ismeri a konvenciókat        --> rossz kód stílus
- Nem ismeri a korábbi döntéseket  --> újra felteszi
  ugyanazokat a kérdéseket
- Nem ismeri a tipikus hibákat     --> belefut mindbe
Eredmény: az első 2-3 napban alig produktív

Onboarding memóriával:
- [14 memória betöltődik a projekt legfontosabb
  tényeiről]
- Ismeri a struktúrát, konvenciókat, döntéseket
- Tudja a tipikus hibákat és megoldásaikat
Eredmény: az első session-től produktív
```

6.10.4 4. példa: A sprint közbeni döntés nyomon követése (PM érték)

```
Sprint 3, hét 1:
PM: "Legyen email értesítés minden rendelés után."
[Memória ment: döntés]

Sprint 3, hét 2:
Másik fejlesztő: "Kell email értesítés a rendelésekhez?"
[Memória betölt]
AI: "Igen, ez a Sprint 3 elején már döntés volt.
     Az implementáció a notification-service-ben van."

Enélkül: a fejlesztő vagy újra megkérdezi a PM-et
(időrablás), vagy másképp implementálja (inkonzisztencia).
```

6.11 Az érték PM szemmel

Foglaljuk össze, mit jelent a Developer Memory egy projekt menedzser számára:

Terület	Memória nélkül	Memóriával
Ismételt kérdések	Ugyanaz a kérdés hetente	Egyszer megválaszolva, örökre megvan
Onboarding	2-3 nap mire produktív	Első session-től hasznos
Döntés konzisztencia	"Ezt már megbeszéltük…"	AI hivatkozik a korábbi döntésre
Hiba megoldás	Újra és újra debugol	"Ismert hiba, megoldás: …"
Kód minőség	Inkonzisztens stílus	Következetes konvenciók
Tudásmegőrzés	Egy ember távozásával elvész	Memória megmarad
Csapat szinkron	"Kérdezd meg Annát"	Mindenki ágense tudja
Sprint átláthatóság	"Miért döntöttünk így?"	Döntés + indoklás elérhető

A szervezeti memória fogalma már évtizedek óta ismert a menedzsment irodalomban. Az a felismerés, hogy egy szervezet tudása több, mint az egyes tagok tudásának összege, már a 90-es évek óta létezik. A Developer Memory ennek a koncepciónak az első igazán gyakorlati, automatizált megvalósítása a szoftverfejlesztésben.

6.12 Mi NEM memória

Fontos tisztázni: a memória nem mindenható.

Ami memória	Ami NEM memória
Projektszintű döntések	Titkos adatok (jelszavak, kulcsok)
Konvenciók és minták	A teljes kódbázis másolata
Korábbi hibák és megoldások	Minden egyes chat üzenet
Architectural Decision Records	Ideiglenes debug információ
Csapat szintű megállapodások	Személyes beállítások
Projekt architektúra leírása	Átmeneti workaround-ok

A memória szelektív — csak azt jegyzi meg, ami hosszú távon hasznos. Pont úgy, ahogy te sem jegyzed meg a mai ebédet, de emlékszel, hogy a csapatod agilis módszertant használ.

A rendszer nem rögzít titkos adatokat (API kulcsok, jelszavak), és nem próbálja lemásolni a teljes kódbázist. A cél: a **meta-tudás** megőrzése — nem a kód maga, hanem a kódról szóló tudás.

Kulcs üzenet PM-nek

Az AI ágens memóriával nem felejt. Ami egyszer megtörtént — döntés, hiba, konvenció — azt emlékszik és használja. Két rétegben működik: a CLAUDE.md adja a statikus szabályokat, a Developer Memory a dinamikus, automatikusan gyarapodó tapasztalatot. A PM-nek ez azért fontos, mert a korábbi döntésekre építhetünk anélkül, hogy újra meg kellene vitatni azokat. A csapat minden tagja (és minden AI ágense) ugyanazt a tudást látja, automatikusan szinkronizálva. Mint egy tökéletes jegyzőkönyv, amit mindenki ír és mindenki olvas — csak éppen automatikusan.

7. fejezet

A szoftverfejlesztés jövője

7.1 Hol tartunk most?

2026 elején a szoftverfejlesztés egy történelmi átmenet közepén van. Ahhoz, hogy megértsd mi történik, érdemes megnézni a számokat: a Stack Overflow Developer Survey 2025 szerint a fejlesztők többsége már használt valamilyen AI eszközt a munkájához, és a többség szerint ez érdemben megváltoztatta a napi munkájukat. A McKinsey felmérései hasonló képet mutatnak: az AI-alapú fejlesztői eszközök 20-50%-os produktivitásnövekedést hoznak a feladattól függően.

Az AI ágensek már képesek:

- Önállóan megoldani valós hibákat (SWE-bench: ~50%, és egyre javul)
- Specifikáció alapján implementálni feature-öket
- Párhuzamosan, koordináltan dolgozni (lásd V. fejezet: Orchestráció)
- Tanulni korábbi tapasztalatokból (lásd VI. fejezet: Memória)
- Teszteket írni, futtatni, és az eredmények alapján javítani a kódot
- Többszáz fájlból álló projektekben navigálni

De még nem képesek:

- Komplex architektúrákat tervezni az üzleti kontextus teljes megértésével
- Emberi csapatdinamikát kezelni
- Kreatív, újszerű megoldásokat kitalálni ahol nincs precedens
- 100%-ban megbízható, production-ready kódot írni review nélkül
- A "miért" kérdést megválaszolni – miért pont ezt a feature-t építjük
- Üzleti prioritásokat felállítani

Hol tartunk tehát? Valahol a "hasznos eszköz" és az "önálló fejlesztő" között. A Gartner legfrissebb előrejelzései szerint 2027-re a szoftverfejlesztő csapatok 80%-a használjon majd valamilyen AI ágenst – de teljes autonómiát még nem várunk.

> **Ajánlott videó**: A Two Minute Papers YouTube csatorna rendszeresen összefoglalja az AI legújabb eredményeit könnyen emészthető formában – érdemes követni a fejlődést.

7.2 Három időhorizont

7.2.1 Rövid táv: 2025–2026 – AI mint páros programozó

```
+-------------------------------------------+
|  MA: AI Pair Programming                   |
|                                           |
|  Fejleszto <-----------> AI Agens          |
|     |                     |               |
|     |  "Javitsd ki ezt"    |               |
|     |-------------------> |               |
|     |                     |  javit        |
|     |  <------------------|  tesztel      |
|     |  "Kesz, nezd meg"    |               |
|     |                     |               |
|  Ember iranyit, AI vegrehajt               |
|  Ember review-ol minden valtoztatast       |
+-------------------------------------------+
```

Ez van most. A fejlesztő megmondja mit kell csinálni, az AI megcsinálja, a fejlesztő ellenőrzi. A PM a fejlesztővel egyeztet, aki az AI-t irányítja.

Hogyan néz ki egy PM napja ebben a fázisban?

Reggel: Átnézed a Jira board-ot. Észreveszed, hogy a fejlesztők az eddigi tempónál 2-3-szor gyorsabban zárnak le feladatokat. A sprintbe felvett feladatok felét már az első napra elvégezték. Utántöltés kell – reggeli standup-on átnézed a backlogot, és kiválasztod a következő prioritásokat.

Délelőtt: Egyeztetsz a senior fejlesztővel, aki megmutatja, hogyan használja a Claude Code-ot egy bonyolultabb feladatnál. Látod, hogy ő irányítja az AI-t, de azért a komplex részeknél még sokat gondolkodik. Megérted: az AI nem varázslat, hanem egy nagyon jó asszisztens.

Délután: Reviewolod a PR-eket (Pull Request – kódjavaslat). Most több PR jön, mint korábban. Néhány nagyon jó, néhány felületesen készült. Észreveszed: a minőség egyenetlen. Meg kell határoznod, mi a minimális minőségi mérce.

Este: Átgondolod a sprint vég becslését. Korrigálnod kell – a csapat gyorsabb, de a review és integráció még mindig ugyanannyi idő.

PM feladata: Ugyanaz mint eddig, de a fejlesztő gyorsabb. A becsléseket le kell kalibrálnod – ami eddig 3 nap volt, most 1 nap lehet. Fontos: az AI által generált kód review-ja **több időt** is igényelhet, mert a PM-nek meg kell győződnie a minőségről.

7.2.2 Közép táv: 2026–2028 – AI csapatok emberi felügyelettel

```
+-------------------------------------------+
|  HOLNAP: AI Csapatok                       |
|                                           |
|           Ember (Architect / PM)           |
|                   |                        |
|          Spec + Review                     |
|                   |                        |
|     +-------------+-------------+          |
|     |             |             |          |
|     v             v             v          |
|  +------+    +------+    +------+         |
```

```
|  |AI Ag.|    |AI Ag.|    |AI Ag.|         |
|  |  #1  |    |  #2  |    |  #3  |         |
|  |front |    |back  |    |test  |         |
|  +------+    +------+    +------+         |
|                                            |
|  Ember ad specifikaciot, AI csapat         |
|  dolgozik, ember review-olja a vegen       |
+--------------------------------------------+
```

Ez felé tartunk. Az orchestrációs réteg (V. fejezet) ennek az előfutára. Egy ember ír egy specifikációt, az AI csapat implementálja, az ember a végeredményt ellenőrzi.

Hogyan néz ki egy PM napja ebben a fázisban?

Reggel: Megnyitod az orchestrátor dashboardot (lásd V. fejezet). Látod, hogy az éjszaka indított 3 change-ből 2 kész van, 1 elakadt. Az elakadtat megnézed: az AI ágensnek nem volt elég kontextus a fizetési modul üzleti szabályairól. Írsz egy kiegészítést a specbe, újraindítod.

Délelőtt: Nem standup van, hanem "spec review". A product owner-rel és az architect-tel átnézitek a következő sprint specifikációit. A specek minősége kritikus – ha pontatlanok, az AI rossz eredményt ad. A beszélgetés nem arról szól, "ki csinálja meg", hanem arról, "hogyan írjuk le pontosan, mit akarunk".

Délután: 5 percet töltesz azzal, hogy átnézed az automatikus review eredményeket. Az AI review rendszer már a legtöbb problémát kiszűri. Csak a komplex architektúrális kérdések kerülnek eléd. Egynél látod, hogy az AI ágens egy új dependency-t húzott be – erről beszélned kell a security csapattal.

Este: Dashboardon látod a sprint metrikákat. A csapat produktivitása 4-5x-e a korábbinak, de a spec írás ideje megnőtt. Ez rendben van – a spec írás a legértékesebb tevékenység.

PM feladata: A specifikáció minősége kritikussá válik. Eddig ha a spec pontatlan volt, a fejlesztő megkérdezte. Az AI nem mindig kérdez – megpróbálja értelmezni. Jobb spec = jobb eredmény. A PM fókusza: **specifikációk írása, review koordinálása, és minőségi kapu definiálása**.

> **Ajánlott cikk**: A Harvard Business Review rendszeresen közöl cikkeket az AI hatásáról a menedzsmentre – érdemes követni az "AI and Management" témakörben.

7.2.3 Hosszú táv: 2028+ – AI-vezérelt fejlesztés emberi irányítással

```
+--------------------------------------------+
|  HOLNAPUTAN: AI-vezerelt fejlesztes        |
|                                            |
|  Ember: Vizio + Minoseg + Uzleti dontes   |
|                  |                         |
|          "Mit akarunk?"                    |
|          "Ez eleg jo?"                     |
|                  |                         |
|                  v                         |
|         +----------------+                |
|         |  AI Orchestrator |               |
|         |  (spec, design,  |               |
|         |   tasks, impl.,  |               |
|         |   test, deploy)  |               |
|         +----------------+                |
|                  |                         |
```

```
|          Ember: szuroproba review          |
|          Ember: release dontes             |
|                                            |
+--------------------------------------------+
```

Ez a legvalószínűbb hosszú távú kép. Az AI nemcsak implementál, hanem tervez is – az ember az üzleti víziót adja, és a minőségi kaput kontrollálja.

Hogyan néz ki egy PM napja ebben a fázisban?

Reggel: Kávés közben átnézed a heti "vision document"-et. Leírtad benne, hogy a következő hónapban milyen üzleti célokat akarsz elérni (pl. "20%-kal csökkenteni a checkout abandonment rate-et"). Az AI orchestrátor már lebontotta ezt feature-ökre, szúrópróba-szerű átválogatást végzel: "Ez jó irány? Ez összhangban van a stratégiával?"

Délelőtt: Két órát töltesz az üzleti stakeholder-ekkel. Az ő igényeiket fordítod le arra a nyelvre, amit az AI rendszer ért: világos célok, mérőszámok, sikerkritériumok. Ez a te legfontosabb képességed – senki más nem ért az üzleti és technikai világ között.

Délután: Az AI rendszer automatikusan deployolt egy új feature-t staging környezetbe. Kipróbálod, néhány dolog nem tetszik. Visszajelzést írsz – nem "javítsd ki a 42. sort", hanem "a checkout flow túl bonyolult, a felhasználó 3 lépés helyett 5-öt lát, egyszerűsítsd". Az AI orchestrátor újratervezi.

Este: Reviewolod a heti metrikákat. A csapatod (ami főleg AI ágensekből áll, néhány senior engineer felügyeletével) 50 feature-t szállított ki ezen a héten. A te dolgod: ebből melyik mozdította előrébb az üzleti célokat?

PM feladata: A PM szerepe a "mit" és "miért" kérdésekre fókuszálódik. Kevesebb "ki csinálja meg?" és "hol tartunk?" – több "mit csinálunk és miért?" és "elég jó ez a felhasználóknak?"

7.3 A PM szerep evolúciója

```
2024:     PM --> Jira ticket --> Dev --> Review --> Deploy
                 (reszletes)    (kodol)  (kezi)

2026:     PM --> Proposal --> AI Agent --> Auto-review --> PM OK --> Deploy
                 (mit+miert)  (spec->kod)  (tesztek)       (check)

2028+:    PM --> Vizio doc --> AI Orchestr. --> AI Review --> PM spot-check
                 (nagy kep)    (minden auto.)   (auto.)       (szuroproba)
```

Ami változik:

Terület	Régi PM	Új PM
Becslések	Napokban/hetekben gondolkodik	Órákban gondolkodik, kapacitást ágensekben méri
Státusz meeting	Napi standup, hetente retro	Dashboard nézegetés, ritkább sync
Spec írás	"Nice to have"	Kritikus – a spec minősége = a kód minősége
Review	Kódot nem néz, ticketet mozgat	Artifact-okat reviewol (proposal, spec)
Koordináció	Ember – ember	Ember –> spec –> AI csapat
Minőség	"A devek megoldják"	Minőségi kapu-t definiál és betartatja

Terület	Régi PM	Új PM
Kockázatkezelés	Manuális figyelemmel kísérés	AI által jelzett kockázatok értékelése
Stakeholder mgmt	Fejlesztői státusz közvetítése	Üzleti célú eredmények kommunikálása

7.4 Iparági szereplők – részletes összehasonlítás

Nem csak az Anthropic dolgozik ezen. Az AI-alapú fejlesztői eszközök piaca robbanásszerűen nő – a Sequoia Capital és az a16z kockázati tőkealapok milliárdos befektetésekkel támogatják ezt a területet. Íme a főbb szereplők részletesebb összehasonlítása:

7.4.1 Az eszközök részletesen

Claude Code (Anthropic) – anthropic.com/research

A könyvben részletesen tárgyalt eszköz. Erősségei: agentic (önállóan cselekvő) működési mód, hook rendszer, MCP szabvány, sub-ágensek, memória rendszer, orchestráció. Parancssori (CLI) és IDE integrációval is használható. A specifikáció-vezérelt megközelítés (OpenSpec) különösen erős. Gyengeségei: parancssori interface-e nem minden felhasználónak intuitív, és az Anthropic ökoszisztémára építkezik (bár az MCP szabvány nyitott).

GitHub Copilot – github.com/features/copilot

A Microsoft/GitHub terméke, a legelterjedtebb AI fejlesztői eszköz. Erősségei: mélyen integrált a GitHub ökoszisztémába (pull request-ek, issues, actions), széles IDE támogatás (VS Code, JetBrains, Neovim), Copilot Workspace ami projekt szinten dolgozik. Gyengeségei: az agent mód még kevésbé érett mint a Claude Code, és kisebb a kontextus-ablak.

Cursor – cursor.com

AI-natív IDE (fejlesztői környezet), ami a VS Code-ra épül. Erősségei: az egész IDE körüli élményt AI-ra tervezték, gyors és intuitív, több AI modellt támogat (Claude, GPT-4, stb.), a "Composer" feature komplex, többfájlos feladatokra képes. Gyengeségei: önmagában IDE, nincs orchestrációs réteg, és a fejlett agentikus képességek még fejlődés alatt állnak.

Devin – devin.ai

A Cognition Labs terméke, az első "teljes AI szoftverfejlesztő". Erősségei: teljesen autonóm – saját böngészővel, terminállal, és szerkesztővel rendelkezik, bonyolultabb feladatokat is képes önállóan megoldani. Gyengeségei: lassú (percek-órák egy feladatra), drága, és a minőség egyenetlen – néha brilliáns, néha katasztrofális. A felhasználó kevésbé tudja irányítani, mint a többi eszközt.

Amazon Q Developer – aws.amazon.com/q/developer/

Az Amazon AWS-be integrált AI fejlesztői eszköze. Erősségei: mélyen integrált az AWS ökoszisztémába (Lambda, DynamoDB, S3, stb.), biztonsági szkennelést is végez, és a nagyvállalati megfelelőség (compliance) területén erős. Gyengeségei: alapvetően AWS-fókuszú, más felhő-szolgáltatókkal való használata korlátolt.

Google Jules

A Google AI kódoló ágense. Korai fázisban van, a fejlesztés gyors. A Google Gemini modellek erejéből merít. Erősségei: a Google Cloud integráció és a Gemini modellek kontextus-ablaka (rekord méretű). Gyengeségei: még nem érett, a Google hajlamos termékeket megszüntetni (lásd: Google Graveyard).

7.4.2 Összehasonlító táblázat

Eszköz	Autonómia	IDE integr.	Orchestráció	Ár (2026)	Legjobb ha…
Claude Code	Magas	CLI + IDE	Van (wt-tools)	$$$	Specifikáció-vezérelt AI csapatokat akarsz
Copilot	Közepes	Kitűnő	Korlátolt	$$	Már GitHub-ot használsz és gyors kiegészítés kell
Cursor	Közepes	Natív IDE	Nincs	$$	Intuitív, vizuális fejlesztői élményt keresel
Devin	Nagyon magas	Saját env.	Van	$$$$	Teljes autonómiát akarsz és bírod a kockázatot
Amazon Q	Közepes	AWS konzol	Korlátolt	$$	AWS ökoszisztémában dolgozol
Jules	Közepes	GCP	Korlátolt	$$	Google Cloud-ban dolgozol

```
Autonomia szint:

Alacsony                                        Magas
+-------------+---------------+-----------------+
|             |               |                 |
Copilot    Cursor        Claude Code          Devin
(kiegeszit)  (szerkeszt)   (agens)          (autonom)
```

A trend egyértelmű: minden szereplő az autonómia felé mozog. A kérdés nem az, hogy az AI ágensek át fogják-e venni a rutin kódolási feladatokat, hanem **mikor** és **milyen szinten**. A Gartner elemzései szerint 2028-ra az AI ágensek a kódolási feladatok 70%-át elvégzik majd.

7.5 AI-natív vállalatok: az új paradigma

Egy teljesen új kategória jelent meg az utolsó évben: az **AI-natív vállalatok**. Ezek olyan startup-ok és cégek, amelyeket az első naptól kezdve AI ágensekkel való együttműködés köré építettek.

7.5.1 Mi az AI-natív vállalat?

Hagyományos startup: felépít egy 5-10 fős fejlesztői csapatot, behúzzák a laptopokat, Jira-t, Slack-et, és elkezdenek kódolni.

AI-natív startup: 1-2 technical founder + AI ágensek csapata. A "fejlesztői csapat" nagyrészt AI ágensekből áll, az emberek specifikációt írnak, reviewolnak, és üzleti döntéseket hoznak.

A Y Combinator – a világ legismertebb startup inkubátora – már 2025-ben jelezte, hogy egyre több AI-natív startup pályázik hozzájuk. Ezek a cégek:

- **1-3 ember** képviseli a teljes "fejlesztői csapatot"
- Specifikációkat írnak, nem kódot
- AI ágensek implementálnak, tesztelnek, deployolnak
- A fejlesztési költség a töredéke a hagyományosnak

- Hihetetlen gyorsan iterálnak – naponta több release

Példa: Képzeld el, hogy egy 2 fős startup 3 hónap alatt létrehoz egy teljes SaaS terméket (webes szoftver szolgáltatásként), ami korábban 10 fős csapatnak 1 évet vett volna igénybe. Az egyik alapító a termék vízióját és az üzleti stratégiát adja, a másik a technikai specifikációkat írja és az AI ágenseket irányítja.

7.5.2 Mit jelent ez a PM-nek?

Ha AI-natív cégnél dolgozol (vagy ilyenné válik a céged):

- A PM és a "tech lead" szerep összeolvadhat
- Specifikáció-írás a legfontosabb technikai képesség
- A csapat mérete nem a fejlesztők számáról szól, hanem az ágensek számáról és a spec minőségéről
- A "velocity" (fejlesztési sebesség) a sokszorosára nőhet
- Az ügyfél-visszajelzés integráció sokkal gyorsabb – reggelre megkapod a feedbacket, délutánra már kijavítod

  Ajánlott olvasmány: Az a16z (Andreessen Horowitz) rendszeresen közöl elemzéseket az AI-natív cégek trendjeiről, és arról hogyan változtatja meg az AI a szoftveripart.

7.6 Gazdasági hatás: számok és becslések

Az AI ágensek nem csak technológiai, hanem gazdasági forradalmat is hoznak. Érdemes megnézni a számokat, mert ezek segítenek a PM-nek meggyőzni a vezetőséget az AI eszközökbe való befektetésről.

7.6.1 Produktivitás-növekedés

A különböző kutatások 20-80%-os produktivitás-növekedést mérnek, a feladat típusától függően:

Feladat típusa	Produktivitás növekedés	Forrás
Rutin kódolás (boilerplate)	50-80%	McKinsey Digital
Hibakeresés és javítás	30-50%	Stack Overflow Survey
Teszt írás	40-60%	Belső mérések
Dokumentáció	60-80%	Iparági átlag
Komplex architektúra tervezés	5-15%	McKinsey
Új technológia tanulása	20-40%	Fejlesztői visszajelzések

Fontos: a produktivitás-növekedés NEM egyenletes. A rutin feladatoknál drámai, a kreatív és architektúrális feladatoknál minimális. Ez azt jelenti, hogy az AI főleg az "unalmas" munkát veszi át, és az emberi munkaerő a magasabb értékű feladatokra fókuszálhat.

7.6.2 Költségmegtakarítás

Egy példával szemléltetve:

```
HAGYOMANYOS FEJLESZTES (5 fos csapat, 6 honapos projekt):
-----------------------------------------------------------
5 fejleszto x 6 honap x ~1.5M Ft/ho (brutt. ber+koltseg) = 45M Ft
+ Infrastruktura, eszkozok:                                 5M Ft
+ PM, tesztelo, designer:                                  15M Ft
Osszesen:                                                  ~65M Ft

AI-TAMOGATOTT FEJLESZTES (2 fejleszto + AI agensek, 3 honap):
---------------------------------------------------------------
```

```
2 fejleszto x 3 honap x ~1.8M Ft/ho (magasabb ber):        10.8M Ft
+ AI eszkozok (Claude, infra):                               2M Ft
+ PM, designer:                                              5M Ft
Osszesen:                                                  ~18M Ft

Megtakaritas: ~72% (47M Ft)
Gyorsabb szallitas: 3 honap a 6 helyett
```

Megjegyzés: ezek becsült értékek, a konkrét számok projektenként változnak. A legfontosabb üzenet nem a pontos összeg, hanem a nagyságrend.

7.6.3 ROI (Return on Investment) számítás PM-eknek

Ha a vezetőséged kérdezi, "megéri-e AI eszközökbe fektetni?", itt egy egyszerű keret:

1. **Befektetés**: AI eszközök havidíja (tipikusan $50-500/fejlesztő/hó) + betanítás ideje (1-2 hét)
2. **Megtakarítás**: Fejlesztési idő csökkenés (tipikusan 30-60%) * fejlesztői bérköltség
3. **ROI**: Általában az első hónapban megtérül

A McKinsey felmérései szerint az AI eszközökbe való befektetés ROI-ja az iparági átlagban 3-5x a befektetett összegnek az első évben.

7.7 Kockázatok és kihívások

Minden technológiai forradalomnál vannak kockázatok. Ezeket fontos ismerni:

7.7.1 1. AI hallucináció

Az AI néha magabiztosan állít olyasmit, ami nem igaz. Kitalál API-kat, hivatkozik nem létező függvényekre, vagy logikusnak tűnő de hibás megoldásokat ad.

Példa: Az AI azt írja: "Használd a `request.user.getActiveSubscription()` metódust" – ami nem létezik a projektben. A kód lefordul, de futáskor hibát dob. Ha nincs teszt lefedettség, ez production-be kerülhet.

Mitigáció: Automatikus tesztek, kód review, és a spec-driven megközelítés (ami mérhető követelményeket ad). Minél több automatizált ellenőrzés van, annál kisebb a kockázat.

7.7.2 2. Biztonsági kérdések

Az AI ágens hozzáfér a fájlrendszerhez, futtat parancsokat, és potenciálisan érzékeny adatokkal dolgozik. Ha rosszindulatú input-ot kap (prompt injection), nem kívánt műveleteket végezhet.

Példa: Ha az AI ágens hozzáfér egy .env fájlhoz ami API kulcsokat tartalmaz, és a kód amit generál véletlenül kilogolja ezeket, biztonsági incidens történhet.

Mitigáció: Sandboxing (elszigetelt futtatás), jogosultsági rendszerek, és a hookrendszer ami blokkolhatja a veszélyes műveleteket (lásd II. fejezet). A legtöbb modern AI eszköz beépített biztonsági rétegeket tartalmaz.

7.7.3 3. Vendor lock-in

Ha egy csapat teljesen egy AI provider-re (pl. Anthropic, OpenAI) épít, annak kiesése megbéníthatja a munkát.

Példa: 2025 elején több AI szolgáltató is tapasztalt órákig tartó kieséseket. Azok a csapatok, amelyek kizárólag egy szolgáltatóra építettek, órákig nem tudtak dolgozni.

Mitigáció: A specifikáció-vezérelt megközelítés provider-független – a specek, design-ok, és taskok szöveges fájlok, amik bármely AI-val használhatók. Az MCP szabvány szintén provider-független. Érdemes "Plan B"-t tartani – pl. ha a Claude Code nem elérhető, a Cursor-t is ismerd.

7.7.4 4. Szabályozási kérdések – az EU AI Act

Az EU AI Act 2024-ben lép életbe fokozatosan, és 2026-ra már számos rendelkezés hatályos. Ez a világ első átfogó AI szabályozása, és közvetlen hatása van arra, hogyan használhatók AI ágensek a szoftverfejlesztésben.

Mi az EU AI Act lényege PM szemszögből?

Az EU AI Act kockázati szintek szerint kategorizálja az AI rendszereket:

Kockázati szint	Példa	Követelmény
Elfogadhatatlan	Social scoring, manipulatív AI	Tiltott
Magas kockázatú	Egészségügyi, pénzügyi, kritikus infra	Szigorú audit, dokumentáció, emberi felügyelet
Korlátozott kockázat	Chatbotok, AI generált tartalom	Átláthatósági követelmény (jelezni kell, hogy AI)
Minimális kockázat	AI kód-segédeszköz, spam filter	Nincs külön követelmény

Mit jelent ez a gyakorlatban egy fejlesztői csapatnak?

- Ha a projektedben **magas kockázatú** AI alkalmazást fejlesztesz (pl. pénzügyi döntés-támogató, egészségügyi szoftver), az AI ágens által generált kódot külön dokumentálni és auditálni kell
- Az AI ágensek használata a fejlesztésben (mint eszköz) általában **minimális kockázat** – nincs külön követelmény
- DE: ha az AI ágens által generált kód egy magas kockázatú rendszer része, a teljes fejlesztési folyamatot dokumentálni kell

A spec-driven megközelítés előnye: A specifikáció-vezérelt fejlesztés (lásd III. és IV. fejezet) teljes audit trail-t biztosít – minden döntés, változás, és AI-művelet dokumentálva van git-ben. Ez pont az, amit az EU AI Act megkövetel.

> **Hivatalos forrás**: Az EU AI Act teljes szövege és magyarázata: artificialintelligenceact.eu

7.7.5 5. Munkaerőpiaci hatás

Az AI ágensek csökkentik a rutin kódolási feladatokat. Ez egyes fejlesztői pozíciókat átalakíthat.

A World Economic Forum Future of Jobs Report 2025 szerint az AI nem annyira megszüntet munkahelyeket, mint inkább átalakítja őket. A szoftverfejlesztés területén:

- A **junior fejlesztői** pozíciók száma csökkenhet, de a szerepük átalakul (több review, kevesebb boilerplate kód írás)
- A **senior fejlesztői** és **architekti** szerepek értéke nő (ők irányítják az AI ágenseket)
- Új pozíciók jelennek meg: "AI Engineer", "Prompt Engineer", "AI Operations"
- A **PM szerep** értéke nő, mert a specifikáció-írás és az üzleti-technikai fordítás egyre fontosabb

Mitigáció: A fejlesztői szerep átalakulása – kevesebb kódírás, több architektúra, review, és AI-felügyelet. Hasonló ahhoz, ahogy a számítógépek nem szüntették meg a könyvelést, hanem átalakították. A táblázatkezelők megjelenése után nem kellett kevesebb könyvelő, hanem másképp dolgozó könyvelő.

7.7.6 6. Minőségi és megbízhatósági kockázat

Az AI gyorsan generálja a kódot, de a minőség egyenetlen. Ha a csapat elbizakodik és csökkenti a review-t, a technikai adósság gyorsan felhalmozódhat.

Mitigáció: Automatizált minőségi kapuk (tesztek, lint, security scan), és a "trust but verify" (bízz benne, de ellenőrizd) megközelítés. A PM feladata: definiálni a minőségi minimumot, és ragaszkodni hozzá.

7.8 Gyakori tévhitek az AI-ról a szoftverfejlesztésben

Sok félreértés kering az AI ágensekről. Itt a leggyakoribbak, és a valóság mellettük:

7.8.1 Tévhit #1: "Az AI helyettesíti a fejlesztőket"

Valóság: Az AI átalakítja a fejlesztői szerepet, de nem szünteti meg. Pont úgy, ahogy az Excel nem szüntette meg a könyvelőket, és az AutoCAD nem szüntette meg az építészeket. A fejlesztők magasabb szintű munkára fókuszálnak: architektúra, review, és az AI irányítása. A Stack Overflow Developer Survey 2025 szerint a fejlesztők többsége pozitívan értékeli az AI eszközöket, és nem fél a munkahely elvesztésétől.

7.8.2 Tévhit #2: "Az AI által generált kód rossz minőségű"

Valóság: Az AI által generált kód minősége erősen függ a specifikáció minőségétől és a review folyamattól. Egy jó specifikációt kapó AI ágens gyakran jobb kódot ír, mint egy siető junior fejlesztő – mert következetesen betartja a stílus-konvenciókat, minden esetben ír teszteket, és nem felejt el hibakezelést. A kulcs: a spec minősége határozza meg az output minőségét.

7.8.3 Tévhit #3: "Csak programozóknak hasznos"

Valóság: Az AI ágensek PM-eknek is közvetlenül hasznosak:

- Specifikációk validálása (az AI "eljátssza" az implementációt és jelezheti a lyukakat)
- Projekt progress automatikus nyomon követése
- Technikai dokumentáció érthetővé fordítása
- "Mi lenne ha" forgatókönyvek gyors kipróbálása

7.8.4 Tévhit #4: "Drága és jó AI eszközök kellenek, olcsóbb nem használni"

Valóság: A legtöbb AI fejlesztői eszköz $20-100/hó/fejlesztő árban mozog. Ha egy fejlesztő óra-bére $50-150, és az AI eszköz napi 1 órát megtakarít, az eszköz az első nap megtérül. Az ROI (megtérülés) szinte mindig pozitív az első héten belül.

7.8.5 Tévhit #5: "Nem kell hozzá technikai tudás"

Valóság: Nem kell kódolni tudni, de értened kell az alapfogalmakat (API, adatbázis, tesztek, deployment). Hasonló, mint az autóvezetéshez: nem kell autószerelőnek lenned, de értened kell a közlekedési szabályokat. Ez a könyv pont ezt adja: a szükséges minimum technikai kontextust.

7.8.6 Tévhit #6: "Az AI kód biztonságilag veszélyes"

Valóság: Az AI által generált kód nem veszélyesebb, mint az ember által írt kód – feltéve, hogy megfelelő review és tesztelés folyamat van. Sőt, az AI következetesebben követ biztonsági best practice-eket, mert nem felejt, nem siet, és nem "majd később megcsinálom". A kockázat nem maga az AI, hanem a review nélküli elfogadás.

7.9 Ami NEM fog eltűnni

Az AI forradalom közepén fontos tisztán látni, mi marad változatlanul értékes:

```
+---------------------------------------------+
|  EMBERI ERTEKEK AMIK MEGMARADNAK             |
|                                              |
|  * Uzleti gondolkodas                        |
|    "Mit akar a felhasznalo?"                 |
|    "Melyik feature hoz bevetelt?"            |
|    "Melyik piacra lepjunk be?"               |
|                                              |
|  * Architekturalis dontesek                  |
|    "Monolitikus vagy mikroszervizes?"         |
|    "Cloud vagy on-premise?"                   |
|    "Build vs. buy?"                           |
|                                              |
|  * Csapat es kultura                         |
|    "Hogyan dolgozunk egyutt?"                |
|    "Mi a csapat erossege?"                   |
|    "Hogyan motivaljuk az embereket?"         |
|                                              |
|  * Minosegi merce                            |
|    "Eleg jo ez a felhasznaloknak?"           |
|    "Megbizhatunk ebben?"                      |
|    "Skalazodik ez 10x felhasznalora?"        |
|                                              |
|  * Etikai es uzleti itelokeepesseg           |
|    "Szabad ezt csinalnunk?"                  |
|    "Mi a kockazat?"                          |
|    "Osszhangban van ez az ertekeinkkel?"     |
|                                              |
|  * Kreativitas es innovacio                  |
|    "Mi lenne ha teljesen maskepp csinalnak?" |
|    "Milyen problemat nem lat meg senki?"     |
|    "Hogyan lephetjuk meg a felhasznalokat?"  |
|                                              |
|  * Empatia es felhasznaloi szemlelet        |
|    "Milyen erzest kelt ez a termek?"         |
|    "Erti-e a nagymamam is?"                  |
|    "Mi frusztralja a felhasznalot?"          |
|                                              |
+---------------------------------------------+
```

A szoftverfejlesztés nem a kódolásról szól – a problémamegoldásról szól. Az AI átveszi a kódolás jelentős részét, de a probléma meghatározása, a prioritások felállítása, és a végeredmény értékelése emberi feladat marad.

Gondolj erre: az AI brilliánsan tud kódot írni, de nem tudja megmondani, hogy érdemes-e egyáltalán megírni. Nem tudja, hogy a felhasználóid többsége mobilról használja a terméked, ezért a desktop verzió kevésbé fontos. Nem tudja, hogy a fő versenytársad épp hasonló feature-ön dolgozik, ezért gyorsabbnak kell lenned. Ezek

emberi döntések, és a PM az, aki ezeket meghozza.

> **Ajánlott cikk**: Az Anthropic kutatási oldala rendszeresen közöl anyagokat arról, hogyan gondolkodnak a felelős AI fejlesztésről – érdemes olvasni, hogy megértsd az AI képességeit és korlátait a fejlesztők szemével.

7.10 Képességek, amiket egy PM-nek most érdemes fejleszteni

A következő években a legértékesebb PM-ek azok lesznek, akik az alábbi képességeket elsajátítják:

7.10.1 1. Specifikáció-írás (Spec Writing)

Ez a legfontosabb új PM képesség. A specifikáció az a dokumentum, ami leírja, MIT kell megcsinálni, MIÉRT, és HOGYAN mérjük a sikert. Ha AI ágensek implementálnak, a spec minősége egyenesen arányos az eredmény minőségével.

Gyakorlati tanács:

- Írj SMART célokat (Specific, Measurable, Achievable, Relevant, Time-bound)
- Használd az OpenSpec formátumot (lásd IV. fejezet) – proposal, design, tasks
- Minden specben legyen: elvárás, sikerkritérium, és "anti-goal" (mit NEM akarunk)
- Kérj AI-tól visszajelzést a specre: "Van valami ami nem világos ebben a specben?"

7.10.2 2. AI műveltség (AI Literacy)

Nem kell AI kutatónak lenned, de értened kell az alap fogalmakat: mi az LLM, mi az a kontextus-ablak, mit jelent a hallucináció, mi az agent loop. Ez a könyv ennek az alapját adja.

Gyakorlati tanács:

- Olvasd el ezt a könyvet :)
- Kövesd a Two Minute Papers YouTube csatornát a legújabb fejleményekért
- Próbáld ki személyesen a ChatGPT-t, Claude-ot, és a Cursor-t – a saját élményed pótolhatatlan
- Olvass rendszeresen: HBR AI cikkek, McKinsey AI insights

7.10.3 3. Prompt engineering alapok

A "prompt" az az utasítás, amit az AI-nak adsz. A jó prompt = jó eredmény. PM-ként nem kell expert szinten értened, de az alapok fontosak:

Gyakorlati tanács:

- Légy specifikus: "Írj egy kereső funkciót" rossz. "Írj egy kereső funkciót ami a felhasználó név és email cím alapján keres a users táblában, és az eredményeket paginálta adja vissza, oldalanként 20 találat" jó.
- Adj kontextust: mondd el mi a projekt, ki a felhasználó, mi a cél
- Kérj strukturált outputot: "A válaszodat 3 részre oszd: probléma, megoldás, kockázatok"
- Iterálj: ha az első eredmény nem jó, pontosítsd az utasítást, ne add fel

7.10.4 4. Minőségi kapu (Quality Gate) definiálás

A PM feladata meghatározni: "mikor tekintünk késznek egy feladatot?" Ez AI ágensek esetén különösen fontos, mert az AI gyorsan szállít, de a "kész" definíciója megváltozott.

Gyakorlati tanács:

- Definiáld a "Definition of Done"-t (pl. tesztek átmennek, lint rendben, dokumentáció kész, review megvolt)

- Automatizáld ami automatizálható (CI/CD pipeline-ban)
- A PM-nek nem kell kódot olvasnia, de látnia kell: átment-e a teszteken, van-e review, megfelel-e a specnek

7.10.5 5. Változás-menedzsment (Change Management)

Az AI eszközök bevezetése a csapatba változás-menedzsmentet igényel. Nem elég telepíteni az eszközt – az embereket is fel kell készíteni.

Gyakorlati tanács:

- Kezdd kicsiben: 1 fejlesztő, 1 projekt, 1 sprint
- Mérd az eredményt: gyorsabb lett? jobb lett? a fejlesztő elégedett?
- Kommunikáld a sikereket és a tanulságokat
- Ne kényszerítsd: aki nem akarja használni, adj időt. A sikertörténetek magukért beszélnek.

7.11 Akcióterv: hogyan kezdj hozzá?

Ha eddig eljutottál a könyvben, valószínűleg felmerült a kérdés: "És most mit csináljak?" Itt egy gyakorlati akcióterv, lépésről lépésre:

7.11.1 1. hét: Ismerkedés (0 kockázat)

Nap	Tevékenység	Cél
Hétfő	Próbáld ki a ChatGPT-t vagy a Claude-ot egy egyszerű feladattal	Érzetet kapni arról, mit tud az AI
Kedd	Kérdezz az AI-tól a saját projektedről (általánosságban)	Látni, hogyan gondolkodik
Szerda	Nézd meg a GitHub Copilot demót	Érteni, mit lát a fejlesztő
Csütörtök	Nézd meg a Cursor demót	Másik megközelítés megismerése
Péntek	Beszélj egy fejlesztővel a csapatodból: használsz már AI-t?	Valós tapasztalat, nem marketing

7.11.2 2. hét: Első kísérlet (alacsony kockázat)

- Válassz egy kis, nem kritikus feladatot a backlogból
- Kérd meg egy fejlesztőt, hogy használja AI-t (ha még nem teszi)
- Mérd: mennyi ideig tartott? Milyen minőségű az eredmény?
- Hasonlítsd össze a korábbi becslésekkel

7.11.3 3. hét: Spec-driven próbálkozás (közepes kockázat)

- Írj egy részletes specifikációt egy közepes méretű feladathoz
- Használd az OpenSpec formátumot (lásd IV. fejezet)
- A fejlesztő az AI-t használja az implementációhoz
- Reviewold együtt az eredményt: a spec alapján mérhető az eredmény?

7.11.4 4. hét: Értékelés és következő lépések

- Gyűjtsd össze a tanulságokat: mi működött, mi nem?
- Számold ki a hatékonyság-növekedést

- Készíts egy rövid beszámolót a vezetőségnek
- Tervezd meg a következő sprintet az új tudás alapján

7.11.5 Folyamatos tevékenységek (1. hónaptól)

- Hetente 30 percet szánj AI hírek olvasására (McKinsey, HBR, a16z)
- Havonta próbálj ki egy új AI eszközt
- Negyedévente értékeld újra a becslési modelleket
- Évente komplett AI-stratégia felülvizsgálat

7.12 Az átmenet kezelése: csapat menedzsment az AI-korszakban

Az AI eszközök bevezetése nem csak technikai, hanem emberi kihívás is. Íme a leggyakoribb helyzetek és a megoldásuk:

7.12.1 "A fejlesztők félnek, hogy az AI elveszi a munkájukat"

Ez a leggyakoribb és legérthetőbb reakció. A valóság az, hogy az AI nem a fejlesztőket váltja ki, hanem a fejlesztés módját változtatja meg.

Mit tegyél PM-ként:

- Kommunikáld nyíltan: "Az AI eszköz, nem helyettesítő. Azért vezetjük be, mert hatékonyabbak leszünk, nem mert kevesebb emberre van szükségünk."
- Mutasd meg a pozitív oldalát: "Kevesebb boilerplate kódot kell írnod, több idő jut az érdekes feladatokra."
- Adj példát más iparágakból: "Az Excel nem szüntette meg a könyvelőket. Az AutoCAD nem szüntette meg az építészeket."

7.12.2 "Egyes fejlesztők lelkesek, mások szkeptikusak"

Ez természetes. Minden új technológiánál vannak korai alkalmazók és szkeptikusok.

Mit tegyél PM-ként:

- Ne kényszerítsd: hadd az önkéntesek kezdjék
- A korai alkalmazók sikereiről szólj a csapatnak
- A szkeptikusok konkrét aggodalmaikra adj konkrét választ
- Adj időt: általában 2-3 hónap kell, mire a többség átáll

7.12.3 "A review folyamat még nem alkalmazkodott"

Ha az AI 5x annyi kódot generál, az eddigi review folyamat nem skálázódik.

Mit tegyél PM-ként:

- Automatizáld ami automatizálható: lint, tesztek, security scan
- A manuális review fókuszáljon az architektúrára és az üzleti logikára, ne a stílusra
- Fontold meg: az AI-t is bevonhatod a review-ba (AI review + emberi spot-check)
- Definiáld, mit KELL embernek reviewolnia, és mit ELÉG automatizáltan

7.12.4 "Nem tudjuk hogyan becsüljünk"

A régi becslési modellek nem működnek. Ha a fejlesztő most 3x gyorsabb, de a review ugyanannyi idő, az összkép komplex.

Mit tegyél PM-ként:

- Az első 2-3 sprintben ne becsülj, hanem mérj: mennyi ideig tart valóban?
- Építs új referenciapontokat az AI-támogatott fejlesztésre
- Különböztesd meg: "AI implementáció ideje" + "review ideje" + "integráció ideje"
- A becslések pontossága 2-3 sprint után javul

7.12.5 "A vezetőség túl nagy elvárásokat támaszt"

Ha a vezetőség hall arról, hogy "az AI 10x gyorsabb", irreális elvárásokat támaszthat.

Mit tegyél PM-ként:

- Mutass valós számokat a saját csapatodból, ne marketing anyagokból
- Kommunikáld: "Az AI a kódolást gyorsítja, de a tervezés, review, és integráció ideje nem változik drasztikusan"
- Adj reális becsléseket: "30-50% össz-gyorsulás, nem 10x"
- A megtakarított időt használd jobb minőségre, nem több feature-re

7.13 Összefoglalás: mit tegyen most egy PM?

Lépés	Mit	Miért	Mikor
1. Ismerd meg	Próbáld ki a Claude Code-ot (lásd Függelék)	Személyes tapasztalat nélkül nem értheted meg	Ez a hét
2. Gondolkodj spec-ben	Kezdj részletesebb specifikációkat írni	A spec minősége = az AI output minősége	Most azonnal
3. Kalibráld újra	A becslési modelleket igazítsd	Ami 5 nap volt, lehet 1 nap + review	Következő sprint
4. Fókuszálj minőségre	Definiáld a "kész" definícióját	Az AI gyorsan ír kódot – de jót?	Következő sprint
5. Kísérletezz	Egy kisebb feladaton próbáld ki a flow-t	Alacsony kockázat, nagy tanulság	2 héten belül
6. Tanulj	Olvass, nézz videókat, beszélgess	Ez a terület gyorsan változik	Folyamatosan
7. Kommunikálj	Oszd meg a tanulságokat a csapattal és a vezetőséggel	A változás-menedzsment a PM feladata	Folyamatosan
8. Iterálj	Értékeld újra negyedévente: mi működik, mi nem?	Az AI eszközök és a legjobb gyakorlatok gyorsan változnak	Negyedévente

Kulcs üzenet PM-nek

A PM szerep nem tűnik el – átalakul. Kevesebb "ki csinálja meg?" és "mikor lesz kész?" – több "mit csinálunk és miért?" és "elég jó ez?". A legjobb PM-ek azok lesznek, akik értik az AI ágensek képességeit és korlátait, és tudják hogyan kell jó specifikációkat írni. A specifikáció-írás, az AI műveltség, és a minőségi kapuk definiálása – ezek lesznek a PM legfontosabb képességei az előttünk álló évtizedben. A World Economic Forum ezt a képet erősíti: az AI nem munkahelyeket szüntet meg, hanem szerepeket alakít át. Ez a könyv az első lépés ezen az úton – de a tanulás soha nem áll meg.

8. fejezet

Függelék

8.1 A. Szószedet

Angol kifejezés	Magyar jelentés	Rövid leírás
A/B Testing	A/B tesztelés	Két (vagy több) változat párhuzamos tesztelése valós felhasználókkal, hogy mérhető legyen, melyik működik jobban. A PM számára fontos, mert adatalapúvá teszi a döntéshozatalt.
Agent / Ágens	Ágens, ügynök	AI rendszer ami önállóan cselekszik: fájlokat olvas, kódot ír, parancsokat futtat. Nem csak válaszol, hanem csinálja is amit kell – ez különbözteti meg egy egyszerű chatbottól.
Agentic Loop	Ágentikus ciklus	Az AI gondolkodási ciklusa: tervez -> cselekszik -> ellenőriz -> ismétli. Addig fut, amíg a feladat kész nincs. Általában látod a terminálban, ahogy "gondolkodik".
API	Alkalmazás-programozási felület	Szabványos mód, ahogy szoftverek egymással kommunikálnak. Gondolj rá úgy, mint egy étterem rendelő ablakára: beküldöd a kérést, visszakapod az eredményt.
Artifact	Műtermék, termék	Az OpenSpec-ben: egy dokumentum (proposal, spec, design, tasks). Minden openspec változásnak megvannak a maga artifact-jai.

Angol kifejezés	Magyar jelentés	Rövid leírás
Blue-Green Deployment	Kék-zöld telepítés	Két azonos éles környezet közötti kapcsolás: az egyiken fut az aktuális verzió (kék), a másikra telepíted az újat (zöld), majd átkapcsolod a forgalmat. Ha valami elromlik, egyetlen kattintással vissza tudsz állni.
Branch	Ág, elágazás	A kód egy párhuzamos verziója amiben valaki dolgozik. Mint egy dokumentum másolata, amiben szabadon változtathatsz anélkül, hogy az eredetit elrontanád.
CI/CD	Folyamatos integráció és szállítás	Automatikus rendszer ami minden kódváltoztatás után teszteket futtat (CI), és ha minden zöld, automatikusan telepíti az élesbe (CD). A fejlesztés "szállítószalagja".
CLI	Parancssori felület	Szöveges felület ahol parancsokat írsz (terminál). Nincs grafikus gomb, mindent gépeléssel irányítasz. A Claude Code is így működik.
Code Coverage	Kódlefedettség	Százalékos mérőeszköz: a tesztek a kód hány százalékát "fedik le", azaz tesztelik. 80% felett általában jó – de a szám önmagában nem garantálja a minőséget.
Commit	Mentési pont	Egy változtatáscsomag elmentése a verziókezelőbe. Minden commit kap egy egyedi azonosítót és egy üzenetet, ami leírja, mit változtattunk.
Context Window	Kontextus ablak	Az AI munkamemóriája – amennyit egyszerre kezelni tud. Ha túl sok információt adsz neki, a régebbi részeket "elfelejti". Claude-nál ez jelenleg ~200 000 token.
DAG	Irányított körmentes gráf	Függőségi térkép: mi függ mitől (orchestrációnál). Biztosítja, hogy a feladatok helyes sorrendben fussanak.
Deploy	Telepítés, élesítés	A kód feltöltése az éles szerverre, ahol a valós felhasználók hozzáférnek.

Angol kifejezés	Magyar jelentés	Rövid leírás
Design	Tervezési dokumentum	Technikai döntéseket rögzítő dokumentum. Az OpenSpec-ben a “hogyan?” kérdésre válaszol.
Docker	Docker konténer-technológia	Egy technológia ami a szoftvert és minden függőségét egy “dobozba” (konténerbe) csomagolja. Így garantáltan ugyanúgy fut mindenhol – a fejlesztő gépén, a teszt szerveren és az élesben is.
Feature Flag	Funkció-kapcsoló	Olyan kapcsoló a kódban, amivel egy új funkcióváltozatot be- vagy kikapcsolhatsz anélkül, hogy újra kellene telepíteni. PM-ként fontos: fokozatos bevezetésnél használják (pl. először csak 5% látja az új funkcióváltozatot).
Git	Git verziókezelő	A legelterjedtebb kód-verziókövető rendszer. A kódot és annak teljes történetét tárolja. Gyakorlatilag minden modern szoftverprojekt használja.
GraphQL	GraphQL lekérdező nyelv	Modernebb alternatíva a REST API-hoz. A kliens pontosan megmondhatja, milyen adatokat kér – se többet, se kevesebbet. A Facebook fejlesztette.
Hallucination	Hallucináció	Amikor az AI magabiztosan állít olyasmit, ami nem igaz. Nem “hazudik” szándékosan – a statisztikai mintaillesztés néha hibás eredményt ad. Ezért fontos az emberi ellenőrzés.
Hook	Kampó, automatizmus	Automatikus művelet ami egy eseményre reagál. Például: “minden commit előtt futtasd le a teszteket.”
IDE	Integrált fejlesztőkörnyezet	Szoftver amiben a fejlesztők dolgoznak (pl. VS Code, JetBrains). Szövegszerkesztőnél jóval több: szín-kiemelést, hibakeresést és külső eszközöket is ad.

Angol kifejezés	Magyar jelentés	Rövid leírás
Kubernetes	Kubernetes (K8s) konténer-orchestráció	Rendszer ami Docker konténereket kezel nagy léptékben: automatikusan indít, leállít és skáláz. Ha a Docker a "doboz", a Kubernetes a "raktárvezető" aki eldönti, hova, hányat és mikor.
Linter	Kódstílus-ellenőrző	Automatikus eszköz ami a kód formai és stilusbeli hibáit jelzi (pl. hiányzó pontosvessző, nem konzisztens névkonvenció). Nem a működését vizsgálja, hanem az olvashatóságát és az egységességét.
LLM	Nagy nyelvi modell	Az AI motor ami a szöveget érti és generálja. A Claude, a GPT és a Gemini mind LLM-ek. Hatalmas mennyiségű szövegen tanultak, ezért értik a kontextust.
MCP	Modell Kontextus Protokoll	Szabvány az AI és külső eszközök összekapcsolására. Az MCP "USB-csatlakozóként" működik az AI világban: egységes módot ad arra, hogy az AI külső adatforrásokhoz és eszközökhöz kapcsolódjon.
Merge	Beolvasztás, összefésülés	Két kódverzió egyesítése. Néha konfliktus keletkezik, ha ugyanazt a sort ketten változtatták – ilyenkor valakinek kézzel kell döntenie.
Microservices	Mikroszolgáltatások	Architektúrális megközelítés, ahol a szoftvert sok kis, független szolgáltatásra bontják (pl. felhasználókezelés, fizetés, értesítések külön-külön). Előnye: egymásra váró csapatok nélkül lehet fejleszteni. Hátránya: összetettebb üzemeltetés.
Monorepo	Monorepo	Egyetlen, közös Git repository amiben a projekt összes komponense él (frontend, backend, közös könyvtárak). Előnye: egyszerű függőségkezelés. Hátránya: nagyon nagy tud lenni.

Angol kifejezés	Magyar jelentés	Rövid leírás
Orchestration	Orchestráció, karmesterség	Több AI ágens koordinált, párhuzamos futtatása. Mint egy karmester aki vezényel: elosztja a feladatokat, figyeli a haladást, és összefogja a végeredményt.
Pipeline	Csővezeték, munkafolyamat	Lépések sorozata egy cél elérésére. A CI/CD pipeline például: tesztel -> építés -> telepítés.
Production	Éles környezet	A szoftver azon verziója amit a felhasználók használnak. Ha "prodba mentünk" – a felhasználók már látják az új verzióváltozatot.
Prompt	Utasítás, kérés	Az AI-nak adott szöveges utasítás. A prompt minősége alapvetően meghatározza a válasz minőségét – ezért mondják: "garbage in, garbage out".
Proposal	Javaslat	"Mit és miért?" dokumentum az OpenSpec-ben. A PM legfontosabb eszköze: itt lehet emberi nyelven megfogalmazni, mit szeretnénk.
PR / Pull Request	Beolvasztási kérelem	Kérés hogy a változtatásaid kerüljenek be a fő kódba. A fejlesztő (vagy az AI) "bemutatja" a munkáját, mások átnézik és jóváhagyják.
Refactoring	Refaktorálás, átrendezés	A kód belső szerkezetének javítása a működés változtatása nélkül. Mint egy lakásban a bútor átrendezése: ugyanazok a bútorok, de használhatóbb az elrendezés. A PM számára azért fontos, mert általában nincs látható eredménye, de hosszú távon felgyorsítja a fejlesztést.
REST API	REST API	A legelterjedtebb mód ahogy webes rendszerek kommunikálnak egymással. Egyszerűen: URL-ekre küld kéréseket (GET, POST, PUT, DELETE) és JSON formátumban kap adatokat vissza.

Angol kifejezés	Magyar jelentés	Rövid leírás
Retrospective	Retrospektív	Sprint végi megbeszélés ahol a csapat átnézi, mi ment jól, mi nem, és min kellene javítani. A PM egyik legfontosabb eszköze a folyamatos javuláshoz.
Review	Áttekintés, ellenőrzés	Kód vagy dokumentum felülvizsgálata. Az AI korszakban kiemelt fontosságú, mert az AI által írt kódot is ugyanúgy át kell nézni.
Rollback	Visszaállás, visszagörgetés	Az előző verzió visszaállítása, ha az új hibás. Mint egy "Ctrl+Z" az egész rendszerre. Ezért fontos, hogy minden deploy visszaállítható legyen.
Ralph Loop	Ralph ciklus	Autonóm munkaciklus ahol az AI ismétlődően dolgozik feladatokon. A wt-tools saját fogalma.
Sandbox	Homokozó	Elszigetelt környezet ahol az AI biztonságosan futtathat parancsokat. Nem fér hozzá a rendszer többi részéhez.
Session	Munkamenet	Egy beszélgetés/munkaszakasz az AI-val. Amikor elindítod a `claude` parancsot és bezárod – ez egy session.
Spec / Specification	Specifikáció	Pontos, mérhető követelményeket leíró dokumentum. Az OpenSpec-ben a "mit, pontosan?" kérdésre válaszol.
Spec-Driven	Specifikáció-vezérelt	Fejlesztési megközelítés ahol minden a specifikációból indul. Először leírod mit akarsz, aztán az AI megvalósítja.
Sprint	Sprint	Általában 1-2 hetes fejlesztési ciklus a Scrum módszertanban. A sprint elején tervezel, a végén demózol és retrospektívet tartasz.
Standup	Napi gyorsegyeztetés	Rövid (általában 15 perces) napi megbeszélés ahol mindenki három kérdésre válaszol: mit csináltam tegnap? Mit fogok csinálni ma? Van-e akadály? Az AI korszakban érdemes kiegészíteni: "Mit delegáltam AI-nak?"

Angol kifejezés	Magyar jelentés	Rövid leírás
Subagent	Alágens	Kisebb, specializált AI ami a fő ágens irányítása alatt dolgozik. Az orchestrációban az egyes feladatokat ezek végzik.
Task	Feladat	Egy konkrét, elvégzendő munkadarab. Az OpenSpec-ben: egy checkbox a tasks.md-ben.
Technical Debt	Technikai adósság	Olyan technikai megoldások, amik most működnek, de hosszú távon problémát okoznak. Mint egy lakáshitel: most beköltözhetsz, de a törlesztést fizetned kell. Ha túl sok technikai adósság gyűlik fel, a fejlesztés drasztikusan lelassul.
Token	Token	Az AI szövegfeldolgozási egysége (~0.75 szó angolul, magyarul több token esik egy szóra). A költség és a kontextusablak mérete tokenben mérhető.
TypeScript	TypeScript	A JavaScript típusos bővítménye. A legtöbb modern webes projektet TypeScript-ben írják, mert a típusok segítenek a hibák korai felismerésében.
Vibe Coding	Hangulat-kódolás	AI-val való informális, tervezés nélküli kódolás. Gyors prototípusokra jó, éles projektekre kockázatos.
Worktree	Munkafa	A git egy funkciója: egy repo több példánya a lemezen. Az orchestráció ezeket használja, hogy több AI ágens párhuzamosan, egymás zavarása nélkül dolgozhasson.

8.2 B. Linkgyűjtemény

8.2.1 Anthropic és Claude Code

Link	Leírás
code.claude.com/docs	Claude Code hivatalos dokumentáció
Claude Code Best Practices	Bevált gyakorlatok Claude Code használatához
Claude Code Hooks	Hook rendszer útmutató
Claude Code Sub-agents	Subágensek használata
CLAUDE.md dokumentáció	Projekt memória fájl
claude.ai	Claude AI webes felület – telepítés nélkül kipróbálható
anthropic.com	Anthropic cég honlapja
Anthropic árazás	Előfizetési csomagok és árak

8.2.2 MCP (Model Context Protocol)

Link	Leírás
modelcontextprotocol.io	MCP bevezető – mi ez és miért fontos
MCP dokumentáció	Architektúra és működés
MCP specifikáció	A teljes MCP szabvány – referenciadokumentáció

8.2.3 Kutatás és benchmarkok

Link	Leírás
SWE-bench kutatás	Claude teljesítménye valós hibák javításában
SWE-bench weboldal	A szoftverfejlesztési benchmark hivatalos oldala

8.2.4 Iparági eszközök

Link	Leírás
GitHub Copilot	Microsoft/GitHub AI kódolási eszköz
Cursor	AI-natív kódszerkesztő
Devin	Cognition Labs autonóm AI fejlesztő

8.2.5 Blogok és cikkek

Link	Leírás
Simon Willison blogja	Az egyik legjobb forrás AI-támogatott fejlesztésről. Willison a Django framework egyik megalkotója, és rendszeresen ír arról, hogyan használja az LLM-eket a mindennapi munkában.
Martin Fowler	A szoftverarchitektúra és a clean code guruja. A “Refactoring” könyv szerzője. Az oldalán található cikkek segítenek megérteni, mit jelent a “jó kód” – ami azért fontos, mert az AI-nak is jó kódot kell írnia.
Kent Beck: Tidy First?	Kent Beck (az Extreme Programming megalkotója) Substack-je arról, hogyan érdemes a kódot kis lépésekben rendbe tenni. PM-ként azért érdekes, mert segít megérteni, miért kérnek a fejlesztők “refaktorálási időt”.
Addy Osmani blogja	A Google mérnökségi vezetője a Chrome csapatnál. Fejlesztői produktivitásról és AI eszközök hatékonyságáról ír – PM szemmel is emészthető stílusban.

8.2.6 YouTube csatornák

Link	Leírás
Anthropic	Az Anthropic hivatalos csatornája: termékbemutatók, kutatási eredmények és interjúk. Elsőkézből kapott információ a Claude fejlődéséről.
Fireship	Rövid (2-5 perces), lényegretörő tech videók. Ha gyorsan meg akarsz érteni egy technológiát (Docker, Kubernetes, TypeScript), itt 5 perc alatt megkapod a lényeget.
Two Minute Papers	Akadémiai AI kutatásokat magyaráz el közérthetően, vizuálisan. “What a time to be alive!” – a jelmondata. Kitűnő ahhoz, hogy a PM értse, mi történik az AI kutatás élén.

Link	Leírás
3Blue1Brown	Vizuális matematikai és AI magyarázatok. Ha meg akarod érteni, hogyan működik belülről egy neurális háló – itt a legjobb helyen jársz. Nem kell hozzá matematikusnak lenni.
TechWorld with Nana	DevOps fogalmak érthetően: Docker, Kubernetes, CI/CD, cloud. Ha a fejlesztőcsapat ezekről beszél és te nem érted – ez a csatorna segít.

8.2.7 Podcastok

Link	Leírás
Lex Fridman Podcast	Hosszú, mélységű interjúk AI kutatóktól, CEO-któl és gondolkodóktól. Hallható benne Sam Altman, Dario Amodei (Anthropic CEO), Elon Musk és mások. Utazásra, futásra ideális.
Hard Fork (New York Times)	Kevin Roose és Casey Newton heti tech podcastja. Közérthető, szórakoztató, és mindig a legaktuálisabb tech híreket tárgyalja – PM-ként ezzel "képben" maradsz a tech világról.
Latent Space	AI engineering podcast. Kicsit technikusabb, de ha egy PM komolyabban szeretné érteni az AI infrastruktúrát és eszközöket, ez a legjobb forrás.

8.2.8 Közösségek és tanulás

Link	Leírás
Anthropic Discord	Hivatalos Anthropic Discord szerver – itt kérdezni tudsz, és megtudod mások tapasztalatait Claude Code-dal.
r/ClaudeAI (Reddit)	Reddit közösség Claude felhasználóknak. Tippek, tapasztalatok, hibajelentések és workaround-ok.
Hacker News	A tech világ "hírportálja" – itt jelenik meg először a legtöbb fontos tech hír és diskurzus. Magas színvonalú hozzászólások.

8.2.9 Szabványok és szabályozás

Link	Leírás
EU AI Act	Az Európai Unió mesterséges intelligencia szabályozása. PM-ként fontos ismerni, mert befolyásolja, hogyan és milyen AI eszközöket használhatsz üzleti környezetben.
MCP specifikáció	A Model Context Protocol teljes specifikációja.

8.2.10 Iparági jelentések és kutatások

Link	Leírás
McKinsey Digital	A McKinsey digitalizációs és AI elemzései. Üzleti döntéshozó szemmel írt jelentések arról, hogyan változtatja az AI az ipart – prezentációkhoz és üzleti esetek alátámasztásához ideális.
Gartner AI kutatások	Gartner AI elemzései és előrejelzései. A "Hype Cycle" diagram különösen hasznos: megmutatja, hol tart egy-egy technológia az érési görbén.

Link	Leírás
Stack Overflow Developer Survey	Éves fejlesztői felmérés: milyen eszközöket használnak, mennyit keresnek, és hogyan látják az AI-t. Adatokkal alátámasztja az érdemi vitákat – ahelyett, hogy "szerintem" legyen az érv.

8.3 C. "Kipróbálom" – Quick Start

8.3.1 Ha nem akarsz semmit telepíteni

A legegyszerűbb mód a Claude kipróbálására a webes felület:

1. Menj a claude.ai oldalra
2. Regisztrálj egy ingyenes fiókot
3. Kezdj el beszélgetni

Ez nem ugyanaz, mint a Claude Code (nem fér hozzá a fájljaidhoz, nem futtat parancsokat), de kipróbálhatod rajta az AI gondolkodásmódját, prompt-írást, és dokumentumok elemzését. PM munkára (emailek fogalmazása, összefoglalók, brainstorming) már ez is hasznos.

8.3.2 Ha szeretnéd kipróbálni a Claude Code-ot

A Claude Code a parancssori eszköz, ami hozzáfér a fájljaidhoz és önállóan dolgozik. Ez a "valós" ágentikus élmény.

8.3.2.1 1. lépés: Telepítés (2 perc)

Nyiss egy terminált (Mac: Terminal.app, Windows: PowerShell, Linux: bármilyen terminál) és futtasd:

Mac / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Mire számíts: A telepítő letölt egy binárist és hozzáadja a rendszer PATH-jához. Nem kell hozzá admin jog. Ha a terminál nem ismeri fel a `claude` parancsot a telepítés után, zárd be és nyisd újra a terminált.

8.3.2.2 2. lépés: Bejelentkezés

```
claude
```

Az első indításnál a böngészőben bejelentkezel a Claude fiókodba. Szükséged lesz fizetős előfizetésre (lásd lentebb az árazást).

Mire számíts: Megnyílik egy böngészőfül ahol bejelentkezhetsz. Utána a terminál visszajelzi, hogy sikeres volt a bejelentkezés.

8.3.2.3 3. lépés: Próbáld ki egy projekten

Navigálj egy meglévő projektbe és indítsd el:

```
cd a-te-projekted
claude
```

Mire számíts: Az AI először "körülnéz" – megnézi a fájlstruktúrát, a README-t, és megpróbálja megérteni a projektet. Ez az első alkalommal néhány másodpercbe telhet.

8.3.2.4 4. lépés: Első parancsok

Próbáld ki ezeket:

```
"Magyarázd el, mit csinál ez a projekt"

"Keress hibákat a kódban"

"Írj teszteket a fő funkcióhoz"

"Javítsd ki a legfontosabb hibát amit találtál"
```

Mire számíts: Az AI olvasni fogja a fájlokat, gondolkodik (látni fogod a "thinking" állapotot), majd válaszol vagy csinál valamit. Az első kérdés után általában 10-30 másodperc múlva jön a válasz, a bonyolultabb feladatoknál akár percekig is dolgozhat.

8.3.2.5 5. lépés: Fedezd fel

- Nyomd meg a `Tab` billentyűt a kontextusváltáshoz (Plan Mode / Normal Mode)
- Írd be `/help` a parancsok listájáért
- Írd be `/init` egy CLAUDE.md fájl generálásához

8.3.3 Hibaelhárítás

Probléma	Megoldás
"command not found: claude"	Zárd be és nyisd újra a terminált. Ha még mindig nem működik, futtasd újra a telepítő parancsot.
"Authentication failed"	Ellenőrizd, hogy aktív-e az előfizetésed a claude.ai oldalon.
Az AI túl lassan válaszol	Nagy projekteknél az első válasz lassabb lehet. Próbáld kisebb kérdéssel kezdeni.
Az AI nem látja a fájljaimat	Ellenőrizd, hogy abban a könyvtárban vagy-e, ahol a fájlok vannak (`pwd` parancs).
"Rate limit exceeded"	Túl sok kérést küldtél rövid időn belül. Várj néhány percet és próbáld újra.

8.3.4 Mi legyen a következő lépés?

Ha már működik a Claude Code és kipróbáltad az alapokat:

1. **CLAUDE.md létrehozása**: Futtasd a `/init` parancsot a projektedben. Ez létrehoz egy CLAUDE.md fájlt ami az AI "memóriáját" tárolja a projektről. Legközelebb már tudni fogja, mi hogyan működik.
2. **Bevált promptok**: Próbáld ki ezeket a haladóbb kérdéseket:
   - "Nézd át a biztonsági szempontból kritikus részeket és jelöld a kockázatokat"
   - "Írj egy összefoglalót a projekt architektúrájáról – PM szemmel, nem fejlesztői"
   - "Milyen technikai adósságot látsz a kódban?"
3. **Tervezési mód**: Nyomj `Tab`-ot a Plan Mode-ba váltáshoz. Így az AI először tervez, és csak a jóváhagyásod után cselekszik. Éles projekteknél ezt használd.

4. **OpenSpec megismerése**: Olvasd el ennek az útmutatónak a 4. fejezetét (OpenSpec), és próbálj meg egy `wt-new` paranccsal új változást indítani.

8.3.5 Árazás (2025. február)

Csomag	Ár	Kinek való	Claude Code használat
Free	$0/hó	Kipróbálás, alkalmi használat	Nem támogatott
Pro	$20/hó	Egyéni felhasználók, kisebb projektek	Támogatott, napi limittel
Team	$25/fő/hó	Kis-közepes csapatok (max ~50 fő)	Támogatott, magasabb limittel
Enterprise	Egyedi	Nagyvállalatok	Támogatott, egyedi limitekkel, SSO, admin konzol

Fontos: A Claude Code használata a Pro csomagnál napi tokenkorlátos. Ha intenzíven dolgozol (pl. orchestrációt futtatsz több ágenssel), a napi limit hamar elfogyhat. Team vagy Enterprise csomag esetén a limitek magasabbak.

Az aktuális árakat mindig az Anthropic árazás oldalon ellenőrizd.

8.4 D. Ajánlott olvasmányok

Ha mélyebben szeretnéd megérteni az AI-vezérelt szoftverfejlesztés világát, az alábbi könyveket ajánlom. Egyik sem vár el programozási ismeretet – mind PM, vezető vagy üzleti döntéshozó szemszögéből ír.

1. Ethan Mollick: Co-Intelligence – Living and Working with AI (2024) Az AI korszak egyik legjobb bevezető könyve. Mollick a Wharton Business School professzora, és nem a technológiát, hanem az emberi oldalt állítja középpontba: hogyan gondolkozz az AI-ról, hogyan használd a mindennapi munkában, és milyen hibákat kerülj el. Gyakorlatias, közérthető, és rengeteg valós példa van benne.

2. Gene Kim, Jez Humble, Nicole Forsgren: Accelerate (2018) Adatokkal alátámasztott elemzés arról, hogy mi különbözteti meg a magas teljesítményű szoftverfejlesztői csapatokat. A CI/CD, a deploy gyakoriság és a "lead time" fogalmak innen származnak. Ha PM-ként meg akarod érteni, miért fontos az automatizáció és miért számít a fejlesztési sebesség – ez a könyv az alap.

3. Martin Fowler: Refactoring – Improving the Design of Existing Code (2018, 2. kiadás) A refaktorálás bibliája. Nem kell programozónak lenned ahhoz, hogy értsd a lényeget: a szoftvert folyamatosan karban kell tartani, mint egy házat. Ez a könyv segít a PM-nek megérteni, miért kérnek a fejlesztők "takarítási időt" és miért nem szabad azt mondani, hogy "ne refaktorálj, írj inkább új feature-öket".

4. Cal Newport: Deep Work (2016) Nem AI-könyv, de találkozik az AI korszak legfontosabb kérdésével: hogyan végezz mély munkát egy világban teli megszakításokkal. PM-ként az AI eszközök segítenek abban, hogy a "sekélyes munkát" (emailek, összefoglalók, státuszjelentések) automatizáld, és a felszabaduló időt mély gondolkodásra használd. Newport keretrendszere segít ezt tudatosan megcsinálni.

5. Marty Cagan: Inspired – How to Create Tech Products Customers Love (2018, 2. kiadás) A termékfejlesztés bibliája. Cagan a Silicon Valley legtapasztaltabb product management tanácsadója, és ez a könyv leírja, hogyan működnek a legjobb termékcsapatok. Az AI korszakban különösen releváns: ha az AI átveszi a kódolás nagy részét, a PM szerepe a termékfelfedezés (discovery) felé tolódik – és erről szól ez a könyv.